Architectuurbesluiten Soevereine Overheidscloud

Bundel ADR's, versie 0.9

1 september 2026

Inhoudsopgave

Inleiding 4
Leeswijzer 5

Overkoepelende besluiten 6
ADR-001: Keuze architectuurmodel 7
ADR-002: Soevereiniteitsambitie en toetsingskader op basis van het EU Cloud Sovereignty Framework 9
ADR-003: Keuze softwarelicenties 11

Generieke besluiten 13
ADR-G01: Datacenterstrategie: locaties, beschikbaarheidszones en georedundantie 14
ADR-G02: Multi-tenancy isolatiemodel: niveau van scheiding Worker nodes 17
ADR-G03: Tenant onboarding en lifecycle management 20
ADR-G04: Integratie met bestaande overheidsnetwerken en netwerksegmentatiemodel 23
ADR-G05: IP-adressering en adresruimtebeheer voor multi-tenant platform 24
ADR-G06: IAM-strategie: scheiding Platform-IDP en Tenant-IDP 27
ADR-G09: Loadbalancing en ingress als platform-eigen dienst voor externe service exposure 31
ADR-G10: Basisniveau rubricering: tot welk VIRBI-niveau 33
ADR-G12: Inrichting internationaal datacenter 36
ADR-G13: Hardware-selectiestrategie: multi-vendor vs single-vendor per generatie 37
ADR-G23: IPv6-modernisatie strategie uitwerking 41
ADR-G25: IPAM-tooling en integratie met bestaande overheids-IP-registraties 44
ADR-G37: DDoS-mitigatie en WAF-integratie 47
ADR-G38: Technische maatregelen voor DEP VERTROUWELIJK workloads 50
ADR-G41: Logging, monitoring en audittrail: observability-strategie 53
ADR-G49: Standaard managed diensten catalogus 54
ADR-G50: Managed database-diensten: scope en engines 57
ADR-G51: Managed message queue diensten 58
ADR-G52: Lifecycle en upgradebeleid voor managed diensten 59
ADR-G53: Secrets- en sleutelbeheer: platform-eigen KMS-architectuur 62
ADR-G54: PKI-architectuur: certificaatbeheer voor platform en tenants 65
ADR-G55: DNS-architectuur: naamgeving, resolving en tenant-scoping 68
ADR-G56: Datacenter network fabric: underlay architectuur en switch-automatisering 71
ADR-G57: Firewall architectuur segmentatie en filtering op platform- en tenant-niveau 73
ADR-G58: Backup en disaster recovery strategie 75
ADR-G59: Gebruiksgebaseerde dienstverlening (metering) 77
ADR-G60: Bootstrap proces en Git-platform voor platformbeheer 79
ADR-G61: Operatorframework: beheer van complexe managed diensten 81
ADR-G62: Non-functionele eisen: beschikbaarheid, RTO/RPO en capaciteitslimieten 83
ADR-G63: SOC-integratie en security monitoring 85
ADR-G64: Autoscaling en resource quota beheer 87
ADR-G65: Multi-vendor strategie voor hardware en netwerkhardware 89
ADR-G66: Haven/Haven+ en standaarden conformance 91
ADR-G67: Resource-hiërarchie: organisatie-, cluster- en projectmodel 93
ADR-G68: Hardware-lifecycle management: refresh, end-of-support en decommissioning 96
ADR-G69: Koppeling met publieke clouddiensten: hybride model 98
ADR-G70: CI/CD platform dienst voor tenant-workloads 100
ADR-G71: Cluster-lifecycle 102
ADR-G72: Bare-metal provisioning 104
ADR-G73: Hypervisor-strategie (no-hypervisor, KubeVirt optioneel) 106
ADR-G74: Secrets- en certificaatbeheer per cluster 108
ADR-G75: Interne IP-adressering, IPv4 intern, IPv6 extern 110

Componentspecifieke besluiten 111
ADR-C01: Bare-metal Kubernetes provisioning-strategie 112
ADR-C02: Host-besturingssysteem voor platformnodes 115
ADR-C03: Ondersteuning voor virtuele machines 117

ADR-C04: Tenantisolatie-model op compute-niveau 121
ADR-C05: Gedistribueerde storage strategie voor bare-metal Kubernetes 124
ADR-C06: Object storage voor ongestructureerde data 128
ADR-C07: Dataplane-strategie voor multi-tenant netwerkisolatie binnen bare-metal clusters 130
ADR-C08: Bare-metal loadbalancing: service-exposure en IP-advertentie 134
ADR-C10: Primaire beheerinterface en self-service model 137
ADR-C11: GPU-management op bare-metal Kubernetes 140
ADR-C12: Declaratief lifecycle management voor platformdiensten 141
ADR-C20: Control plane topologie voor multi-tenant clustermodel 145
ADR-C23: Beheermodel voor immutable nodes 149
ADR-C24: Update-strategie voor platformnodes 152
ADR-C38: Service mesh strategie: L7 verkeersbeheer, mTLS en observability per tenant 154
ADR-C43: Granulaire IP-toewijzing en advertentie per tenant 157
ADR-C55: GitOps mate van menselijke interventie in een geautomatiseerd platform 159
ADR-C60: Hardware provisioning: geïntegreerde compute- en netwerk-provisioning 162

Overige besluiten **165**
ADR-N01: Service model 166
ADR-N02: Organisatorische inrichting 168

Inleiding

Dit document bevat de architectuurbesluiten (Architectural Decision Records, ADR's) voor het ontwerp van de soevereine overheidscloud. De besluiten zijn uitgewerkt in meerdere sessies met architecten van tientallen overheidsorganisaties.

De ADR's vormen een levend geheel waar nog aan gewerkt wordt. Deze PDF is een momentopname per 1 september 2026. Omdat het werk doorloopt, kunnen er nog inconsistenties in samenhang en benaming zitten. Daarnaast worden keuzes technisch getest, en op basis van nieuwe inzichten kunnen besluiten worden aangepast. Lezers worden ook uitgenodigd om eventuele fouten en inconsistenties te helpen verbeteren. Deze kunnen gemeld worden via: NDS-cloud@minbzk.nl.

Leeswijzer

Opbouw

De ADR's zijn in deze bundel gegroepeerd op prefix.

Prefix	Betekenis
ADR-001 t/m ADR-003	Overkoepelende besluiten die alle andere ADR's bepalen
ADR-G##	Generieke besluiten, gelden ongeacht het architectuurmodel
ADR-C##	Componentspecifieke besluiten, voortvloeiend uit de gekozen aanpak
ADR-N##	Overige besluiten

Nummers zijn niet doorlopend. Een ontbrekend nummer betekent dat die ADR niet in deze momentopname is opgenomen of niet (meer) bestaat.

Status

Elke ADR doorloopt een rijpingsproces. De status staat bovenaan iedere ADR en bepaalt hoe het besluit gelezen moet worden. Alleen ADR's met status Vastgesteld zijn formeel vastgesteld. Een ADR kan bij nieuwe inzichten ook terug in status gaan.

Status	Betekenis
Voorgesteld	Architectuurvraag is geïdentificeerd, startpunt voor discussie
Brons	Onderzocht en onderbouwd, draagt bij aan het collectieve begrip, nog niet gereviewed
Zilver	Richting besloten en gereviewed, details kunnen nog wijzigen
Goud	Compleet, consistent, consensus. Gereed voor formele vaststelling
Vastgesteld	Formeel vastgesteld via governance. Wijzigingen via herzieningsprocedure

Opbouw van een ADR

Iedere ADR volgt het MADR-formaat (Markdown Architectural Decision Records) met de secties context en probleemstelling, besluit, onderzochte implementaties, technologiekeuze, motivatie, consequenties en open vragen. Afhankelijk van de status zijn nog niet alle secties ingevuld. Sessienotities bevatten informele aantekeningen uit de architectuursessies en maken geen deel uit van het besluit.

Verwijzingen naar andere ADR's zijn in deze PDF klikbaar.

Overkoepelende besluiten

ADR-001: Keuze architectuurmodel

Status	**Vastgesteld**
Laag	Overkoepelend
Maakt mogelijk	ADR-C01, ADR-C02, ADR-C03, ADR-C04, ADR-C07, ADR-C10, ADR-C12
Datum	2026-04-16

Context en probleemstelling

Het cloudplatform kan op drie manieren worden opgezet:

- als OpenStack-gebaseerd IaaS-platform (Model A),
- als geïntegreerd product van een Europese leverancier (Model B),
- of als samenstelling van losse CNCF-componenten (CNCF alternatieven zoals NeoNephos zijn ook in scopce) op bare-metal Kubernetes (Model C).

De keuze voor het architectuurmodel is bepalend voor alle vervolgbeslissingen in de stack en heeft directe gevolgen voor de leveranciersonafhankelijkheid, de integratielast, de operationele complexiteit en de mate van controle over de volledige technologiestack. Het model moet voldoen aan de Aanbestedingswet (geen vendor lock-in), het EU Cloud Sovereignty Framework (SEAL-4) en de Haven standaard.

Besluit

Losse CNCF-componenten op bare-metal Kubernetes. Maximale keuzevrijheid en leveranciersonafhankelijkheid (model C).

Motivatie

Model C biedt maximale keuzevrijheid en leveranciersonafhankelijkheid door het platform op te bouwen uit best-of-breed open source componenten. Er is geen afhankelijkheid van één leverancier of product. De actieve CNCF-community en brede industrie-adoptie bieden continuïteitsgaranties. De hoge integratielast wordt geaccepteerd als investering in volledige controle over de stack.

Consequenties

Alle vervolgkeuzes worden gemaakt op basis van losse componenten. De integratielast ligt volledig bij het platformteam. Geen single point of support; elke component heeft een eigen lifecycle. Diepgaande K8s- en infrastructure-expertise is vereist. Substantiële investering in interne expertise voor de volledige levenscyclus.

Sessienotities

De selectie vereist een substantiële investering in de opbouw van interne expertise voor de volledige levenscyclus, van initiële ontwikkeling en realisatie tot aan het onderhoud, kwetsbaarheidsaanpassingen, integratie en de continue doorontwikkeling.

ADR-002: Soevereiniteitsambitie en toetsingskader op basis van het EU Cloud Sovereignty Framework

Status	**Vastgesteld**
Laag	Overkoepelend
Afhankelijk van	ADR-001
Maakt mogelijk	ADR-G10, ADR-G38, ADR-C02, ADR-C07, ADR-C05
Datum	2026-07-07

Context en probleemstelling

Het NDS Cloud-platform wordt gebouwd als soevereine cloudinfrastructuur voor de Nederlandse overheid. De opdracht stelt expliciet dat de architectuur moet worden getoetst aan het EU Cloud Sovereignty Framework en het bijbehorende SEAL-4 assurance level. Maar wat betekent SEAL-4 concreet voor elke laag van de stack, en waar is SEAL-4 niet haalbaar?

Het EU Cloud Sovereignty Framework definieert acht soevereiniteitsdoelstellingen (SOV-1 t/m SOV-8) en vijf assurance levels (SEAL-0 t/m SEAL-4). SEAL-4 “Full Digital Sovereignty” betekent dat technologie en operatie volledig onder EU-controle staan, uitsluitend onder EU-recht vallen, en geen kritieke niet-EU afhankelijkheden hebben.

De architectuurvraag is: op welk SEAL-niveau positioneert het platform zich per soevereiniteitsdoelstelling, waar is SEAL-4 haalbaar, waar niet, en hoe wordt omgegaan met de domeinen waar SEAL-4 structureel onbereikbaar is?

Het SEAL-niveau per doelstelling bepaalt concrete technologiekeuzes: als SEAL-4 op technology sovereignty (SOV-6) wordt nagestreefd, moeten alle componenten open source zijn. Als SEAL-4 op supply chain sovereignty (SOV-5) wordt nagestreefd, zou alle hardware uit de EU moeten komen — wat op dit moment niet mogelijk is voor CPU’s, GPU’s en netwerk-ASIC’s.

Besluit

De Clouddienst hanteert SEAL-4 (Full Digital Sovereignty) als ambitieniveau voor de volledige softwarestack van de Clouddienst, met een expliciete uitzondering op SEAL-3 (Digital Resilience) voor de hardwarestack. De grens tussen de twee domeinen ligt bij de hardware-abstractielaag: alles boven die grens (besturingssysteem, Kubernetes, platformdiensten) valt onder SEAL-4, alles eronder (CPU, firmware, microcode, netwerk-ASIC’s) valt onder SEAL-3.

Dit leidt tot twee soevereiniteitsdomeinen:

Softwaredomein (SEAL-4): alle software waaruit de Clouddienst zelf bestaat is open source, volledig auditeerbaar en minimaal onder EU-controle beheerd. De Clouddienst draait in de OverheidsDataCenters. Er is geen afhankelijkheid van niet-EU partijen voor de werking van deze softwarestack. Het platformteam heeft volledige toegang tot de broncode, configuratie en operationele tooling van de Clouddienst.

Hardwaredomein (SEAL-3): CPU’s (Intel, AMD), GPU’s (NVIDIA, AMD), netwerk-ASIC’s (Broadcom, NVIDIA Spectrum) en de bijbehorende firmware worden geproduceerd door niet-EU fabrikanten. De EU heeft op

dit moment geen eigen CPU- of GPU-fabricage op productieniveau. Dit is een structurele afhankelijkheid die niet door architectuurkeuzes kan worden opgelost maar die wel moet worden beheerd via een migratiepad en supply chain resilience strategie.

Motivatie

De keuze voor twee soevereiniteitsdomeinen is een eerlijke en verdedigbare positie. Het is sterker dan SEAL-3 claimen voor alles (dat onderschat de feitelijke controle over de softwarestack) en eerlijker dan SEAL-4 claimen voor alles (dat negeert de hardware-realiteit).

De grens bij de hardware-abstractielaag is bewust gekozen. Het besturingssysteem (ADR-C02) vormt de vertrouwensgrens: alles vanaf het OS en hoger is volledig onder eigen controle. De hardware eronder is een black box waarvan de integriteit niet onafhankelijk kan worden geverifieerd firmware en microcode zijn propriëtair en niet auditeerbaar. Door deze grens expliciet te benoemen kan het platform eerlijk communiceren over wat het wel en niet garandeert.

De hardware-uitzondering is geen permanente acceptatie maar een actuele constatering met een migratiepad. Zodra EU-hardware beschikbaar komt (RISC-V processoren, Europese ASIC-fabricage via de EU Chips Act) kan het platform migreren naar SEAL-4 op de volledige stack.

Consequenties

Voor alle technologie-ADR's: elke technologiekeuze wordt getoetst aan het SEAL-niveau. Een component dat niet open source is, niet onder EU-controle kan worden beheerd, of een afhankelijkheid van een niet-EU partij introduceert, is in strijd met SEAL-4 op het softwaredomein en mag niet worden gekozen tenzij er geen open source alternatief bestaat en het expliciet als uitzondering wordt gedocumenteerd.

Voor de opdracht: de soevereiniteitsanalyse die de opdracht vereist ("identificatie van afhankelijkheden/mate van soevereiniteit in de hele stack") wordt ingevuld door elke ADR te toetsen aan de hier vastgelegde SEAL-niveaus per SOV-doelstelling. Het totaalbeeld ontstaat door de individuele toetsingen te aggregeren.

Voor certificering: het platform streeft naar certificering op SEAL-4 voor het softwaredomein. De hardware-uitzondering op SEAL-3 wordt transparant gedocumenteerd in de certificeringsaanpak. Het is aan het certificeringsorgaan om te bepalen of deze differentiatie acceptabel is.

Voor de marktplaats: het platform positioneert zich als aanbieder in de overheidsmarktplaats voor clouddiensten (domeinarchitectuur §5). De SEAL-toetsing per SOV-doelstelling is onderdeel van het toetredingsdossier.

Voor de hardware-roadmap: de SEAL-3 uitzondering is niet vrijblijvend. Er moet een actief migratiepad worden bijgehouden dat periodiek (jaarlijks) evalueert of EU-hardware-alternatieven beschikbaar en productierijp zijn geworden. Dit wordt opgenomen in de platformroadmap.

Sessienotities

> Ambitieniveau is SEAL-4 (Full Digital Sovereignty), met uitzondering voor hardware/firmware waar SEAL-3 geldt. De grens ligt bij de hardware-abstractielaag. Twee soevereiniteitsdomeinen: softwarestack (SEAL-4 hard) en hardwarestack (SEAL-3 met migratiepad).

ADR-003: Keuze softwarelicenties

Status	**Vastgesteld**
Laag	Overkoepelend
Maakt mogelijk	ADR-002
Datum	2026-07-15

Context en probleemstelling

Op basis van de Notitie verkenning heeft het Overheidsbreed Overleg Digitale Overheid (OBDO) besloten gebruik te maken van Open Source Software (OSS). De specifieke voorwaarden waaronder OSS gebruikt mag worden hangt af van de gebruikte licenties, en onderlinge compatibiliteit van de gebruikte OSS licenties van componenten/bibliotheken in de keten. Voor de Clouddienst is nadere besluitvorming nodig over de OSS licenties.

Besluit

De Clouddienst gebruikt uitsluitend OSI-approved licenties

Motivatie

De Cloud and AI Development Act (CADA) en EU Open Source Strategy zijn onderdeel van het EU Sovereign Tech Package. Het kabinet heeft daarop gereageerd in het Fiche: Cloud & AI Development Act8 en het Fiche: Mededeling Europese Tech Soevereiniteit, inclusief Open Source Strategie.

“Het kabinet verwelkomt de Open Source Strategie en ondersteunt maatregelen die bijdragen aan open standaarden, interoperabiliteit, digitale gemeenschapsgoederen en de ontwikkeling van Europese open source alternatieven. Daarbij zal het kabinet verduidelijking vragen over de reikwijdte van het voorgestelde open-source-first-principe.”

Ook Nederlandse Marktpartijen roepen op tot Open Source. Het kabinet heeft daar op 3 juli jl. op gereageerd en schrijft “het platform is zoveel als mogelijk gebaseerd op open source software.”. Deze ADR borgt besluitvorming m.b.t. te gebruiken Open Source Licenties in de context van de Clouddienst.

Om SEAL4 te kunnen halen schrijft het EU Cloud Sovereignty Framework het volgende voor, vertaald naar de Nederlandse context.

> SOV-4: “volledige beschikbaarheid van technische documentatie, broncode, en operationeel digitaal vakmanschap borgen lange-termijn autonomie”
>
> SOV-5: “oorsprong van software: waar en door wie de software architectuur en implementatie is bedacht, de locatie en jurisdictie waar de software governance (lifecycle management) is geborgd incl. packaging, distributie, en updates”, en “mate van afhankelijkheid van niet-EU leveranciers, faciliteiten, of proprietary technologie”, en “zichtbaarheid in de leverancier en leveranciersketen, incl. auditeerbaarheid”.

> SOV-6: “of de software toegankelijk is onder open licenties, met rechten om te auditen, aanpassen, herdistribueren, om transparantie en aanpasbaarheid te borgen”.

Momenteel is er echter nog geen vastgesteld Overheidsbreed Open Source beleid. Wel staat in de Handleiding Wet hergebruik van overheidsinformatie (Who)12: “…dienen voorwaarden zo veel mogelijk gesteld te worden via standaardlicenties…”. Het Overheidsbreed standpunt voor de inzet van generatieve AI adopteert de Open Source Definitie van het Open Source Initiative.

Voor de Clouddienst wordt ook de OSI-definitie gehanteerd met aanvullend criterium dat alle software OSI-approved is, niet (slechts) OSI-compatible, zodat kan worden voldaan aan SOV-4, SOV-5, en SOV-6. Hierbij citeren we uit het BNC-fiche: “Het kabinet onderschrijft het belang van open source als fundamenteel middel ter versterking van technologische soevereiniteit”.

Consequenties

Er zijn Open Source softwarelicentiespecialisten nodig. De Software Bill of Materials (SBOM) dient verrijkt te worden met een License Bill of Materials waarin consistentie van alle gebruikte licenties transparant traceerbaar is opdat licentiecompliance geborgd is.

Generieke besluiten

ADR-G01: Datacenterstrategie: locaties, beschikbaarheidszones en georedundantie

Status	Goud
Laag	Datacenter
Afhankelijk van	ADR-001, ADR-002
Maakt mogelijk	ADR-G12, ADR-C05, ADR-G04
Datum	2026-08-11

Context en probleemstelling

De Clouddienst gebruikt fysieke infrastructuur: het platform draait op servers in gebouwen met stroom, koeling en netwerkaansluitingen. De keuze waar die gebouwen staan, hoeveel het er zijn, en hoe ze zich tot elkaar verhouden is de meest fundamentele beslissing in de hele architectuur. Alles bouwt voort op deze keuze: storage-replicatie, cluster-topologie en failover-strategie.

De architectuurvraag is: hoeveel datacenterlocaties zijn nodig, hoe zijn deze geografisch verspreid, en welk redundantiemodel wordt gehanteerd om de beschikbaarheids- en continuïteitseisen van het platform te waarborgen?

De eisen die de keuze inperken:

- **SEAL-4 soevereiniteit (ADR-002)**: alle data wordt verwerkt en opgeslagen in Nederland (of binnen de EU/EER voor specifieke use cases). De fysieke infrastructuur en de control plane bevinden zich volledig in de EU.
- **C5-criterium georedundantie**: de BSI C5-standaard vereist dat locaties voldoende ver uit elkaar liggen om georedundantie te bereiken. Bij twee gelijktijdige locatieuitvallen moet minimaal één derde locatie beschikbaar blijven. De functionaliteit van de redundantie moet jaarlijks worden getest.
- **BIO en NIS2**: de Cyberbeveiligingswet (NIS2-implementatie) vereist security-by-design en continuïteitsmaatregelen voor essentiële diensten. De BIO stelt eisen aan beschikbaarheid per classificatieniveau.
- **DEP VERTROUWELIJK (ADR-G10)**: workloads met dit classificatieniveau stellen aanvullende eisen aan fysieke beveiliging, toegangscontrole en scheiding van datacenterruimtes.
- **Opdracht**: de opdracht specificeert dat de datacenterstrategie minimaal moet behandelen: redundantie van de datacenters, kwaliteitsniveau, aantal locaties, dienstverleningsmodel, eisen aan datacenters in het buitenland (EU en non-EU), eisen aan stroomtoevoer en koeling.
- **Domeinarchitectuur**: de domeinarchitectuur benoemt “availability zones” als begrip en verwijst naar het Sovereign Cloud Stack (SCS) en GovStack model voor de definitie hiervan.
- **Classificatie Datacentra** Elke fysieke datacenterfaciliteit die als AZ voor de Clouddienst wordt ingezet, moet minimaal aantoonbaar voldoen aan ANSI/TIA-942-C Rated-3 (Concurrently Maintainable), bij voorkeur door een geldige onafhankelijke TIA-942-certificering. De TIA-942-C Rated-3-eis heeft betrekking op de fysieke datacenterinfrastructuur en niet op de beschikbaarheid van het Kubernetes-platform of de regio.

Daarnaast zijn er praktische randvoorwaarden:

- De Clouddienst wordt gehuisvest in de OverheidsDataCenters (ODC’s). Er zijn op dit moment meerdere ODC’s operationeel, verspreid over Nederland.

- De netwerkverbindingen tussen locaties verlopen via Diginetwerk en het Rijksoverheidsnetwerk (RON). De beschikbare bandbreedte en latency tussen locaties beïnvloeden het replicatiemodel.
- Een internationaal datacenter kan relevant zijn voor diplomatieke en defensie-workloads die buiten Nederland moeten worden verwerkt.

De definitie van Availability zone is als volgt “Afzonderlijke locatie binnen een datacenter-regio die is ontworpen om geïsoleerd te zijn van storingen in andere zones, zodat een combinatie van meerdere zones verhoogde beschikbaarheid en fouttolerantie biedt.”

Besluit

De Clouddienst hanteert een **regio/availability zone-model**. Het platform bestaat uit **drie regio’s**, elk met **drie availability zones (AZs)**. Een regio is een geografisch gebied, vergelijkbaar met een provincie of grootstedelijk gebied. Elke AZ binnen een regio moeten zodanig fysiek onafhankelijk zijn dat een storing of gepland onderhoud aan één AZ niet leidt tot het verlies van een andere AZ.

Deze onafhankelijkheid omvat minimaal: stroomvoorziening en elektrische distributie, koeling, telecom- en netwerkpaden, brandcompartimentering en brandveiligheid, fysieke toegangsvoorzieningen, relevante andere common-mode failure-risico’s. Een AZ kan bestaan uit één of meerdere fysieke datacenterruimtes binnen een ODC, of uit een volledig afzonderlijk ODC.

Alle AZs binnen een regio zijn verbonden via minimaal twee fysiek diverse netwerkpaden met lage latency. De netwerkarchitectuur moet het verlies van één netwerkpad kunnen opvangen zonder verlies van de vereiste platformfunctionaliteit. De geografische afstand tussen AZs wordt bepaald op basis van latency, replicatievereisten, regionale risico’s en common-mode failure-risico’s.

Alle drie de regio’s zijn actief. Tenants kunnen per workload kiezen in welke regio of regio’s zij willen draaien, afhankelijk van de gestelde eisen. Het beschikbaarheidsmodel kent twee niveaus binnen een regio:

Één AZ uitgevallen: de Clouddienst is in **degraded mode** binnen die regio. Quorum is behouden (2 van 3 AZs beschikbaar). Capaciteit is verminderd, workloads worden herplaatst op de twee resterende AZs.

Twee AZs uitgevallen: de regio is **niet operationeel**. Kubernetes-quorum is verbroken: etcd accepteert geen schrijfoperaties meer en de control plane kan geen wijzigingen doorvoeren. Workloads die over meerdere regio’s zijn gespreid blijven beschikbaar vanuit de andere regio’s.

Een DR-locatie buiten Nederland is een apart besluit (ADR-G12).

Motivatie

Quorum per regio: etcd, Ceph (ADR-C05) en het management-cluster (ADR-C20) vereisen een oneven aantal replica’s voor consensus. Drie AZs per regio maakt een 2-van-3 quorum mogelijk: bij uitval van één AZ draaien de overige twee door zonder menselijke interventie.

Capaciteit: bij drie gelijke AZs hoeft elke AZ slechts 50% overcapaciteit te hebben om het verlies van één AZ op te vangen.

Drie actieve regio’s: workloads die over meerdere regio’s worden gespreid blijven beschikbaar bij een volledige regio-uitval. Tenants met de hoogste beschikbaarheidseisen kunnen hun workloads over twee of drie regio’s spreiden.

Consequenties

Netwerk binnen een regio: AZs zijn verbonden via redundante metro-verbindingen met RTT <10ms voor synchrone replicatie. De beschikbare capaciteit moet worden gevalideerd (ADR-G04).

Netwerk tussen regio's: inter-regio verkeer verloopt asynchroon. Bandbreedte moet voldoende zijn voor de vastgestelde RPO per opslagklasse.

Storage: de Ceph CRUSH-map (ADR-C05) moet zone-aware zijn binnen een regio. Cross-regio replicatie verloopt asynchroon via RBD mirroring.

Cluster-topologie: het management-cluster (ADR-C20) wordt per regio over alle drie de AZs gespreid. Tenant-clusters kunnen binnen een regio over twee of drie AZs worden gespreid, en optioneel over meerdere regio's.

Capaciteitsplanning: Elke AZ wordt ontworpen op minimaal 150% van de normale eigen belasting, zodat na verlies van één AZ voldoende capaciteit beschikbaar is om de workloads over de twee resterende AZs te verdelen. Deze 150%-eis is een platformcapaciteitseis en staat los van de TIA-942-C Rated-3-classificatie.

Fysieke beveiliging: elke AZ moet voldoen aan de eisen voor DEP VERTROUWELIJK (ADR-G10).

Verificatie en bewijsvoering: Per fysieke datacenterfaciliteit wordt aantoonbaar gemaakt dat: 1. de faciliteit minimaal voldoet aan ANSI/TIA-942-C Rated-3, 2. redundante elektrische en mechanische voorzieningen aanwezig zijn, 3. de vereiste onafhankelijke distributiepaden aanwezig zijn, 4. onderhoud kan worden uitgevoerd zonder onderbreking van de ICT-belasting, 5. de fysieke onafhankelijkheid tussen AZs is vastgesteld, 6. relevante common-mode failure-risico's zijn geïdentificeerd en gemitigeerd, 7. voldoende capaciteit beschikbaar is bij verlies van één AZ. Een onafhankelijke TIA-942-certificering heeft hierbij de voorkeur.

Sessienotities

> Minimaal drie beschikbaarheidszones, fysiek gescheiden ODC-locaties. Georedundantie conform C5. Internationaal datacenter als apart besluit (ADR-G12). Design voor verlies van één zone zonder impact op beschikbaarheid.

ADR-G02: Multi-tenancy isolatiemodel: niveau van scheiding Worker nodes

Status	**Vastgesteld**
Laag	Multi-tenancy
Afhankelijk van	ADR-002
Maakt mogelijk	ADR-C04, ADR-G03, ADR-G67
Datum	2026-07-29

Context en probleemstelling

Het NDS Cloud-platform bedient meerdere overheidsorganisaties tegelijkertijd op gedeelde fysieke infrastructuur. De organisaties verschillen sterk in grootte, risicoprofiel en classificatieniveau. Het isolatiemodel bepaalt fundamenteel hoe sterk de scheiding is tussen tenants op compute-, netwerk- en storageniveau, en daarmee de veiligheidspositie, resource-efficiëntie en operationele complexiteit van het platform.

Een te zwak model voldoet niet aan de BIO2- en EUCS SEAL-4-eisen voor werklasten met een hoger risicoprofiel. Een te rigide model leidt tot prohibitief hoge kosten en te hoge minimale tenant-omvang. De keuze moet herleidbaar zijn naar een aantoonbaar isolatieniveau dat voor álle overheidslagen bruikbaar is.

Onderzochte opties

Optie 1: Dedicated fysieke clusters per tenant

Elke tenant krijgt een of meerdere eigen Kubernetes-cluster op dedicated fysieke nodes. De control plane (API server, etcd, controller-manager, scheduler) draait als Hosted Control Plane op het management-cluster; de worker-nodes zijn exclusief voor de tenant.

Voordelen: sterkste isolatievorm geen gedeelde kernel, geen gedeelde API server, geen gedeeld etcd. Elke worker-node is aantoonbaar aan precies één tenant toe te wijzen. Onafhankelijke cluster-lifecycles per tenant. Voldoet aan isolatie-eisen.

Nadelen: minimaal drie fysieke worker-nodes per tenant; hogere resource-overhead voor kleine tenants.

Optie 2: Gedeelde clusters met namespace-isolatie

Alle tenants draaien op één gedeeld Kubernetes-cluster, gescheiden via namespaces en RBAC.

Voordelen: hoogste resource-efficiëntie; snelle tenant-onboarding; één cluster te beheren.

Nadelen: gedeelde kernel een container breakout raakt andere tenants; noisy-neighbor problemen op CPU en geheugen; complex beheer van RBAC en NetworkPolicies; voldoet niet aan EUCS SEAL-4 voor werklasten met hoger classificatieniveau. Niet acceptabel voor het beoogde DEP VERTROUWELIJK-basisniveau (ADR-G10).

Optie 3: Virtuele clusters (vCluster) op gedeelde fysieke clusters

Elke tenant krijgt een eigen virtuele API server en control plane via vCluster, draaiend op gedeelde fysieke nodes.

Voordelen: tenant heeft eigen API server en control plane; betere isolatie dan namespaces; lage resource-overhead.

Nadelen: kernel blijft gedeeld dit maakt het isolatieniveau is onvoldoende voor hogere risicoklassen; het sync-mechanisme van vCluster introduceert een extra aanvalsoppervlak; commerciële leveranciersafhankelijkheid (Loft Labs). Niet SEAL-4-compliant.

Optie 4: Hybride — dedicated clusters voor gevoelige tenants, gedeeld voor overige

Tenants met een hoger risicoprofiel krijgen een dedicated cluster; tenants met lagere classificatie draaien op een gedeeld cluster.

Voordelen: resource-efficiënt voor laag-risico-tenants; sterke isolatie waar nodig.

Nadelen: twee parallelle operationele modellen; classificatiecomplexiteit en grensgevallen bij wijziging van risicoprofiel; inconsistente beveiligingspositie maakt platform-brede audits en certificering moeilijker.

Besluit

Dedicated fysieke clusters per tenant.

Elke tenantorganisatie krijgt een eigen Kubernetes-cluster op dedicated fysieke worker-nodes. Dit levert volledige compute-isolatie: geen gedeelde kernel, geen gedeelde control plane. Elke worker-node is aantoonbaar toe te wijzen aan één tenant.

Hoe de control-planes worden ingericht is vastgesteld in C20

Motivatie

De keuze wordt gedreven door drie harde eisen die de overige opties uitsluiten of ernstig beperken.

Ten eerste: de gestelde betrouwbaarheidsesien impliceren nu dat de isolatie aantoonbaar en auditeerbaar is op elke laag. Gedeelde kernels (Optie 2 en 3) maken dit onmogelijk omdat een kernel breakout is niet aantoonbaar afwezig via softwareconfiguratie alleen.

Ten tweede: het basisclassificatieniveau DEP VERTROUWELIJK (ADR-G10) stelt minimumeisen aan scheiding die in een gedeeld-cluster-model niet op afdoende wijze zijn te borgen via RBAC en NetworkPolicies.

Ten derde: als clusterbeheerplatform (ADR-C20) maakt de operationele last van dedicated clusters beheersbaar.

De hybride aanpak (Optie 4) is expliciet afgewezen omdat twee parallelle operationele modellen de platform-brede certificering en auditeerbaarheid structureel bemoeilijken. Consistentie van het isolatiemodel is voor een overheidsbrede dienst zwaarder dan de resource-besparing bij laag-risico-tenants.

Consequenties

De minimale tenant-omvang is drie fysieke worker-nodes. Dit is een harde ondergrens die bij tenant-onboarding (ADR-G03) als drempeleis wordt gecommuniceerd.

Tenant-onboarding is gelijk aan het provisioneren van een cluster. De onboarding-flow (ADR-G03) moet hierop worden ingericht.

De organisatiehiërarchie hoe tenantorganisaties worden gemapt op clusters, projecten en namespaces wordt uitgewerkt in een apart besluit (ADR-G67).

Netwerksegmentatie tussen tenant-clusters moet op fysiek netwerkniveau worden afgedwongen, niet alleen via software-defined policies. Dit is een eis aan de underlay-architectuur (ADR-G56).

ADR-G03: Tenant onboarding en lifecycle management

Status	**Vastgesteld**
Laag	Multi-tenancy
Afhankelijk van	ADR-G02, ADR-G06, ADR-G67
Maakt mogelijk	
Datum	2026-07-07

Context en probleemstelling

Het platform bedient meerdere overheidsorganisaties. Elke organisatie moet worden aangemeld, voorzien van quota en toegangsrechten, en uiteindelijk ook weer worden verwijderd of overgezet — op een herhaalbare, auditeerbare en veilige manier. De beslissing om een nieuwe organisatie toe te laten mag de technische onboarding niet blokkeren met weken aan handmatig werk; omgekeerd mag technische versnelling niet leiden tot onzorgvuldige besluitvorming over toegang tot overheidsinfrastructuur.

De architectuurvraag is: hoe wordt de complete onboarding-flow ingericht, van toelating tot werkende tenant-omgeving, en hoe verloopt het bijhouden en beëindigen van de relatie?

Deelvragen:

- Wie mag een tenant aanmelden? Hoe wordt die bevoegdheid gevalideerd?
- Welke identiteitsbron bepaalt wie tot de tenant hoort?
- Hoe diep gaat de delegatie? Beheert het platformteam alles, of kan een tenant-admin zelf gebruikers en sub-projecten beheren?
- Hoe wordt lifecycle-informatie (quota-wijzigingen, contactpersonen, kostencodes) bijgewerkt?
- Hoe wordt een tenant beëindigd zonder data-verlies bij gerechtvaardigde claims of auditverplichtingen?

Eisen die de keuze inperken:

- **onboarding moet binnen 1 dag kunnen** (uitgangspunt werkgroep).
- **Auditeerbaarheid**: elke tenant-lifecycle-actie moet traceerbaar zijn (NIS2, BIO).
- **Federatie**: overheden hebben eigen identiteitssystemen; het platform mag deze niet overnemen maar moet eraan federeren.
- **Minimale handmatige tussenkomst**: op termijn tientallen tot honderden tenants maakt volledig handmatig onboarding onhoudbaar.

Besluit

Het platform gebruikt een **federatief onboarding-model met een centraal overheidsidentiteitsregister als trust-anker en gedelegeerd tenantbeheer**.

Het onboarding-proces kent twee fasen:

1. **Bestuurlijke toelating**: een geautoriseerde vertegenwoordiger van de overheidsorganisatie wordt bekend gemaakt bij het platform, gekoppeld aan de identiteit zoals geregistreerd in een centraal overheidsidentiteitsregister. Deze rol kan binnen de organisatie verder delegeren.

2. **Operationele onboarding (doorlopend)**: gedelegeerde tenant-admins kunnen binnen hun eigen tenant sub-projecten, gebruikers en quota beheren via self-service (ADR-C10).

Onderzochte implementaties

Federatief met centraal identiteitsregister (gekozen): Het platform vertrouwt op het centrale register van overheidsorganisaties (https://organisaties.overheid.nl/), en een centraal overheidsidentiteitsregister voor vertegenwoodigers ervan. Bijvoorbeeld een uitbreiding van de bestuurlijke driehoek-registratie uit Besluit Sturing Digitale Overheid als bron-van-waarheid voor welke personen namens welke organisatie bestuurlijk bevoegd zijn. Technische implementatie via federatie (ADR-G06, minimaal SAML/OIDC, bij voorkeur OAuth2). Voordeel: geen nieuwe identiteitsbron, sluit aan op bestaande overheidsgovernance. Nadeel: afhankelijk van de kwaliteit van het centrale register.

Platform-eigen identiteit met handmatige validatie: Elke tenant krijgt een door het platformteam toegekende identiteit. Maximaal controle, maar schaalt slecht en dupliceert bestaande identiteitsadministratie.

Volledig self-service met externe identity providers: Elke organisatie brengt zijn eigen identity provider mee (zoals veel SaaS-platforms doen). Te open voor overheidsgebruik: geen centrale toets op bestuurlijke bevoegdheid.

Hybride model (gekozen combinatie): Optie 4 (federatief centraal) voor de eerste autorisatie op organisatieniveau, Optie 1 (platform-eigen met validatie) als fallback voor organisaties die nog niet in het centrale register staan. De centrale lijn is leidend; de fallback is tijdelijk.

Technologiekeuze

De keuze is procesmatig en organisatorisch; de technologische invulling volgt uit ADR-G06 (IAM-strategie) en ADR-G67 (resource-hiërarchie).

Richting voor technische invulling: federatie met een centraal overheids-IDP, resource-hiërarchie via Kubernetes CRD's (tenant, sub-project, namespace), self-service via platform-API (ADR-C10).

Motivatie

De federatieve aanpak respecteert dat overheidsorganisaties autonome entiteiten zijn met eigen identiteitsadministratie. Het centrale register als trust-anker voorkomt dat het platform zelf een bestuurlijke scheidsrechter moet worden tussen organisaties — dat is een bestuurlijke taak die elders belegd moet zijn.

Gedelegeerd tenantbeheer is de enige schaalbare werkwijze: het platformteam kan geen centrale goedkeurder worden voor elke gebruiker en elk sub-project in honderden tenants.

Consequenties

- **Afhankelijkheid van een centraal overheidsidentiteitsregister** dat nog niet in zijn huidige vorm bestaat voor dit doel. Dit moet met BZK / PGDI worden afgestemd. Totdat dit register bestaat, wordt een tijdelijke fallback gebruikt (handmatige validatie door platformteam).

- **Onboarding-SLA van 1 dag** stelt eisen aan automatisering: na bestuurlijke toelating moet de technische provisioning volledig geautomatiseerd zijn (tenant-namespace, IAM-koppeling, quota-toekenning, billing-code, KMS-namespace — zie ook ADR-G53).
- **Offboarding-procedure**: een tenant-beëindiging moet data-retention (7-jaar administratieplicht) en archivering regelen. Dit is geen eenvoudige 'delete'-operatie.
- **Tenant-admin delegatie** vereist een duidelijk RBAC-model (ADR-G67) dat de hiërarchie organisatie → sub-project → gebruiker respecteert.
- **Documentatie en onboarding-kit**: elke nieuwe tenant-admin krijgt een gestandaardiseerde onboarding-kit met documentatie, voorbeeld-pipelines en contactpunten bij het platformteam.
- **Contactpersoon-beheer**: beveiligingsmeldingen, incidenten en wijzigingscommunicatie vereisen actuele contactpersonen per tenant. Dit moet lifecycle-beheerd zijn.

Sessienotities

Onboarding moet kunnen in 1 dag (bestuurlijk). Optie 4 (federatief centraal) met toevoeging van ennoblement-verhaal. Optie 1 als fallback voor eerste autorisatie op organisatieniveau.

ADR-G04: Integratie met bestaande overheidsnetwerken en netwerksegmentatiemodel

Status	Goud
Laag	Networking
Datum	2026-03-04

Context en probleemstelling

Het platform moet aansluiten op Diginetwerk, NAFIN en het Rijksoverheidsnetwerk (RON). Tegelijkertijd moet sterke netwerksegmentatie worden geboden tussen tenants, beheerverkeer en classificatieniveaus. Het gaat hier echt alleen om netwerken van en naar de Cloud, niet over netwerken die alleen binnen de soevereine overheidscloud beschikbaar zijn.

Besluit

Brede integratie met alle bestaande overheidsnetwerken inclusief VPN-koppelingen. Netwerken moeten gebridged kunnen worden.

Motivatie

Dit moet allemaal kunnen. VPN’s moeten ook allemaal overal kunnen. Koppelen met heel veel externe partners.

Consequenties

De netwerklaag wordt complex. Verdere uitwerking nodig per koppelvariant.

ADR-G05: IP-adressering en adresruimtebeheer voor multi-tenant platform

Status	Goud
Laag	Networking
Maakt mogelijk	ADR-G23, ADR-G25
Datum	2026-07-07

Context en probleemstelling

Elke tenant op het platform heeft IP-adressen nodig voor workloads, services, loadbalancers en egress. Op een multi-tenant bare-metal platform moeten die IP's conflictvrij worden toegewezen, uniek zijn over ODC-locaties heen, en uitwisselbaar zijn met overheidsnetwerken (Diginetwerk, NAFIN) die ook hun eigen adresruimte hanteren.

Zonder centraal adresruimtebeheer ontstaan overlappende IP-ranges tussen tenants, dubbele administraties en conflicten bij koppeling met externe netwerken. Op platformschaal (tienduizenden servers, honderden tenants) is dit niet handmatig te beheren.

De architectuurvraag is: welk adresseringsprotocol is primair, hoe wordt de adresruimte ingedeeld en wie beheert die indeling?

Eisen die de keuze inperken:

- **Overheidsbeleid IPv6-first** ligt vast in onder meer NORA en Forum Standaardisatie ("pas toe of leg uit"). IPv6 is geen optie maar een uitgangspunt. Wij lezen het overheids-IPv6-beleid zo dat het bindt aan extern bereikbare diensten, niet aan interne infra.
- **Multi-tenancy**: elke tenant moet zijn eigen conflictvrije adresruimte kunnen gebruiken.
- **Integratie overheidsnetwerken** (ADR-G04): het platform moet uitwisselbaar zijn met Diginetwerk en NAFIN, die dual-stack zijn.
- **Legacy-compatibiliteit**: niet alle bestaande overheidssystemen ondersteunen IPv6; dual-stack blijft nodig zolang deze legacy bestaat.

Besluit

IPv6 is het primaire adresseringsprotocol voor het externe/ingress-vlak. IPv4 wordt dual-stack ondersteund als terugvaloptie voor integratie met legacy-systemen.

De interne ip adressen, voor de pod- en service-CIDR's, blijven IPv4. Tenant-workloads die intern IPv6 nodig hebben kunnen dat per cluster opt-in krijgen.

De adresruimte-indeling volgt een hiërarchische structuur: ODC-niveau → cluster-niveau → tenant-niveau → namespace/service-niveau. Elke tenant krijgt een eigen, niet-overlappende prefix toegewezen.

Onderzochte implementaties

IPv6-only (ideaal, niet haalbaar op korte termijn): Alleen IPv6. Meest schaalbaar, minste adresruimtebeheerlast. Blokkeert echter integratie met elk systeem dat nog niet IPv6 ondersteunt — dat is een groot deel van het bestaande overheidslandschap.

Dual-stack IPv6-primair: IPv6 als native protocol voor alle platform-interne communicatie en als voorkeur voor nieuwe tenant-workloads. IPv4 beschikbaar als aanvulling voor externe connecties die IPv6 niet ondersteunen. Vereist dat alle componenten dual-stack-capable zijn.

IPv4-primair met IPv6 als aanvulling (gekozen): Omgekeerde keuze, niet in lijn met overheidsbeleid. Zou IPv6-adoptie verder vertragen.

NAT64/DNS64 voor IPv6-only clusters: Intern IPv6-only, extern verkeer naar IPv4-endpoints vertalen via NAT64. Verkleint dual-stack complexiteit intern, maar introduceert NAT-gateway als operationeel aandachtspunt. Optie voor later, niet direct.

Technologiekeuze

Protocol-keuze is vastgelegd (IPv4-primair met IPv6 als aanvulling). Concrete tooling voor adresruimtebeheer is uitgewerkt in ADR-G25 (IPAM-tooling) en toekomstige modernisering in de IPv6-strategie in ADR-G23.

Voor adresruimte-toewijzing:

- **Platform-prefix**: één ULA-prefix (fc00::/7) of een PI-prefix uit RIPE-allocatie voor de platform-interne adresruimte.
- **Per-tenant prefix**: /48 per tenant uit de platform-prefix, wat ruimte biedt voor 65.536 subnetten per tenant (ruim voldoende).
- **Per-namespace/service**: automatisch toegewezen uit de tenant-prefix door CNI (Cilium, ADR-C07).

Motivatie

De onderliggende netwerk fabric maakt gebruik van IPv6 link-local voor BGP peering, dus de infrastructuur zelf is IPv6-aware. Dual-stack is beschouwd, maar de operationele overhead voor het duale beheer behoeft niet de voorkeur. De benodigde IPv6 support voor compliance is geregeld voor externe services (ingress, load balancing).

Centraal adresruimtebeheer voorkomt dat tenants onderling conflicterende IP-ranges kiezen. Door de prefix hiërarchisch te delegeren kunnen tenants binnen hun eigen ip-prefixes vrij werken zonder dat het platform elke subnet-toewijzing moet goedkeuren.

Consequenties

- **Legacy-systemen**: overheidssystemen die alleen IPv4 ondersteunen, vereisen IPv4-connectiviteit voor de bereikbare services. Hiermee zijn er geen blockers voor onboarding ervan.
- **Adresruimte-allocatie vereist een bronhouder** (platformteam) die per nieuwe tenant een prefix uitdeelt en dit registreert in IPAM (ADR-G25).
- **Koppeling met overheidsnetwerken**: Diginetwerk en NAFIN IP-capaciteit en -beleid moeten worden afgestemd (ADR-G04).

- **Documentatie en standaarden**: tenant-documentatie hoeft niet IPv6-eerst zijn. Dit zou een cultuurwijziging zijn geweest voor teams die jarenlang IPv4-first hebben gewerkt.

ADR-G06: IAM-strategie: scheiding Platform-IDP en Tenant-IDP

Status	**Vastgesteld**
Laag	Perifere diensten
Afhankelijk van	ADR-001, ADR-002, ADR-G02
Maakt mogelijk	ADR-G03, ADR-G67
Datum	2026-07-07

Context en probleemstelling

Het platform bedient twee fundamenteel verschillende groepen: het platformteam dat de infrastructuur beheert, en tenants (overheidsorganisaties) die hun eigen workloads en applicaties draaien. Beide groepen hebben identiteiten nodig om met het platform te interacteren — maar hun identiteitsbehoeften zijn verschillend van aard en mogen nooit worden vermengd.

De kernvraag is: hoe worden identiteiten georganiseerd over de verschillende lagen van het platform, zodanig dat platformbeheer volledig gescheiden blijft van de tenantomgeving, en de tenant volledige controle houdt over zijn eigen identiteitsdomein?

Een gedeelde IDP voor zowel platformbeheer als tenants is een onacceptabel veiligheidsrisico: een compromittering op platformniveau zou directe toegang kunnen geven tot tenantomgevingen, en omgekeerd. Dit is geen afweging maar een harde architectuurgrens.

Aanvullend principe uit de ODCN-praktijk: de scheiding tussen IDP (authenticatie) en access management (autorisatie) is een bewezen aanpak die het platform overneemt. De IDP bewijst identiteit; het access management besluit wat die identiteit mag doen. Deze verantwoordelijkheden worden nooit in één component samengebracht.

De eisen:

- **SEAL-4**: de Platform-IDP mag geen afhankelijkheid hebben van niet-EU-partijen.
- **Scheiding van verantwoordelijkheden**: platformbeheer heeft toegang tot infrastructuur (servers, Kubernetes-clusters) maar nooit tot de applicatielaag van tenants.
- **Tenant-autonomie**: de tenant voert zelf access management uit op de eigen omgeving; het platform stelt alleen aansluitvoorwaarden.
- **Maximale compatibiliteit**: de federatieve koppeling voor tenants moet werken voor de breedte van de overheid, inclusief organisaties die via SSC-ICT of andere shared service centra zijn aangesloten.

Besluit

Het platform hanteert **vijf gescheiden identiteitslagen**. De Platform-IDP en de Tenant-IDP zijn strikt gescheiden en mogen nooit worden samengevoegd.

De vijf identiteitslagen

Laag 1 — Applicatielaag (applicatie-eigen IDP) De identiteiten van eindgebruikers van applicaties die op het platform draaien. Dit is de verantwoordelijkheid van de tenant-applicatie zelf. Het platform bemoeit zich hier niet mee. Hier kan sprake zijn van DigiD, eHerkenning, of een eigen IDP van de overheidsorganisatie.

Laag 2 — Tenantomgeving (federatief koppelvlak) De identiteiten van tenantbeheerders: medewerkers van de overheidsorganisatie die binnen de eigen tenant-cluster resources aanmaken en beheren. Het platform biedt hier geen eigen IDP maar een **gestandaardiseerd federatief koppelvlak**: minimaal SAML/OIDC, bij voorkeur OAuth2. Zo'n koppelvlak wordt bijvoorbeeld aangeboden als whitelabel vanuit ODCN. Een tenant federeert hierop — bijvoorbeeld via SSC-ICT, die op hun beurt delegeren naar hun klantorganisaties. Het platform stelt aansluitvoorwaarden maar controleert de interne identiteitsadministratie van de tenant niet.

Laag 3 — Platform-IDP De identiteiten van platformbeheerders: medewerkers van NDS die de infrastructuur, Kubernetes-clusters en platformdiensten beheren. Dit is een eigen, platform-beheerde IDP (bijv. soeverein alternatief voor vCloud). Deze IDP heeft geen enkele koppeling met laag 2 of lager. Een platformbeheerder die inlogt via de Platform-IDP heeft uitsluitend toegang tot de platform-infrastructuurlaag, niet tot tenant-workloads of -data.

Laag 4 — Periferie-IDP De identiteiten voor alles wat nodig is om het platform aan te bieden maar niet tot de kerninfrastructuur behoort: monitoring-systemen, vendor-portals, leverancierscommunicatie, beheertools aan de buitenrand. Gescheiden van de Platform-IDP om de blast radius bij een compromittering te beperken.

Laag 5 — Out-of-band-IDP De identiteiten voor fysieke en netwerkinfrastructuur: toegang tot datacenters, OOB-beheernetwerken (IPMI/iDRAC), netwerkinfrastructuur en out-of-band management. Dit is het meest gevoelige toegangsdomein en heeft de sterkste isolatie. Toegang vereist aanvullende authenticatiefactoren (hardware tokens, smartcards).

Harde grenzen

De Platform-IDP (laag 3) en de Tenant-IDP (laag 2) **mogen nooit dezelfde zijn**. Er is geen uitzondering op deze regel.

Een tenant die geen eigen IDP heeft, kan het platform niet gebruiken. Het platform levert geen IDP-dienst aan tenants. Dit is een bewuste keuze: het platform is infrastructuur, geen identity provider.

Onderzochte implementaties

Eén gedeelde IDP (afgevallen): één centrale IDP voor zowel platformbeheer als tenants. Afgevallen omdat een compromittering op platformniveau directe impact zou hebben op alle tenants. Onacceptabel voor een platform dat DEP VERTROUWELIJK workloads host.

Gov-Connext (federatief, overheids-eigen): een soevereine overheidsoplossing voor federatieve identiteit. Relevant als potentieel koppelvlak voor laag 2 (tenantomgeving), in combinatie met SSO-Rijk. Nader te evalueren op volwassenheid en aansluitcondities.

SSO-Rijk (gekozen als laag-2-koppelvlak): het bestaande rijksbrede SSO-platform. Biedt SAML/OIDC-federatie met een groot deel van de overheidsbrede identiteitsinfrastructuur. Whitelabel-aanbod vanuit ODCN maakt het mogelijk dit aan te bieden als platform-koppelvlak zonder dat de tenant direct met SSO-Rijk hoeft te integreren.

ODCN IDP/access management-scheiding (overgenomen als principe): ODCN hanteert al een strikte scheiding tussen IDP (wie ben je?) en access management (wat mag je?). Dit principe wordt integraal overgenomen in de NDS Cloud-architectuur.

Technologiekeuze

De laag-2-federatie kan verlopen via **SSO-Rijk** (SAML/OIDC/OAuth2), e.g. whitelabel aangeboden via ODCN, voor Rijksoverheden. Dat zal of uitgebreid moeten worden voor lagere overheden, of federatie met eigen IDP van de afnemer. Aangezien dat hun eigen verantwoordelijkheid is, wordt elke IDP (ook US Big Tech) geaccepteerd door NDS Cloud.

De technologiekeuze voor de Platform-IDP (laag 3), Periferie-IDP (laag 4) en Out-of-band-IDP (laag 5) is nog niet vastgesteld. Dit zijn implementatiekeuzes die niet de architectuurgrens raken maar wel de operationele inrichting.

Motivatie

De vijf-lagenstructuur is geen theoretisch model maar een operationele vereiste. Platformbeheer moet kunnen inloggen op de infrastructuur zonder ook maar de mogelijkheid te hebben tenant-data te kunnen zien. Dit is zowel een beveiligingsprincipe als een governanceprincipe: het platform is een vertrouwde infrastructuuraanbieder die de klantomgeving niet binnentreedt.

De keuze om geen platform-IDP voor tenants te leveren is bewust. Het bieden van een centrale tenant-IDP zou het platform verantwoordelijk maken voor de identiteitsadministratie van overheidsorganisaties — een verantwoordelijkheid die niet bij de platformoperator thuishoort en die de autonomie van tenants zou beperken.

De aansluitvoorwaarde (tenant heeft eigen IDP nodig) is een drempel maar een verdedigbare. Overheidsorganisaties die bijvoorbeeld via SSC-ICT of een ander shared service centrum zijn aangesloten hebben via het federatieve koppelvlak al een werkende route. De effectieve drempel is daarmee laag voor de doelgroep.

Consequenties

Tenant-onboarding (ADR-G03): elke tenant moet bij onboarding een federatief koppelvlak aanleveren dat voldoet aan de aansluitvoorwaarden (SAML/OIDC/OAuth2 via bijvoorbeeld SSO-Rijk of direct). Tenants zonder eigen IDP kunnen niet worden geonboard totdat federatie is gerealiseerd. Dit is een expliciete aansluitvoorwaarde.

RBAC en resource-hiërarchie (ADR-G67): de autorisatiestructuur (wie mag wat binnen de tenantomgeving) bouwt voort op de federatieve identiteit van laag 2. De resource-hiërarchie (organisatie → cluster → namespace) spiegelt de identiteitsstructuur van de tenant.

PaaS-diensten (ADR-G49, ADR-G50): bij managed diensten zoals databases ontstaat een aparte vraag. Als het platform een database levert aan een tenant, zijn de databasegebruikers tenantidentiteiten (laag 2) maar de database-instantie draait op platforminfrastructuur (laag 3). Het authenticatiedomein voor database-credentials moet worden uitgewerkt als onderdeel van de managed diensten-architectuur. Dit is een open vraagstuk dat bij de implementatie van elke PaaS-dienst apart moet worden beantwoord.

Out-of-band toegang (laag 5): toegangsprocedures voor fysieke datacenters en OOB-netwerken moeten worden uitgewerkt in de operationele runbooks. De IDP-keuze voor laag 5 is een beveiligingsgevoelige hardware-beslissing (hardware tokens, smartcards).

Geen centrale tenant-IDP: het platform levert geen IDP-dienst aan tenants. Tenants die een IDP nodig hebben voor hun eigen applicaties (laag 1) regelen dit zelf. Dit is expliciet opgenomen in de documentatie en aansluitcondities.

ADR-G09: Loadbalancing en ingress als platform-eigen dienst voor externe service exposure

Status	Goud
Laag	Perifere diensten
Afhankelijk van	ADR-C07, ADR-C08
Maakt mogelijk	ADR-G37
Datum	2026-06-04

Context en probleemstelling

Elke tenant-workload die bereikbaar moet zijn voor gebruikers of andere systemen heeft loadbalancing nodig: verkeer moet worden verdeeld over instanties, TLS moet worden getermineerd, en er moet een gedefinieerde ingress-path zijn voor elke applicatie. Op hyperscale cloud is dit een platformdienst (ELB, GCP LB) die je opt-in afneemt. Op bare-metal moet deze functionaliteit expliciet worden ingericht.

De architectuurvraag is: biedt het platform loadbalancing en ingress als ingebouwde dienst, of verwacht het dat tenants dit zelf regelen of extern afnemen?

Drie posities:

1. **Platform-eigen dienst (verplicht intern)**: het platform levert loadbalancing en ingress als onderdeel van de basisdienst. Tenants gebruiken deze via standaard-API's (Kubernetes Service, Ingress, Gateway API).
2. **Tenant-eigen verantwoordelijkheid**: tenants brengen hun eigen ingress-controller mee (HAProxy, Traefik, etc.).
3. **Externe afhankelijkheid**: loadbalancing verloopt via een extern, al bestaand overheidssysteem.

Eisen die de keuze inperken:

- **Self-service**: tenants moeten zelfstandig endpoints kunnen aanmaken zonder platformteam-interventie.
- **Multi-tenancy**: loadbalancing mag geen informatie van de ene tenant naar de andere lekken.
- **Standaard-API's**: tenants moeten via de Kubernetes-native API's (Service, Ingress, Gateway API) kunnen werken — geen custom workarounds.
- **Integratie met downstream-diensten** (DDoS/WAF — ADR-G37, TLS — ADR-G54): loadbalancing is het natuurlijke integratiepunt voor edge-security.

Besluit

Het platform levert géén L7-loadbalancing en ingress als (verplicht) volledig platform-eigen dienst; dat is de tenant-eigen verantwoordelijkheid. L4-loadbalancing wordt geleverd door Cilium (ADR-C08).

Tenants gebruiken standaard Kubernetes-API's (`Service` van type `LoadBalancer`, `Ingress`, `HTTPRoute` via Gateway API) zonder hun eigen ingress-stack te hoeven installeren.

Onderzochte implementaties

Platform-eigen (gekozen): Het platform biedt zowel L4 (via Cilium LB IPAM + BGP, ADR-C08) als L7 (via gekozen ingress-controller) als standaarddienst. Tenants declareren hun behoefte via standaard Kubernetes-resources. Voordeel: één uniforme aanpak, geïntegreerde observability (Hubble), consistente security-policies (G37, G54), lagere operationele last voor tenants.

Tenant-eigen ingress (bring-your-own): Elke tenant installeert zelf een ingress-controller in het eigen cluster. Meer flexibiliteit per tenant, maar verdubbelde operationele last en geen centrale audit over edge-endpoints. Niet passend bij een managed platform.

Externe loadbalancer (Diginetwerk/NAFIN): Gebruik van een externe overheidsoplossing voor loadbalancing. Veronderstelt een dergelijke dienst die er niet is in deze vorm; introduceert bovendien een externe afhankelijkheid in een kritieke datapath.

Technologiekeuze

De keuze voor loadbalancing als tenant-eigen verantwoordelijkheid is gemaakt. De concrete L7-softwarekeuze (Envoy via Gateway API, HAProxy, etc.) wordt later gemaakt, binnen de eigen tenant.

Voor L4 volgt de keuze uit ADR-C08 (Cilium LB IPAM + BGP).

Motivatie

Loadbalancing is geen randdienst maar een kernonderdeel van het self-service platform. L7-ingress is altijd nodig en hoort bij de basisbelofte van een cloudplatform: “ik deploy een app, ik krijg er een URL voor, klaar.”

Door dit als platformdienst te leveren kunnen de downstream-diensten (DDoS-mitigatie, WAF, TLS-terminatie, observability) op één plek worden geïntegreerd in plaats van per tenant. Dit is efficiënter en veiliger.

Consequenties

- **Platformteam verantwoordelijk voor de volledige L7-stack**: inclusief upgrades, certificaatbeheer (via cert-manager, ADR-G54), observability en incident response.
- **Softwarekeuze moet nog worden gemaakt** door afnemers zelf, binnen hun eigen tenant (e.g. Envoy/Gateway API vs HAProxy).
- **DDoS/WAF integratie** (ADR-G37) hangt aan deze laag en moet daar worden geconfigureerd.
- **TLS-terminatie** gebeurt standaard op de ingress-laag met certificaten van de platform-PKI (ADR-G54). Tenants kunnen desgewenst passthrough-TLS configureren als hun eigen certificaten moeten worden gebruikt.
- **Gateway API als toekomstrichting**: de Kubernetes-community beweegt richting de nieuwe Gateway API (GA sinds 2023). De keuze hiervoor is aan de afnemers zelf.
- **Tenants mogen eigen ingress-controllers installeren binnen hun tenant-namespace** voor specifieke use-cases, maar de platform-ingress blijft de standaard en is onderdeel van de SLA.
- **Externe bereikbaarheid** vereist BGP-advertentie van ingress-IP’s naar Diginetwerk en/of publiek internet (ADR-G04, ADR-G56).

ADR-G10: Basisniveau rubricering: tot welk VIRBI-niveau

Status	Goud
Laag	Beveiliging
Maakt mogelijk	ADR-G38
Datum	2026-08-07

Context en probleemstelling

Overheidsinformatie wordt conform VIRBI 2024 (Voorschrift Informatiebeveiliging Rijksdienst — Bijzondere Informatie) in rubriceringsklassen ingedeeld: openbaar, departementaal vertrouwelijk, staatsgeheim confidentieel, staatsgeheim geheim, staatsgeheim zeer geheim. Ook zijn NIS2/Cbw/BIO2 van toepassing. Hoe hoger de klasse, hoe strenger de technische, fysieke en procedurele beveiligingseisen. De VIRBI-rubricering bepaalt de vertrouwelijkheidsklasse van informatie. De uiteindelijke beveiligingsmaatregelen worden daarnaast bepaald op basis van de BIV-beoordeling (Beschikbaarheid, Integriteit en Vertrouwelijkheid) en de daaruit voortvloeiende risicoanalyse.

Conform notitie Verkenning en Kamerbrief is Hooggerubriceerde informatie out of scope. Dat betekent dat voor Rijksoverheid de Clouddienst een ondersteuningsgrens biedt tot departementaal vertrouwelijk. Voor Provincies, Waterschappen, en Gemeentes die uitgaan van BIO1.04 is het equivalent BBN2. Vanuit NIS2/Cbw en BIO2 ligt de focus primair op risicobeheersing, governance en assurance. Nadere details daarvan worden uitgewerkt in het Operating Model. Zodra dat opgeleverd is zal deze ADR dus her-ijkt worden.

De architectuurvraag is: voor welke rubriceringsklasse kan de Clouddienst aantoonbaar worden ingezet binnen de vastgestelde risicobereidheid en beveiligingsdoelstellingen, en welke consequenties dit heeft voor workloads met een hogere rubricering.

Eisen die de keuze inperken:

- **BIO-basisnorm** de Clouddienst voldoet aan de voor de dienstverlening toepasselijke BIO2-maatregelen. Aanvullende beveiligingsmaatregelen worden risicogebaseerd toegepast op basis van rubricering en classificatie van de workload.
- **EUCS SEAL-4** is de soevereiniteits-ambitie (ADR-002) en stelt strenge eisen aan supply chain, niet direct aan rubricering.
- **Overheidsdoelgroep**: het merendeel van de overheidsworkloads valt in openbaar en departementaal vertrouwelijk. Staatsgeheim-werkloads zijn relatief klein in volume.
- **Economische haalbaarheid**: indien de Clouddienst staatsgeheimen ondersteunt zou dat fysiek geïsoleerd moeten zijn van de rest, wat de schaalvoordelen van een multi-tenant platform tenietdoet.

Besluit

De Clouddienst ondersteunt workloads met informatie die is gerubriceerd tot en met **Departementaal Vertrouwelijk (DEP VERTROUWELIJK)** conform VIRBI 2024. Workloads die informatie bevatten met een hogere rubricering (Staatsgeheim Confidentieel, Geheim of Zeer Geheim) vallen buiten de architectuurscope van de Clouddienst en dienen te worden verwerkt op daarvoor geautoriseerde infrastructuur.

De weerstand tegen statelijke actoren geldt als basiseis voor het de Clouddienst. Een statelijke actor heeft tijd en kiest de weg van de minste weerstand of kleinste kans op detectie. De Clouddienst bort dat tenants/workloads met een lager beveiligingsniveau geen risico opleveren voor hoger ingedeelde tenants.

Onderzochte implementaties

Deze ADR beschrijft geen technologie maar een scope-afbakening. De opties zijn beleidsmatig:

Alleen openbaar: de Clouddienst beperkt zich tot openbare gegevens. Te beperkend: het grootste deel van overheidswerkprocessen verwerkt tenminste departementaal vertrouwelijke informatie (persoonsgegevens, beleidsinformatie in voorbereiding, interne besluitvorming).

Tot en met departementaal vertrouwelijk (gekozen): de Clouddienst ondersteunt openbaar en DEP VERTROUWELIJK. Dit dekt verreweg de meeste overheidsworkloads zonder de zware eisen van staatsgeheim-infrastructuur.

Tot en met staatsgeheim confidentieel: Zou een fundamenteel andere architectuur vereisen, inclusief aanvullende organisatorische, personele, fysieke en technische beveiligingsmaatregelen conform VIRBI. Dit valt buiten de doelstelling van een generiek multi-tenant cloudplatform.

Tot en met staatsgeheim geheim of zeer geheim: Vereist een volledig apart platform met aparte hardware en aparte governance. Niet in scope van NDS Cloud.

Technologiekeuze

N.v.t. — dit ADR stelt de rubriceringsgrens vast; de bijbehorende technische maatregelen zijn uitgewerkt in ADR-G38.

Motivatie

De keuze voor DEP VERTROUWELIJK als bovengrens is een pragmatische balans:

- **Dekking**: Deze scope sluit aan bij het verwachte gebruik van het platform door de meeste overheidsorganisaties, waarbij het merendeel van de workloads geen verwerking van staatsgeheim gerubriceerde informatie vereist. Voor workloads boven deze klasse bestaan al gespecialiseerde oplossingen (zoals Defensie's eigen systemen, MIVD-infrastructuur, BVD-netwerken).
- **Haalbaarheid**: Voor DEP VERTROUWELIJK kunnen de vereiste beveiligingsdoelstellingen worden gerealiseerd door een combinatie van organisatorische, procedurele en technische maatregelen binnen de beoogde platformarchitectuur. Voor hogere rubriceringsniveaus nemen de eisen aan fysieke beveiliging, personele beveiliging, isolatie en assurance aanzienlijk toe.
- **Multi-tenancy werkt**: De gekozen architectuur gaat uit van logisch gescheiden multi-tenancy. Voor workloads tot en met DEP VERTROUWELIJK wordt deze architectuur, in combinatie met aanvullende beveiligingsmaatregelen, passend geacht binnen de vastgestelde risicobeheersing. Hogere rubriceringen vereisen een afzonderlijke beoordeling en vallen buiten de scope van dit platform.
- **Helderheid voor afnemers**: een scherpe bovengrens voorkomt verwachtingen die de Clouddienst niet waar kan maken, en voorkomt juridische risico's voor overheidsorganisaties die per ongeluk hogergeclassificeerde informatie op ongeschikte infrastructuur plaatsen.

Deze architectuurbeslissing stelt een expliciete bovengrens aan de ondersteunde rubriceringsklasse. Workloads met een hogere rubricering vallen buiten de scope van het platform. Indien de scope in de toekomst wordt uitgebreid, vereist dit een nieuwe architectuurbeslissing en een herbeoordeling van de beveiligingsarchitectuur, governance en assurance.

Consequenties

- **Workloads boven DEP VERTROUWELIJK** worden niet toegelaten op de Clouddienst en dienen te worden verwerkt op daarvoor geschikte en geautoriseerde infrastructuur. Tenants die dergelijke workloads hebben, kunnen een deel van hun portfolio op dit de Cloudienst plaatsen en een deel elders — dit vereist een workload-classificatie-proces aan tenant-zijde.
- **Aanvullende maatregelen per workload-klasse** (ADR-G38): binnen de DEP-VERTROUWELIJK-bovengrens kan een tenant met specifiek hogere vereisten aanvullende technische maatregelen krijgen (bijv. dedicated nodes, versterkte monitoring, verzwaarde encryptie). Dit is een keuze op tenant-niveau.
- **Classificatiemodel per workload-klasse nodig**: de Clouddienst moet een classificatie-mechanisme bieden. De tenant blijft verantwoordelijk voor de classificatie en rubricering van zijn informatie. De Clouddienst faciliteert het vastleggen van deze classificatie en het afdwingen van daarop gebaseerde beleidsregels (e.g. toegangscontrole, netwerksegmentatie, storage-encryptie-policy).
- **Documentatie en bestuurlijke verantwoording**: Tijdens onboarding (ADR-G03) bevestigt de tenant dat workloads binnen de ondersteunde rubriceringsgrens vallen. De tenant blijft verantwoordelijk voor juiste classificatie van informatie. Het platform ondersteunt deze verantwoordelijkheid met beleidscontroles, maar neemt deze verantwoordelijkheid niet over.
- **Compliance-toetsing**: De assurance- en certificeringsdocumentatie van de Clouddienst beschrijft expliciet de ondersteunde rubriceringsklasse, de gehanteerde uitgangspunten en de bijbehorende beveiligingsmaatregelen zodat afnemers dit kunnen betrekken in hun eigen risicoafweging en compliance-verantwoording.
- **Koppeling met andere rubriceringskaders**: naast VIRBI gelden soms sectorspecifieke kaders (zoals DEFSEC voor Defensie). De Clouddienst kent één rubriceringsmodel (VIRBI) en hanteert NIS2/Cbw/BIO2; tenants moeten zelf zorgdragen voor eventuele mappings.
- **Risicobeoordeling**: De vastgestelde rubriceringsgrens vormt een uitgangspunt voor risicobeoordelingen, assurance, incidentrespons, leveranciersmanagement en wijzigingsbeheer. Wijzigingen die de ondersteunde rubriceringsklasse kunnen beïnvloeden vereisen een herbeoordeling van deze ADR.
- **Weerstand tegen dreigingen**: Naast de VIRBI-rubricering kan voor een workload een verhoogd weerstandsniveau vereist zijn, bijvoorbeeld vanwege dreiging door statelijke actoren. De tenant blijft verantwoordelijk voor het aangeven van deze aanvullende beveiligingseisen; de Clouddienst vertaalt deze naar aanvullende beleidsregels en technische maatregelen voor zover deze binnen de vastgestelde platformscope vallen.

ADR-G12: Inrichting internationaal datacenter

Status	Zilver
Laag	Datacenter
Afhankelijk van	ADR-G01
Datum	2026-06-19

Context en probleemstelling

Datacenter buiten Nederland voor diplomatieke/defensie-workloads.

Besluit

In het platform is ruimte voor een DR (disaster recovery) datacenter wat buiten Nederland kan staan. Dit datacenter wordt niet primair gebruikt worden voor live migraties, maar is enkel beschikbaar als recovery omgeving.

Motivatie

Een extern datacenter is geen onderdeel van de eerste wave, wel wordt in de architectuur rekening gehouden met de mogelijkheid voor een extern internationaal datacenter.

Consequenties

Kan in vervolg sessies later verder worden ingevuld.

ADR-G13: Hardware-selectiestrategie: multi-vendor vs single-vendor per generatie

Status	Brons
Laag	Hardware
Afhankelijk van	ADR-001
Maakt mogelijk	
Datum	2026-07-29

Context en probleemstelling

Het platform draait op bare-metal servers in meerdere OverheidsDataCenters. Die servers worden ingekocht maar bij wie, hoe, en hoeveel leveranciers tegelijk?

Dit is geen inkoopdiscussie. Het is een architectuurvraag omdat de keuze voor één of meerdere hardwareleveranciers directe gevolgen heeft voor de provisioning-pipeline, de firmware-beheerlaag, de certificeringsstrategie en de operationele complexiteit. Een server van leverancier A heeft andere BMC-firmware, andere BIOS-instellingen, andere NIC-drivers en andere out-of-band-protocollen dan een server van leverancier B. Hoe meer variatie in de hardwarelaag, hoe meer paden de provisioning-software moet ondersteunen.

De architectuurvraag is: accepteert het platform meerdere hardwareleveranciers tegelijk in productie, of wordt er per hardwaregeneratie voor één leverancier gekozen, en wat zijn de gevolgen voor de lagen erboven?

De eisen die de keuze inperken:

- **Aanbestedingswet**: de architectuur mag niet zodanig worden ingericht dat slechts één leverancier kan leveren. De opdracht eist open standaarden en breed inzetbare oplossingen. Dit betekent niet dat je altijd meerdere leveranciers tegelijk moet gebruiken, maar wel dat de architectuur niet structureel afhankelijk mag zijn van één leverancier.
- **Soevereiniteit (ADR-002)**: SEAL-4 vereist dat de opdrachtgever controle heeft over de supply chain. Afhankelijkheid van één leverancier kan een soevereiniteitsrisico zijn als die leverancier wegvalt of wordt overgenomen (cf. Broadcom/VMware).
- **Provisioning-complexiteit (ADR-C60)**: elke extra hardwarevariant betekent extra testpaden in de provisioning-pipeline. Metal³/Ironic (optie B in C60) abstraheert via IPMI/Redfish, maar de praktijk leert dat BMC-implementaties tussen leveranciers significant verschillen.
- **Certificering**: als het platform een certificeringstraject doorloopt (EUCS, BIO), moet de hardwareconfiguratie herhaalbaar en gedocumenteerd zijn. Meer varianten betekent meer certificeringswerk.
- **Schaal en levertijd**: bij tienduizenden servers is leveringszekerheid een risico. Afhankelijkheid van één leverancier maakt kwetsbaar voor supply chain verstoringen (chiptekorten, geopolitieke restricties).
- **Provisioning** De gekozen switches moeten metal-stack ondersteunen (e.g. SONiC/Edgecore)

Besluit

[Nog geen besluit — voorgelegd ter bespreking]

Onderzochte implementaties

Optie A: Single-vendor per hardwaregeneratie

Per inkoopcyclus (typisch 3–5 jaar) wordt één serverleverancier geselecteerd via aanbesteding. Alle servers in die generatie zijn identiek qua platform, BMC-type en firmware-versie. Bij de volgende generatie kan een andere leverancier worden gekozen.

Voordelen:

- Eén firmware-baseline, één BMC-type, één set drivers per generatie — minimale variatie in provisioning
- Testmatrix is klein: één hardwareprofiel per generatie dat uitputtend wordt getest
- Certificering is eenvoudiger: één referentieconfiguratie per generatie
- Operationeel simpeler: het team hoeft maar één type hardware diepgaand te kennen
- Schaalvoordeel in inkoop

Nadelen:

- Single point of failure in de supply chain per generatie — als de leverancier niet kan leveren, is er geen alternatief
- Minder onderhandelingsmacht na contractering
- Generatiewisseling is een groot migratiemoment: nieuwe firmware, nieuwe drivers, potentieel nieuwe provisioning-paden
- Perceptie van vendor lock-in kan politiek gevoelig liggen in overheidscontext

Referenties: dit is het model dat de meeste hyperscalers hanteren (Google, Meta). Per serverrack-generatie wordt één ODM/OEM geselecteerd. De uniformiteit maakt automatisering op schaal mogelijk.

Optie B: Multi-vendor tegelijk in productie

Meerdere leveranciers leveren tegelijk servers die naast elkaar in dezelfde clusters draaien. De provisioning-pipeline moet alle varianten ondersteunen.

Voordelen:

- Geen afhankelijkheid van één leverancier — leveringsrisico gespreid
- Continue concurrentiepositie — leveranciers weten dat ze vervangbaar zijn
- Kan verplicht zijn vanuit aanbestedingsrechtelijke overwegingen als de opdracht zo wordt uitgelegd
- Flexibiliteit om per use case de beste hardware te kiezen (bijv. GPU-servers van leverancier X, storage-servers van leverancier Y)

Nadelen:

- Provisioning moet meerdere BMC-implementaties ondersteunen — Redfish is een standaard maar interpretatie verschilt sterk tussen leveranciers

- Firmware-management wordt per leverancier een apart pad: andere update-tools, andere validatie, andere rollback-procedures
- Testmatrix explodeert: elke combinatie van hardware × firmware × OS × Kubernetes-versie moet worden gevalideerd
- Meer operationele kennis nodig: het team moet meerdere hardwareplatforms diepgaand kennen
- Certificering wordt complexer: meerdere referentieconfiguraties

Referenties: dit is het model dat CERN en sommige telecom-providers gebruiken. Het werkt op schaal, maar vereist een volwassen hardware-abstractielaag en een groter platform-engineeringteam.

Optie C: Functioneel gescheiden multi-vendor

Eén leverancier per functie: bijv. leverancier A voor compute-nodes, leverancier B voor storage-nodes, leverancier C voor GPU-nodes. Binnen elke functie is het single-vendor.

Voordelen:

- Combineert best-of-breed per functie met beperkte provisioning-complexiteit
- Elke functie heeft een homogene hardware-baseline
- Leveringsrisico gespreid over functies
- Dekt de aanbestedingseis af: meerdere leveranciers in scope

Nadelen:

- Nog steeds meerdere firmware-paden en BMC-varianten — minder dan optie B, meer dan optie A
- Grens tussen functies kan verschuiven: een compute-node die ook lokale storage levert past niet meer in het model
- Per functie alsnog single-vendor risico

Referenties: dit is het model dat veel enterprise-omgevingen hanteren. Het is een pragmatisch compromis maar vereist duidelijke definities van serverfuncties.

Technologiekeuze

[Nog niet gemaakt — dit is een architectuurkeuze die technologie-agnostisch is. De keuze bepaalt wél de eisen aan de provisioning-pipeline (ADR-C60) en het firmware-beheer (ADR-G14).]

Motivatie

[Uit te werken na bespreking]

De kernafweging is: **operationele eenvoud vs. leveranciersonafhankelijkheid**. Bij de schaal van dit platform (tienduizenden servers) weegt operationele eenvoud zwaar — elke extra variant vermenigvuldigt de testlast. Tegelijk weegt soevereiniteit zwaar — afhankelijkheid van één leverancier staat op gespannen voet met SEAL-4.

Optie C biedt een middenweg die beide belangen adresseert, mits de serverfuncties helder zijn gedefinieerd en stabiel blijven.

Optie B meerdere vendoren tegelijkertijd in productie, met een limiet van twee a drie vendoren. (dit kan de keuze worden)

Consequenties

[Uit te werken na besluit]

Ongeacht de keuze:

- De provisioning-pipeline (ADR-C60) moet worden ontworpen op de maximale hardwarevariatie die het besluit toelaat. Bij optie A is dat minimaal; bij optie B maximaal.
- ADR-G14 (hardware lifecycle) wordt direct beïnvloed: firmware-beheer bij multi-vendor is fundamenteel anders dan bij single-vendor.
- De certificeringsstrategie moet per hardwareconfiguratie een referentie-image definiëren.
- Het inkooptraject (buiten scope architectuur) wordt ingekaderd door deze keuze.

ADR-G23: IPv6-modernisatie strategie uitwerking

Status	Goud
Laag	Networking
Afhankelijk van	ADR-G05
Datum	2026-07-07

Context en probleemstelling

ADR-G05 stelt IPv4-primair met IPv6 als aanvulling vast. Die principebeslissing laat het operationele model open: een dual-stack platform kan op veel manieren worden ingericht, van “beide protocollen volledig gelijkwaardig” tot “IPv6-native met IPv4 als exit-pad op specifieke punten.” Het verschil tussen deze benaderingen heeft grote operationele gevolgen.

De architectuurvraag is: hoe wordt IPv6-first concreet uitgewerkt in NetworkPolicies, routing, service discovery en operationele tooling — en welke tijdlijn geldt voor het uitfaseren van IPv4, om op termijn IPv6 volledig te omarmen?

Concrete deelvragen:

- Worden NetworkPolicies per protocol geschreven (afzonderlijk IPv4 en IPv6) of protocol-onafhankelijk?
- Welke componenten krijgen native IPv6 en welke alleen dual-stack?
- Waar zit IPv4-connectiviteit nog wél — alleen aan de rand voor legacy-integratie, of tot diep in het platform?
- Is er een uitfaseer-tijdlijn voor IPv4 of blijft dual-stack permanent?

Eisen die de keuze inperken:

- **Overheidsbeleid IPv6-first** (Forum Standaardisatie, NORA).
- **Legacy-compatibiliteit**: een deel van het overheidslandschap en externe ketens is nog IPv4-only.
- **Complexiteit-beperking**: protocol-dubbel beleid verdubbelt regelsets en is een bron van fouten.
- **Auditeerbaarheid**: NetworkPolicies moeten leesbaar blijven; protocol-specifieke duplicatie maakt audits zwaar.

Besluit

Het platform hanteert een **operationeel model met protocol-onafhankelijke netwerkpolicies en een expliciete IPv4-uitfaseerstrategie**.

Concreet:

- Alle NetworkPolicies worden **protocol-onafhankelijk** geschreven (CIDR-onafhankelijk, gebruik van labels en selectors). Het dataplane (Cilium, ADR-C07) past de policy toe op beide protocollen simultaan.
- Service discovery verloopt via IPv6 waar mogelijk (DNS AAAA-records, ADR-G55). Dual-stack DNS wordt ondersteund.

- IPv4 is beschikbaar voor externe endpoints die geen IPv6 ondersteunen. Intern platformverkeer is IPv4 met IPv6-first voor alle componenten dat ondersteunen, voor tenant-workloads die dat willen
- **Geen harde IPv4-uitfaseerdatum** wordt nu vastgesteld — dit is afhankelijk van externe ketens. Wel wordt jaarlijks een herbeoordeling gedaan op basis van de IPv6-adoptie in het overheidslandschap.

Onderzochte implementaties

Protocol-onafhankelijke policies (gekozen): NetworkPolicies schrijven `from: podSelector: {matchLabels: ...}` in plaats van CIDR-blocks. Cilium past de policy toe op beide protocollen. Voordeel: één regelset, geen duplicatie, policies blijven leesbaar. Nadeel: vereist discipline — CIDR-gebaseerde policies bestaan nog voor externe integraties.

Protocol-specifieke policies (afgewezen): Aparte policies voor IPv4 en IPv6. Exacte controle maar verdubbelde onderhoudlast en een bekende bron van configuratiefouten (policy bijwerken voor IPv6 maar vergeten voor IPv4 of omgekeerd).

IPv6-only intern, NAT64 aan de rand: Intern platformverkeer volledig IPv6-only, IPv4-verkeer naar/van externe endpoints wordt vertaald via NAT64 + DNS64. Operationeel eleganter, maar vereist een volwassen NAT64-oplossing en DNS64-ondersteuning. Potentiële vervolgstap; nu niet direct beslist.

Harde IPv4-uitfaseerdeadline: Een vaste datum (bijv. 2030) waarop IPv4 binnen het platform wordt uitgeschakeld. Dwingt externe ketens tot aanpassing. Niet haalbaar zonder overheidsbrede coördinatie — buiten macht van dit platform.

Technologiekeuze

Cilium (ADR-C07) ondersteunt dual-stack IPv6/IPv4 en protocol-onafhankelijke NetworkPolicies via `CiliumNetworkPolicy`. Geen aparte toolingkeuze nodig.

Voor de toekomstige NAT64-uitbreiding: Tayga of Jool als open source NAT64-implementaties. Beslissing uitgesteld tot de behoefte zich daadwerkelijk manifesteert.

Motivatie

Protocol-onafhankelijke policies zijn de enige manier om dual-stack schaalbaar te beheren. Iedere policy-duplicatie is een potentiële afwijking tussen IPv4- en IPv6-gedrag die in een audit moet worden verantwoord. Door policies op labels en selectors te baseren wordt het dataplane verantwoordelijk voor de protocol-translatie.

Geen harde uitfaseerdatum is een bewuste keuze: het platform kan niet eenzijdig dicteren wanneer externe ketens IPv6-klaar zijn. Een jaarlijkse herbeoordeling voorkomt dat dual-stack een “eeuwige permanent” wordt zonder dat iemand het nog kritisch bekijkt.

Consequenties

- **Netwerkcomponenten hoeven niet IPv6-first te worden geconfigureerd**: Cilium, CoreDNS, ingress-controllers (ADR-G09), operators, storage (Ceph, ADR-C05).
- **Dual-stack testen** is een permanente verantwoordelijkheid: elke nieuwe component wordt getest op beide protocollen.
- **Tenants worden aangemoedigd IPv6-first te werken**: platformdocumentatie is IPv6-first, voorbeelden zijn IPv6-first, en tenant-admin trainingen behandelen IPv6 als norm.
- **Legacy-tooling**: sommige observability- en security-tools hebben historisch zwakke IPv6-ondersteuning. Dit is een selectiecriterium bij toolkeuze.
- **Uitfasering IPv4**: de jaarlijkse herbeoordeling wordt geagendeerd bij het programmamanagement. Criteria voor uitfasering: IPv6-adoptie-percentage in overheidsnetwerken, IPv6-bereikbaarheid van kern-GDI-voorzieningen, restgebruik IPv4 op het platform.
- **Documentatie**: het platform publiceert een IPv6-handleiding voor tenants met best practices voor IPv6-native applicatieontwerp.

Sessienotities

> Protocol-onafhankelijke policies voorkomen afzonderlijk beleid voor IPv4 en IPv6. Tijdlijn uitfaseren IPv4 is niet door het platform te bepalen — jaarlijks herbeoordelen.

ADR-G25: IPAM-tooling en integratie met bestaande overheids-IP-registraties

Status	Goud
Laag	Networking
Afhankelijk van	ADR-G05
Datum	2026-07-07

Context en probleemstelling

ADR-G05 stelt dat de platform-adresruimte hiërarchisch wordt beheerd (platform → ODC → cluster → tenant → namespace). Op schaal van duizenden subnetten is dit alleen werkbaar met een IP Address Management (IPAM) systeem dat automatisch toewijzingen doet, conflicten voorkomt en een enkelvoudige bron-van-waarheid biedt.

Tegelijk hebben overheidsorganisaties vaak al bestaande IPAM-systemen (DDI-oplossingen zoals BlueCat, Infoblox, of zelfbouw). Die bevatten allocaties die het platform moet respecteren — dubbele registratie van dezelfde IP-range in twee systemen is een gegarandeerde bron van conflicten.

De architectuurvraag is: welke IPAM-tooling wordt op het platform ingezet, en hoe verhoudt die zich tot bestaande overheids-IP-registraties?

Deelvragen:

- Heeft het platform zijn eigen IPAM of integreert het met een bestaande overheids-IPAM?
- Welk systeem is de bron-van-waarheid voor platform-interne prefixes?
- Hoe wordt tenant-adresruimte gesynchroniseerd met tenant-interne administraties?
- Welke API-koppelvlakken zijn geschikt voor integratie?

Eisen die de keuze inperken:

- **Eén bron-van-waarheid per CIDR-range**: dubbele administraties leiden tot conflicten.
- **API-gedreven**: handmatig IPAM-beheer schaalt niet op dit platform.
- **Federatie-model**: verschillende overheidsorganisaties hebben verschillende IPAM-oplossingen; het platform moet er flexibel mee kunnen koppelen.
- **IPv4-primair met IPv6 als aanvulling** (ADR-G23): de IPAM-oplossing moet IPv6-native zijn, geen IPv4-only legacy-tool.

Besluit

Het platform integreert met externe overheids-IPAM via **standaard API-protocollen in een federation-model**. Het platform voert geen eigen IPAM voor ranges die elders al beheerd worden; voor platform-interne ranges is NetBox de primaire IPAM-component.

Concreet:

- **Platform-interne adresruimte** (platform-prefix, ODC-prefixes, platform-services): beheerd in NetBox als primaire bron-van-waarheid.

- **Tenant-adresruimte**: afgeleid uit de platform-prefix; per tenant een /48-subnet toegewezen in NetBox.
- **Externe overheids-IPAM**: het platform consumeert via API waar overheidsnetwerk-allocaties vandaan komen (Diginetwerk, NAFIN-ranges). Geen dubbele registratie.

Onderzochte implementaties

NetBox (open source, Python/Django): Breed gebruikt open source IPAM + DCIM-platform. REST- en GraphQL-API, rijke integratiemogelijkheden. Actieve community, ook in gebruik bij Nederlandse overheidsorganisaties. Voordeel: open, uitwisselbaar, breed gedragen. Nadeel: Python/Django stack vereist operationeel beheer.

phpIPAM: Lichtgewicht open source IPAM. Kleinere community, minder rijke API. Minder geschikt voor dit schaalniveau.

Infoblox (commercieel): Enterprise DDI-platform, marktleider. Rijke integratie met overheidsnetwerken. Nadeel: vendor lock-in, licentiekosten, past minder bij open source-strategie van het platform.

BlueCat (commercieel): Vergelijkbaar met Infoblox. Vergelijkbare afwegingen.

NIPAP (open source, Python): Specialistisch IPAM-project met focus op IPv6 en grote hiërarchieën. Kleinere community dan NetBox. Interessant voor specifieke use-cases.

Federatie via REST/DHCP/RADIUS-API's (gekozen complementair model): Het platform consumeert externe IPAM-allocaties via API's zonder die zelf te beheren. Standaardprotocollen (REST, DHCPv6) maken dit leveranciersonafhankelijk.

Technologiekeuze

NetBox als primaire platform-interne IPAM, **federation via standaard API's** voor externe overheids-IPAM-systemen.

NetBox biedt voldoende functionaliteit voor platform-interne behoefte, is open source, en wordt door andere overheidsorganisaties ingezet (wat koppeling vergemakkelijkt). Voor bestaande overheids-IPAM-systemen (BlueCat, Infoblox bij specifieke organisaties) gebruikt het platform hun API's zonder de ranges te dupliceren.

Motivatie

Een enkelvoudige platform-IPAM maakt het beheer werkbaar. Door voor externe ranges te federeren in plaats van te dupliceren wordt conflicten voorkomen. NetBox is breed gedragen en sluit aan op hoe andere overheidsplatforms hun IPAM inrichten, wat integratie vergemakkelijkt.

Door te kiezen voor open source voor de platform-IPAM blijft het platform compliant met de opdracht (open standaarden, geen vendor lock-in). Commerciële IPAM-oplossingen blijven mogelijk aan tenant-zijde; de integratie is API-gedreven en leveranciersonafhankelijk.

Consequenties

- **NetBox wordt een platformdienst**: HA-opzet, backup, monitoring, onderdeel van platformdiensten-catalogus (ADR-G49).
- **Koppelvlakken met bestaande IPAM-systemen**: per tenant of per overheidsnetwerk moet een koppeling worden ingericht. Dit vereist per tenant-organisatie een eenmalige afstemming.
- **Integratie met Cilium**: per-tenant IP-pools (ADR-C43) worden op basis van NetBox-allocaties geconfigureerd. Dit vereist een sync-mechanisme (GitOps via CoreDNS-configs of een custom operator).
- **Bron-van-waarheid-afspraak**: voor elke CIDR-range moet duidelijk zijn welk systeem de bron is. Dit wordt in het tenant-onboarding-proces (ADR-G03) vastgelegd.
- **IPv6-workflow**: NetBox ondersteunt IPv6, maar veel bestaande overheids-IPAM-systemen zijn IPv4-first. Dit kan een knelpunt zijn bij integratie; de platform-prefix-hiërarchie (ADR-G05) is daarom platform-beheerd.
- **Documentatie**: tenants krijgen een IPAM-handleiding die uitlegt hoe zij hun allocaties ophalen en registreren.

Sessienotities

Federation-model met externe IPAM respecteert bestaande administraties en voorkomt dubbele bron van waarheid. NetBox voor platform-interne ranges.

ADR-G37: DDoS-mitigatie en WAF-integratie

Status	Goud
Laag	Perifere diensten
Afhankelijk van	ADR-G09, ADR-G10
Maakt mogelijk	ADR-G63
Datum	2026-07-07

Context en probleemstelling

Publiek bereikbare overheidsdiensten zijn aantrekkelijke doelen voor Distributed Denial of Service (DDoS) aanvallen, scanning-activiteiten en application-layer aanvallen (SQL-injection, XSS, credential-stuffing). Het platform host zowel vitale overheidsdiensten (DigiD-integraties, uitvoeringsorganisaties) als minder kritieke workloads. Een uniform zwaar beschermingsregime voor alle workloads is duur; een uniform licht regime biedt onvoldoende bescherming waar het nodig is.

De architectuurvraag is: welke DDoS-mitigatie en WAF-functionaliteit biedt het platform, op welk niveau, en hoe wordt dit gedifferentieerd naar tenant en workload-classificatie?

Drie functionele lagen:

1. **Volumetrische DDoS** (L3/L4): grote hoeveelheden verkeer, botnets. Mitigatie vereist upstream capaciteit (ISP-niveau of specialistische scrubbing-centers).
2. **Application-layer DDoS** (L7): slim verkeer dat er normaal uitziet maar applicaties overbelast. Mitigatie via rate-limiting, behavioral analysis.
3. **WAF (Web Application Firewall)**: content-inspectie tegen bekende en nieuwe applicatie-aanvallen (OWASP Top 10).

Eisen die de keuze inperken:

- **BIO / NIS2**: tenants met vitale of gevoelige workloads moeten aantoonbare WAF-bescherming hebben.
- **Soevereiniteit (ADR-002)**: externe DDoS-scrubbing via niet-EU-partijen (Cloudflare, Akamai) is een soevereiniteitsrisico; Nederlandse/EU-alternatieven hebben voorkeur.
- **Kostenefficiëntie**: niet elke tenant heeft hoogste bescherming nodig; gedifferentieerd model is redelijk.
- **Integratie met de loadbalancing-laag (ADR-G09)**: WAF hoort logisch op of achter de ingress-controller.

Besluit

Het platform biedt **gedifferentieerde DDoS- en WAF-bescherming per tenant- en workload-classificatie**: een basis-bescherming platform-breed voor alle workloads, en een uitgebreide bescherming als opt-in voor tenants die hoger risicoprofiel hebben of vitale workloads hosten.

Concrete niveaus:

- **Basis (standaard, alle tenants)**: L3/L4 rate-limiting op de ingress-laag, basis-WAF-regels (OWASP CRS), platform-niveau anomaliedetectie via observability (ADR-G41).
- **Uitgebreid (opt-in, doorbelast)**: uitgebreide WAF-regelsets (custom rules per applicatie), upstream volumetrische DDoS-bescherming via gecontracteerde NL/EU-scrubbing, dedicated incident response voor security-events.

Onderzochte implementaties

ModSecurity met OWASP Core Rule Set (basis-WAF): Standaard open source WAF, integreerbaar met NGINX, Envoy, HAProxy. OWASP CRS is de industriestandaard regelset. Voordeel: volledig open source, geen vendor lock-in, breed gedragen. Geschikt voor basis-bescherming.

Coraza (open source WAF, Go): Moderne ModSecurity-vervanger, geschreven in Go, CNCF-incubating. Beter integreerbaar met cloud-native stacks. Strategisch geprefereerde implementatierichting; ModSecurity blijft een gevalideerd alternatief zolang Coraza-integratie nog niet is afgerond.

Open-appsec: Open source WAF met machine learning-gebaseerde detectie. Afgevallen: kleinere community, onvoldoende productiebewijs op schaal voor een overheidsplatform.

Commerciële WAF (F5 Advanced WAF, Imperva, Akamai): Rijker feature-set, vendor support. Past minder goed bij open source-strategie maar kan aanvullend worden ingezet.

Upstream volumetrische DDoS-mitigatie: moet buiten het platform geregeld worden — via ISP, Diginetwerk of gespecialiseerde NL/EU-scrubbing-dienst. Geen platform-interne oplossing voor true volumetric (10+ Gbps aanvallen).

Cloudflare / Akamai scrubbing: Wereldwijde scrubbing-netwerken. Nadeel: soevereiniteit. Alleen inzetbaar voor openbare workloads waar soevereiniteit minder kritisch is.

Technologiekeuze

Voor basis-bescherming: **Coraza met OWASP CRS** op de platform-ingress (ADR-G09); ModSecurity als interim-alternatief tot volledige Coraza-integratie.

Voor uitgebreide bescherming: combinatie van (a) custom WAF-regels op ingress, (b) contract met NL/EU-scrubbing-aanbieder voor volumetrische mitigatie, (c) aangepaste rate-limiting en anomaliedetectie.

De selectie van de scrubbing-aanbieder is een inkoopbeslissing die vóór vaststelling van deze ADR wordt afgerond. Eis: NL of EU-gevestigde aanbieder.

Motivatie

Gedifferentieerde bescherming is een combinatie van tenant-afhankelijkheid en keuze op basis van eigen risicoprofiel. Een uniforme zware bescherming voor alle tenants zou duur zijn en onnodig voor niet-kritieke workloads. Een uniforme lichte bescherming zou tekortschieten voor vitale overheidsdiensten.

Door standaard een basis te bieden en uitbreidingen opt-in te maken, draagt het platform bij aan security-by-default terwijl tenants verantwoordelijkheid nemen voor hun eigen risicoprofiel.

Consequenties

- **Twee dienstniveaus nodig**: platform-ingress moet per workload de policy kunnen toepassen (standaard basis, opt-in uitgebreid via annotaties of aparte Ingress-class).
- **Kosten uitgebreide variant worden doorbelast** aan tenants die de opt-in kiezen (ADR-G59).
- **WAF-regels onderhoud**: OWASP CRS wordt regelmatig bijgewerkt. Regelwijzigingen kunnen false positives introduceren; een testproces vóór productie is nodig.
- **Tenants met custom applicatie-regels** moeten zelf hun WAF-tuning aanleveren via een gestandaardiseerde API of via platformteam-ondersteuning.
- **DDoS-incident response** vereist een procedure die SOC-integratie (ADR-G63) koppelt aan de upstream-scrubbing-aanbieder.
- **Observability**: WAF-events moeten worden gelogd richting de observability-stack (ADR-G41) en het SOC (ADR-G63).
- **Vitale-workload-classificatie**: tenants met vitale of gerubriceerde workloads (ADR-G10) moeten het uitgebreide niveau afnemen — dit is onderdeel van de aansluitvoorwaarden.
- **Soevereiniteit**: de gekozen scrubbing-aanbieder moet NL of EU-gevestigd zijn. Dit sluit Cloudflare en Akamai uit voor vitale workloads.

Sessienotities

[04-03-2026] Combinatie van tenant-afhankelijkheid en keuze op basis van eigen risicoprofiel. Basis voor iedereen, uitgebreid als opt-in met doorbelasting.

[07-07-2026] ADR-G35-verwijzing gecorrigeerd (bestaat niet); G10 toegevoegd aan afhankelijkheden; Open-appsec expliciet afgevallen; Coraza aangewezen als geprefereerde WAF-implementatie; scrubbing-aanbiederselectie formeel uitgesteld naar vaststelling.

ADR-G38: Technische maatregelen voor DEP VERTROUWELIJK workloads

Status	Brons
Laag	Beveiliging
Afhankelijk van	ADR-G10, ADR-G02, ADR-G53, ADR-G54
Maakt mogelijk	ADR-G63
Datum	2026-08-07

Context en probleemstelling

ADR-G10 stelt dat de Clouddienst workloads tot en met DEP VERTROUWELIJK ondersteunt. ADR-G02 bepaalt dat isolatie software-defined is. Die uitgangspunten laten open welke concrete technische maatregelen worden getroffen — “software-defined isolatie voor DEP VERTROUWELIJK” kan variëren van minimale NetworkPolicies tot een verzwaard stelsel met gecrypteerde volumes, dedicated nodes en uitgebreide audit logging.

Een gedifferentieerd model is nodig: niet elke workload op het platform vereist dezelfde zware maatregelen. Een openbare brochure-site en een workload die persoonsgegevens verwerkt verdienen verschillende beveiliging, ook binnen dezelfde tenant.

De architectuurvraag is: welke technische maatregelen gelden platform-breed (voor alle workloads), en welke aanvullende maatregelen gelden voor workloads met verzwaarde eisen?

Eisen die de keuze inperken:

- **BIO basisnormen** gelden platform-breed.
- **DEP VERTROUWELIJK-eisen** voor workloads die dat rubriceringsniveau vereisen (versterkte encryptie, audit logging, toegangscontrole).
- **NIS2**: tenants met vitale voorzieningen hebben aanvullende monitoring- en meldingsverplichtingen.
- **Operationele haalbaarheid**: elk aanvullend regime kost operationele capaciteit. Niet elke tenant kan of wil alles zelf beheren.

Besluit

De Clouddienst hanteert een **gelaagd maatregelenmodel** met een sterke basis voor alle workloads, aangevuld met opt-in verzwaringen per workload-klasse. Security is geen add-on maar standaard — de basis is al substantieel.

Platform-brede basismaatregelen (alle workloads):

- Immutable OS zonder SSH (ADR-C02)
- Cilium NetworkPolicies verplicht (default-deny per namespace)
- mTLS voor intra-cluster verkeer waar beschikbaar (ADR-C38, opt-in per tenant)
- Encryptie-at-rest voor alle Ceph-volumes (ADR-C05)
- Encryptie-in-transit via TLS met certificaten van platform-PKI (ADR-G54)

- Audit logging voor Kubernetes API-calls, naar platform-observability (ADR-G41)
- Centrale KMS voor secrets (ADR-G53)
- Dedicated cluster per tenant (ADR-C04) — geen gedeelde control plane
- DDOS bescherming. Een aanval op een tenant/workload mag niet de beschikbaarheid van de Cloud-dienst in gevaar kunnen brengen

Aanvullende maatregelen per workload-klasse (opt-in, doorbelast):

- Dedicated nodes (geen gedeelde hardware met andere tenants)
- Aanvullende netwerk-isolatie (air-gapped namespaces, micro-segmentatie)
- Verzwaarde WAF (ADR-G37 uitgebreide variant)
- Extended audit logging (langere bewaartermijn, detail-niveau)
- Per-tenant KMS-instantie in plaats van gedeelde KMS-instantie
- Aanvullende SOC-monitoring (ADR-G63) en incident response

Onderzochte implementaties

Dit ADR is een architectuurprincipe en maatregelenkatalogus, geen technologiekeuze. De implementaties zijn de concrete maatregelen hierboven, die elk hun eigen ADR hebben of krijgen.

Gelaagd model (gekozen): Basis + opt-in verzwaring. Balans tussen beveiliging en bruikbaarheid.

Minimaal platform-breed, maximaal per tenant: Tenants zijn volledig zelf verantwoordelijk voor hun beveiliging. Te zwak uitgangspunt voor een overheidsplatform; ondergraaft security-by-default.

Maximaal platform-breed, geen differentiatie: Alle workloads krijgen de zwaarste maatregelen. Duur, complex, en nuttige flexibiliteit gaat verloren.

Technologiekeuze

De afzonderlijke technische maatregelen verwijzen naar hun eigen ADRs:

- Network-isolatie: ADR-C07, ADR-C38, ADR-C43
- Encryptie (at-rest): ADR-C05
- Encryptie (in-transit): ADR-G54
- Secrets: ADR-G53
- Audit logging: ADR-G41, ADR-G63
- Immutable OS: ADR-C02
- Tenant-isolatie: ADR-C04

Motivatie

Security-by-default is een architectuurprincipe, niet een optionele eigenschap. Door de basis al sterk te maken hoeft een tenant niet expliciet om beveiliging te vragen — die is er standaard. Dit sluit aan op de BIO-filosofie en voorkomt dat onzorgvuldig geconfigureerde workloads onbeschermd draaien.

Gedifferentieerd betekent dat tenants met verzwaarde risicoprofielen (vitale voorzieningen, DEP VERTROUWELIJK-zwaar, persoonsgegevens-intensief) extra maatregelen kunnen afnemen zonder dat alle andere tenants die kosten dragen.

Consequenties

- **Classificatiemodel per workload-klasse** moet worden uitgewerkt: tenants classificeren hun workloads (bijv. “openbaar”, “DEP VERTROUWELIJK-standaard”, “DEP VERTROUWELIJK-verzwaard”) en de Clouddienst past automatisch de juiste maatregelen toe.
- **Basismaatregelen gelden voor alle workloads** — ook voor dev/test-omgevingen. Dit voorkomt dat productie-patronen moeten worden aangeleerd bij promotie van dev naar prod.
- **Opt-in maatregelen vereisen een aanvraag- en goedkeurings-proces**: sommige verzwaringen (dedicated nodes) hebben capaciteits-impact en zijn niet onbeperkt beschikbaar.
- **Doorbelasting** (ADR-G59) moet de verschillende niveaus kunnen factureren.
- **Documentatie en audit**: het maatregelenkatalogus wordt gepubliceerd en jaarlijks herzien in lijn met wijzigingen in BIO, NIS2 en VIRBI.
- **Security-roadmap**: nieuwe maatregelen (bijv. confidential computing, post-quantum crypto) worden aan dit model toegevoegd naarmate de technologie volwassen wordt.
- **Compliance-audits**: de Clouddienst moet aantonen dat de basismaatregelen daadwerkelijk zijn toegepast op alle workloads. Dit vereist een continue compliance-monitoring (via OPA-policies of gelijkwaardig).

Sessienotities

> Alle workloads draaien standaard in een zo goed mogelijke omgeving. Security is geen add-on maar standaard. Classificatiemodel per workload-klasse nodig.

ADR-G41: Logging, monitoring en audittrail: observability-strategie

Status	Brons
Laag	Beveiliging
Afhankelijk van	ADR-C05
Datum	2026-08-07

Context en probleemstelling

Hoe worden logging, monitoring en audittrails ingericht voor NIS2, BIO2

Besluit

Uitgesteld. Richting: platform-eigen SIEM voor de Clouddienst, plus een tenant-eigen keuze.

Motivatie

Waarschijnlijk moet het allemaal kunnen.

Consequenties

Kritiek voor NIS2-compliance. Moet in vervolgsessie worden vastgesteld.

Sessienotities

> De Clouddienst heeft een SIEM en elke tenant kan dat ook hebben. Je moet kunnen kiezen hoeveel je wilt zien. Ook moet je kunnen kiezen om je logging alleen in het platform te houden.

ADR-G49: Standaard managed diensten catalogus

Status	Zilver
Laag	Managed diensten
Maakt mogelijk	ADR-G50, ADR-G51, ADR-G52, ADR-G61
Datum	2026-08-07

Context en probleemstelling

Tenant-teams hebben naast basale compute en storage ook hogere-orde PaaS-diensten nodig: databases, message queues, caches, search-indexen, analytics-platforms. Zonder een catalogus van managed diensten (PaaS) moeten tenants deze zelf opzetten binnen hun cluster, wat de platformwaarde verlaagt (tenants doen werk dat de Clouddienst zou kunnen standaardiseren) en multi-tenant operatie ondergraaft (elke tenant vindt hetzelfde wiel opnieuw uit).

Tegelijk is “alles standaard aanbieden” onmogelijk en onwenselijk: een catalogus met teveel diensten vanaf livegang van de Clouddienst is niet te beheren, niet te onderhouden, en leidt onvermijdelijk tot verwaarloosde diensten die een onderhoudsschuld kunnen worden.

De architectuurvraag is: welk model van managed diensten-catalogus hanteert de Clouddienst, hoe groot is het initiële aanbod, en hoe groeit het?

Drie mogelijke modellen:

1. **Vaste, smalle catalogus**: de Clouddienst selecteert een beperkt aantal diensten (bijv. drie tot vijf) en beheert die volledig. Groeit zeer traag.
2. **Brede catalogus vanaf start**: vanaf livegang een breed aanbod. Risico op onderhoudsschuld.
3. **Groeiende catalogus met governance**: begin klein, groei op basis van tenant-vraag met een toetsingsproces voor nieuwe diensten.

Eisen die de keuze inperken:

- **DevSecOps-bruikbaarheid**: een DevSecOps-team moet er direct mee kunnen werken — eerste diensten moeten aansluiten op de meest gebruikte patronen.
- **Haven/Haven+-conformance** (ADR-G66): Haven+ noemt PostgreSQL, S3-compatible object storage en enkele andere diensten als onderdeel van de standaard.
- **Operationele capaciteit**: elke dienst in de catalogus kost platformteam-capaciteit voor lifecycle, security, support.
- **SEAL-4 / soevereiniteit**: diensten in de catalogus moeten op open source gebaseerd zijn en zelf gehost kunnen worden.

Besluit

De Clouddienst hanteert een **groeiende catalogus met governance**: beginnen met een beperkt aantal diensten die aansluiten op de meest gebruikte patronen, doorgroeien naar een breder aanbod op basis van bewezen tenant-vraag. NB, deze ADR zou nog kunnen wijzigen op basis van het nader op te stellen Operating Model.

Initiële catalogus (v1, sessie- en Haven/Haven+-gedreven):

- Managed PostgreSQL (ADR-G50)
- S3-compatibele object storage (ADR-C06)
- Block en file storage (ADR-C05)
- Container registry (onderdeel van CI/CD-dienst, ADR-G70)
- Secrets management (ADR-G53)

Volgende ronde (v2, op basis van vraag):

- Managed Redis / Valkey (cache)
- Managed Kafka (message queue, ADR-G51)
- Managed observability stack (opt-in logging/metrics-dienst, ADR-G41)
- Managed CI/CD-runners (ADR-G70)

Later (v3+):

- Data-analytics (Trino, Spark-on-Kubernetes, ClickHouse) — op basis van vraag
- Managed MongoDB — alleen als tenant-vraag dit rechtvaardigt (operationele complexiteit vs PostgreSQL)
- Managed Elasticsearch / OpenSearch — op basis van vraag

Onderzochte implementaties

Dit ADR beschrijft het cataloguse-model en -governance, niet de individuele diensten (die volgen in G50, G51, etc.).

Governance-proces voor nieuwe diensten:

1. Tenant-vraag wordt verzameld in een gestandaardiseerd aanvraagproces.
2. De Clouddienst toetst aanvraag: operationele haalbaarheid, open source-beschikbaarheid, bestaande catalogus-dekking, SEAL-4-compatibiliteit.
3. Bij voldoende vraag (bijv. 3+ onafhankelijke tenants) wordt een dienst in een proof-of-concept gezet.
4. Na succesvolle PoC en operationele validatie wordt de dienst toegevoegd aan de catalogus.

Technologiekeuze

Per PaaS-component wordt de technologiekeuze gemaakt in een eigen ADR (G50, G51, etc.). De cataloguse-beheertool zelf is gekoppeld aan het operatorframework (ADR-G61) en de self-service portal (ADR-C10).

Motivatie

“We beginnen met 1 en groeien door” is de enige werkbare strategie. Een brede catalogus vanaf livegang leidt tot onderhoudsschuld en een beeld dat de Clouddienst kan leveren wat het in werkelijkheid niet onderhoudt. Een te smalle catalogus maakt de Clouddienst minder aantrekkelijk dan alternatieven.

Het governance-proces voorkomt dat de catalogus groeit op basis van individuele verzoeken zonder bredere rechtvaardiging. Door dit expliciet te maken wordt voorkomen dat het platformteam ad-hoc moet besluiten. NB, dit kan nog wijzigen op basis van het nader op te stellen Operating Model.

Consequenties

- **Governance-proces nodig** voor het toevoegen van nieuwe diensten: aanvraag, toetsing, PoC, opname. Dit moet worden opgezet en bemand.
- **Lifecycle-beleid per dienst** (ADR-G52): elke dienst in de catalogus valt onder hetzelfde upgrade- en deprecation-beleid.
- **Operatorframework** (ADR-G61) is de technische ruggengraat van de catalogus: elke managed dienst is een operator (of een set operators) die via self-service wordt geïnstantieerd.
- **Documentatie per dienst**: voor elke dienst is een standaard-documentatiepakket vereist (quickstart, operationele runbook, SLA, bekende beperkingen).
- **Deprecation-beleid**: diensten kunnen ook uit de catalogus — als tenant-gebruik te laag is of als de operator onhoudbaar wordt. Dit vereist een deprecation-procedure met migratiepad voor bestaande tenants.
- **Haven/Haven+-conformance**: de initiële catalogus moet aansluiten op de Haven/Haven+-basisvereisten (ADR-G66). NB, Haven+ is op dit moment out of scope.
- **Niet-catalogus-diensten**: tenants mogen zelf diensten in hun cluster installeren die niet in de catalogus staan; dat is hun verantwoordelijkheid. De platform-SLA geldt alleen voor catalogus-diensten.

ADR-G50: Managed database-diensten: scope en engines

Status	Brons
Laag	Managed diensten
Afhankelijk van	ADR-G49
Datum	2026-04-17

Context en probleemstelling

Welke database-engines biedt het platform aan?

Besluit

Database-as-a-Service via self-service operator-catalogus zonder vaste engine-selectie.

Motivatie

[Nog uit te werken]

Consequenties

Operator-framework nodig. Compliance per engine borgen.

ADR-G51: Managed message queue diensten

Status	Voorgesteld
Laag	Managed diensten
Afhankelijk van	ADR-G49
Datum	2026-04-20

Context en probleemstelling

Welke brokertechnologieën als managed dienst? We onderscheiden netwerk en integratie als aparte diensten. Een hogere laag dienst heet ook hogere laag integratiedienst nodig. Een afnemer gaat geen hogere laag dienst afnemen zonder een managed message queue dienst erbij te nemen. Kan in een later stadium verder gespecificeerd worden.

Opletten op diensten die standaard in het platform geregeld moeten worden.

Besluit

Uitgesteld. Eerst requirements vanuit afnemers nodig.

Motivatie

Eerst requirements nodig.

Consequenties

Niet in eerste release.

ADR-G52: Lifecycle en upgradebeleid voor managed diensten

Status	Zilver
Laag	Managed diensten
Afhankelijk van	ADR-G49
Maakt mogelijk	ADR-G62
Datum	2026-03-04

Context en probleemstelling

De managed diensten catalogus (ADR-G49) bevat diensten met verschillende upgradecycli: Kubernetes-versies, operator-versies, onderliggende software-versies (PostgreSQL major-upgrades, Kafka-versies, etc.). Zonder een expliciet upgradebeleid ontstaat versplintering: elke tenant op een andere versie, security-patches die maanden blijven liggen, en onhoudbare operationele last voor het platformteam.

Tegelijk kan het platform niet zonder overleg versies forceren — sommige upgrades (major database-versies) vereisen applicatiewijzigingen aan tenant-zijde. Te strikt = tenants zitten vast met onwerkbare migraties. Te lax = security-schuld stapelt op.

De architectuurvraag is: welk upgradebeleid hanteert het platform voor managed diensten, en hoe wordt de balans gevonden tussen tenant-autonomie en platform-uniformiteit?

Eisen die de keuze inperken:

- **BIO / NIS2**: security-patches moeten binnen redelijke termijn worden doorgevoerd; out-of-support componenten zijn een audit-bevinding.
- **Tenant-werkbaarheid**: tenants hebben planning nodig voor aanpassingen aan hun kant.
- **Operationele schaalbaarheid**: het platformteam kan niet tientallen versies parallel onderhouden.
- **Haven+-lifecyclebeleid**: Haven+ stelt dat “een beperkt aantal versies achtergelopen mag worden, tenzij het security gerelateerde patches betreft voor hoog risico beveiligingsissues”.

Besluit

Het platform hanteert een **gedifferentieerd upgradebeleid per diensttier met een escalerend afdwingingsregime**.

Management plane (niet-onderhandelbaar): Upgrades aan de platformdiensten zelf (Kubernetes-control plane, Cilium, cert-manager, Vault/OpenBao) worden uitgevoerd volgens platform-planning. Rolling updates zodat workloads niet geraakt worden. Tenants worden geïnformeerd maar hebben geen veto.

Workload plane — minor/patch upgrades: Worden door het platform gepland en uitgevoerd binnen een aangekondigd venster (minimaal 2 weken vooraf). Tenants kunnen bezwaar maken met onderbouwing; in uitzonderingsgevallen wordt een maand uitstel gegeven.

Workload plane — major upgrades: Vereisen actieve afstemming met tenants. Aankondiging minimaal 3 maanden vooraf, migratie-windows van 3–6 maanden. Na het venster wordt de oude versie uitgeschakeld.

Security-patches (hoog risico): Overrule alles. Worden binnen 72 uur uitgerold na CVSS ≥ 7.0 CVE-publicatie of PoC-release. Tenants worden geïnformeerd, maar uitstel is niet mogelijk.

Onderzochte implementaties

Dit ADR beschrijft beleid, geen technologiekeuze.

Strikt platformgedreven (gekozen met nuance): Platform bepaalt tempo, tenants krijgen voorspelbare planning. Werkbaar voor management plane, genuanceerd voor workload plane.

Volledig tenant-gedreven: Tenants kiezen zelf wanneer ze upgraden. Leidt tot permanente versplintering; niet haalbaar op platformschaal.

Strikte deadline-handhaving zonder uitzonderingen: Alle tenants moeten binnen X dagen upgraden. Rigoureus maar onrealistisch voor complexe applicaties.

Escalerend regime (gekozen): Eerste aanbod gentle, vervolgens verplicht, uiteindelijk geforceerd. Redelijk voor beide kanten.

Technologiekeuze

Geen aparte technologiekeuze — upgradebeleid is een proces dat wordt ondersteund door het bestaande operatorframework (ADR-G61) en de lifecycle-management-component (ADR-C12).

Motivatie

Platform-gedreven upgrades op management plane zijn noodzakelijk voor security en operationele beheersbaarheid. Een management plane dat per tenant op een andere versie draait is niet te ondersteunen.

Workload plane is genuanceerder omdat tenants hier echte werk-impact ervaren. Door major upgrades ruim aan te kondigen en windows te geven, krijgen tenants de tijd om hun applicaties aan te passen. Door security-patches niet-onderhandelbaar te maken wordt voorkomen dat het platform een verzameling onpatched componenten wordt.

Het escalerend regime (informeren → plannen → afdwingen → uitschakelen) is redelijk voor beide kanten: tenants weten waar ze aan toe zijn, het platform heeft een afdwingingsmechanisme.

Consequenties

- **SLA's per diensttier**: elk diensttier heeft een expliciete upgrade-SLA die deel uitmaakt van het aansluitcontract.
- **Rolling updates voor management plane**: vereist zero-downtime upgrade-patronen. Dit stelt eisen aan elke managed dienst (HA-configuratie, rolling-restart-capaciteit).
- **Communicatieproces**: het platform moet een formeel aankondigingskanaal hebben voor upgrades (mailing list, dashboard in self-service portal). Tenants moeten een vastgelegd contactpunt hebben voor communicatie over upgrades.
- **Migratie-ondersteuning**: bij major upgrades biedt het platform ondersteuning (runbooks, migratietooling waar mogelijk, contactpunt bij platformteam).

- **Monitoring versiegebruik**: het platform moet zicht hebben op welke tenants welke versies draaien. Dit is input voor deprecation-beslissingen en security-respons.
- **Security-patch-procedure**: een gedocumenteerd proces voor snelle security-patch-uitrol (CVE-monitoring, impact-assessment, communicatie, uitrol, verificatie).
- **Versiegrenzen**: maximaal 2 versies achterlopen op hoofdversie, behalve tijdens aangekondigde migratievensters. Deze grens wordt afgedwongen door de escalatie.
- **Deprecation-beleid**: diensten uit de catalogus (ADR-G49) kunnen ook worden uitgefaseerd. Dit volgt hetzelfde escalerende regime met langere tijdlijnen (6–12 maanden).

Sessienotities

Management plane niet onderhandelbaar. Workload plane heeft beperkte ruimte. Bij management plane updates: rolling updates zodat workload plane niet geraakt wordt. Altijd risico, dus goed plannen en communiceren.

ADR-G53: Secrets- en sleutelbeheer: platform-eigen KMS-architectuur

Status	Voorgesteld
Laag	Perifere diensten
Afhankelijk van	ADR-002, ADR-G02, ADR-G10
Maakt mogelijk	ADR-G38, ADR-G54
Datum	2026-08-11

Context en probleemstelling

Elke laag van de Clouddienst heeft secrets nodig: TLS-certificaten, etcd-encryptiesleutels, service-accounttokens, databasewachtwoorden, API-sleutels, SSH-keypairs. Zonder een centrale KMS-architectuur worden deze secrets ad-hoc beheerd opgeslagen in Git, doorgegeven als omgevingsvariabelen, of verspreid over meerdere systemen zonder audittrail. Dit leidt onvermijdelijk tot lekken van gegevens en compliance-problemen.

De architectuurvraag is tweeledig:

1. **Waar worden de encryptiesleutels zelf beheerd?** (de root of trust)
2. **Hoe worden secrets op platform- en tenant-niveau aangemaakt, gedistribueerd en geroteerd?**

- **SEAL-4**: Geen afhankelijkheid van een externe cloudprovider voor sleutelopslag of -generatie.
- **BIO / NIS2**: auditeerbare toegang tot secrets. Elke read/write op een geheim moet gelogd zijn met identiteit, tijdstip en doel.
- **Multi-tenancy**: tenants mogen elkaars secrets niet kunnen lezen, ook niet via een gedeelde KMS-instantie. Sleutels zijn tenant-gebonden.
- **Rotatie**: alle secrets moeten roteerbaar zijn zonder downtime. De Clouddienst mag geen geheimen bevatten met een onbeperkte geldigheidsduur.
- **GitOps-compatibiliteit**: secrets mogen niet als plaintext in Git staan. Het deployment-model (ADR-C12, ADR-C55) vereist een oplossing voor encrypted-at-rest secrets in de repository of een pull-model waarbij pods secrets ophalen zonder ze ooit in Git te hoeven plaatsen.
- De Clouddienst is ingericht voor de standaard rubricering DepV (ADR-G10). Secrets voor BIV 333 dienen in een isolated vault te staan. Dat zou verhoogde beveiliging zijn, waarvoor dus specifieke maatregelen nodig zijn.

Besluit

[Voorgesteld — ter bespreking]

De Clouddienst gebruikt een **open source KMS-laag (HashiCorp Vault of OpenBao) als centrale secrets-engine**, ondersteund door een **hardware-beveiligde root of trust (HSM of cloud-agnostic seal-mechanisme)**. Elke tenant krijgt een eigen namespace binnen de KMS met strikte toegangsgrenzen. De KMS is onderdeel van de Clouddienst en wordt niet uitbesteed aan een externe cloudprovider.

Ook hier duidelijke scheiding voor de infrastructuur onderlaag, en de omgeving van de klant. Omdat deze onder preferere diensten valt, gaat dat echt over de klant omgeving.

Secrets management kan 3 typen vaults bevatten (Central, Local, Isolated). Central vault is de default. Local vault vanwege latency of hoge performance eisen, isolated vault vanwege zeer hoge security eisen. Tenants kunnen zeer diverse eisen stellen aan secrets management. Daarom is één centrale vault onvoldoende voor de Clouddienst.

Onderzochte implementaties

HashiCorp Vault (BSL-licentie): De facto standaard voor KMS in Kubernetes-omgevingen. Brede ecosysteemondersteuning (Vault Agent, Vault Secrets Operator, External Secrets Operator). Ondersteuning voor HSM-backends (PKCS#11), audit logging, dynamische secrets, PKI-engine, SSH-signing. Nadeel: HashiCorp heeft in 2023 de licentie gewijzigd van Apache 2.0 naar BSL (Business Source License). Voor gebruik zonder commercieel alternatief is dit vooralsnog acceptabel, maar het is een soevereiniteitsrisico op termijn.

OpenBao (Apache 2.0, Vault-fork): Community-fork van Vault na de BSL-wijziging, beheerd door de Linux Foundation. API-compatibel met Vault. Nog relatief jong als zelfstandig project (2024), maar groeiende community en actieve ontwikkeling. Voor een overheidsplatform met een lange levensduur is de open licentie een argument.

Kubernetes-native (Secrets Store CSI Driver + externe backends): Koppeling van Kubernetes-pods aan externe secret stores (AWS Secrets Manager, Azure Key Vault, GCP Secret Manager, Vault) via CSI-volumes. Lost het distributieprobleem op maar niet het opslagprobleem — er is nog steeds een backend-KMS nodig. Geen volledige oplossing op zichzelf.

Sealed Secrets (Bitnami): Versleutelt Kubernetes Secrets zodat ze veilig in Git kunnen worden opgeslagen. Lichtgewicht en GitOps-vriendelijk. Beperkte scope: geen dynamische secrets, geen HSM-integratie, geen audit logging, geen cross-cluster rotatie. Geschikt als aanvulling, niet als volledige KMS.

EJBCA + Vault (gecombineerd): EJBCA als PKI-engine (zie ADR-G54) gecombineerd met Vault voor secretsdistributie. Scheidt PKI-beheer van generiek secrets-beheer. Verhoogt operationele complexiteit maar geeft een duidelijkere verantwoordelijkheidsverdeling.

Technologiekeuze

[Nog vast te stellen]

Aanbeveling ter bespreking: **OpenBao als primaire secrets-engine**, met een PKCS#11-HSM backend voor de root seal. Reden: Apache 2.0-licentie past bij de open source-verplichting van de Clouddienst, API-compatibiliteit met Vault verlaagt de migratiedrempel, en Linux Foundation-beheer geeft duurzaamheidsgarantie.

Motivatie

Centraal sleutelbeheer is geen optionele toevoeging. Zonder een expliciete kezuearchitectuurbesluit is er risoco dat secrets op ad-hoc wijze beheerd gaan worden.

De keuze voor een open source KMS met HSM-backend realiseert drie doelen tegelijk: soevereiniteit (root sleutels in eigen beheer), compliance (audittrail per secret), en operationele beheerbaarheid (geautomatiseerde rotatie).

Consequenties

- De Clouddienst moet een HA-KMS-cluster draaien (minimaal 3 nodes, verspreide plaatsing over ODC-locaties).
- Elke platformdienst die secrets gebruikt, moet worden geconfigureerd om secrets dynamisch op te halen via de KMS API, niet als statische environment variables.
- Het HSM-keuzeproces valt onder ADR-G13 (hardware-selectiestrategie) de platformoperator moet een HSM inkopen en beheren die voldoet aan FIPS 140-2 Level 2 of hoger.
- Tenant-namespaces in de KMS moeten worden aangemaakt als onderdeel van het tenant onboarding-proces (ADR-G03).
- PKI-functionaliteit (certificaatuitgifte) valt onder ADR-G54 en kan draaien als Vault/OpenBao PKI-engine.

Sessienotities

Te beslissen: HashiCorp Vault (volwassen, BSL) vs OpenBao (open, jonger). En: hoeveel HSM-hardware en van welke leverancier? Dit raakt ADR-G13.

ADR-G54: PKI-architectuur: certificaatbeheer voor platform en tenants

Status	Voorgesteld
Laag	Perifere diensten
Afhankelijk van	ADR-G53, ADR-002, ADR-G02, ADR-C38
Maakt mogelijk	
Datum	2026-04-13

Context en probleemstelling

Elk TLS-verbinding op het platform — tussen pods, tussen clusters, tussen beheertools en de API server, tussen tenants en platformdiensten — vereist een geldig certificaat. Zonder een expliciete PKI-architectuur groeien certificaten organisch: zelf-gesigneerde certs in etcd, handmatig beheerde certs voor ingress, vergeten verloopdatums, incidenten tijdens rotatie.

De architectuurvraag is: hoe wordt de volledige certificaatketen ingericht, van root CA tot eindcertificaat, voor het platform en voor tenants?

Dit omvat drie niveaus:

1. **Platform-PKI**: certificaten voor interne platformcommunicatie (etcd, API servers, Kubelet, Cilium mTLS, beheerinterfaces).
2. **Tenant-PKI**: tenants die eigen certificaten nodig hebben voor hun workloads (mTLS tussen services, klant-facing TLS).
3. **Vertrouwensketen**: hoe verhouden de overheids-CA-infrastructuur (PKIoverheid) en de platform-CA zich tot elkaar?

Eisen die de keuze inperken:

- **SEAL-4 / soevereiniteit**: Hoeveel partijen willen we certificaten laten leveren? En hoe hard moet dan nageleevd worden op sovreiniteits in de supplyketen?
- **Automatisering**: op een platform van tienduizenden servers is handmatig certificaatbeheer niet schaalbaar. Uitgifte, verlenging en intrekking moeten volledig geautomatiseerd zijn.
- **Korte levensduur**: beveiligingshygiëne schrijft voor dat certificaten een korte geldigheidsduur hebben (90 dagen of korter voor workload-certs). Dit vereist een CA die hoge uitgifte-volumes aankan.
- **Multi-tenancy**: elke tenant heeft een eigen CA of een eigen intermediate CA, zodat tenants elkaars certificaten niet kunnen vertrouwen of uitgifte kunnen triggeren.
- **PKIoverheid-integratie**: overheidswebsites die publiek bereikbaar zijn, vereisen certificaten van een door de overheid erkende CA (PKIoverheid). De platform-PKI moet naast PKIoverheid kunnen bestaan, niet er tegen in.

Besluit

[Voorgesteld — ter bespreking]

Het platform gebruikt een **hiërarchische PKI met een offline root CA en online intermediate CA's per tenant-klasse**, geïmplementeerd via de **PKI-engine van de platform-KMS (ADR-G53) gecombineerd met cert-manager** voor automatische uitgifte in Kubernetes.

De hiërarchie is:

- **Offline root CA** — beheert de root sleutel in HSM (zie ADR-G53), wordt alleen gebruikt om intermediate CA's te ondertekenen.
- **Platform intermediate CA** — voor alle interne platformcertificaten.
- **Tenant intermediate CA's** — per tenant een eigen intermediate, zodat tenants certificaten kunnen uitgifte binnen hun eigen naamruimte zonder toegang tot elkaars CA.

Onderzochte implementaties

cert-manager (Kubernetes-native): De standaard voor certificaatbeheer in Kubernetes. Ondersteunt meerdere issuers (Let's Encrypt via ACME, Vault via de PKI-engine, EJBCA, self-signed). Automatische verlenging, intrekking en rotatie. Breed community-support, CNCF-project. Beperking: cert-manager is een distributie-mechanisme, geen CA — er is altijd een backend CA nodig.

Vault/OpenBao PKI engine: Ingebouwde PKI-functionaliteit in de KMS (ADR-G53). Kan als intermediate CA fungeren, gecombineerd met een externe offline root. Voordeel: één component minder in de stack (KMS én PKI in één). Beperking: minder volwassen dan EJBCA voor complexe PKI-hiërarchieën met CRL/OCSP-distributiepunten.

EJBCA (open source PKI): Volwassen PKI-platform, specifiek gebouwd voor certificaatbeheer. Ondersteunt CRL, OCSP, HSM-integratie, uitgifte-policies per profiel, en complexe hiërarchieën. Gebruikt door overheden in meerdere landen. Nadeel: operationeel zwaarder dan Vault PKI-engine; aparte component in de stack.

Step CA (Smallstep): Lichtgewicht CA, gericht op automatisering en korte levensduur. Native ACME-support, OIDC-integratie, gemakkelijk te integreren met cert-manager. Minder features dan EJBCA voor complexe PKI-vereisten. Geschikt voor platform-intern gebruik.

PKIoverheid als platform-CA: Alle certificaten via de rijks-PKI laten lopen. Niet praktisch voor interne platform-certs (te traag, te duur per cert, geen automatisering voor hoge volumes). Wel van toepassing voor publiek-facing tenant-endpoints.

Technologiekeuze

[Nog vast te stellen]

Aanbeveling ter bespreking: **cert-manager als distributiemechanisme + Vault/OpenBao PKI-engine als online intermediate CA + offline root CA op HSM**. Dit sluit aan op de KMS-keuze uit ADR-G53 (één component minder) en op het bestaande Kubernetes-ecosysteem. EJBCA als alternatief als de PKI-vereisten van de overheid complexere CRL/OCSP-distributiepunten vereisen.

Motivatie

Een geautomatiseerde, hiërarchische PKI is een basisvereiste voor mTLS (ADR-C38), voor veilige etcd-communicatie, en voor de vertrouwensketen richting SEAL-4. Zonder dit is elk “alles is versleuteld”-statement niet aantoonbaar.

De koppeling aan de bestaande KMS-infrastructuur (ADR-G53) vermijdt operationele duplicatie: het sleutelbeheer zit al in de KMS, de PKI-engine is een logische uitbreiding daarvan.

Consequenties

- Er moet een procedure worden opgesteld voor het genereren en bewaren van de offline root CA-sleutel (key ceremony). Dit is een formeel proces met meerdere aanwezigen.
- cert-manager wordt onderdeel van de platform-baseline (net als Cilium, Rook/Ceph): elke cluster krijgt cert-manager geïnstalleerd.
- Tenant onboarding (ADR-G03) omvat het aanmaken van een tenant-specifieke intermediate CA.
- Publiek-facing certificaten voor tenant-workloads die under PKIoverheid moeten vallen, vereisen een aparte uitgifte-workflow. Dit moet worden uitgewerkt in samenwerking met Logius.
- CRL/OCSP-distributiepunten moeten bereikbaar zijn vanuit alle ODC-locaties en voor tenants.

Sessienotities

> Open vraag: hoe verhoudt de platform-PKI zich tot PKIoverheid? Moeten tenant-facing certs altijd van PKIoverheid komen, of is een platform-CA voldoende voor interne overheidsgebruikers? Dit raakt beleidsgebied buiten de architectuur.

ADR-G55: DNS-architectuur: naamgeving, resolving en tenant-scoping

Status	Goud
Laag	Perifere diensten
Afhankelijk van	ADR-G02, ADR-C04, ADR-G53, ADR-C07
Maakt mogelijk	
Datum	2026-07-07

Context en probleemstelling

DNS is de adresboek-infrastructuur van het hele platform. Elk Kubernetes-cluster, elke service, elke tenant-workload en elke beheercomponent heeft een naam nodig die op het juiste IP-adres uitkomt. Op een multi-tenant platform met meerdere ODC-locaties en integraties op overheidsnetwerken (Diginetwerk, NAFIN, RON) is DNS een belangrijk onderdeel.

De architectuurvraag is: hoe wordt DNS ingericht voor platform-intern gebruik, voor tenants, en voor de koppeling met bestaande overheidsnetwerken — op een manier die isolatie, schaalbaarheid en soevereiniteit garandeert?

Concrete deelvragen:

1. **Interne platform-DNS**: hoe resolven platformcomponenten elkaar (etcd, Hypershift control planes, beheertools)?
2. **Tenant-DNS**: hoe resolven services binnen een tenant-cluster, en hoe worden tenant-domeinen beheerd?
3. **Split-horizon**: dezelfde naam, verschillende antwoorden afhankelijk van wie vraagt (intern vs. extern, of tenant A vs. tenant B).
4. **Externe koppeling**: hoe integreert platform-DNS met overheids-DNS-infrastructuur (Diginetwerk, NAFIN, publiek internet)?
5. **Naamgevingsconventie**: welke domeinhiërarchie hanteert het platform?

Eisen die de keuze inperken:

- **Multi-tenancy / isolatie**: tenant A mag geen DNS-queries kunnen uitvoeren die informatie geven over de naamruimte van tenant B. DNS-lekken (zone walking, poisoning) zijn een reëel risico in gedeelde resolvers.
- **SEAL-4 / soevereiniteit**: de primaire DNS-infrastructuur moet in eigen beheer zijn. Afhankelijkheid van een externe DNS-provider (Cloudflare, Google 8.8.8.8) voor interne resolutie is niet in lijn met de doelstelling.
- **Automatisering**: op een platform met tientallen clusters en honderden tenants is handmatig DNS-beheer niet schaalbaar. DNS-records moeten automatisch worden aangemaakt bij het inrichten van services en clusters.
- **Overheidsnetwerken**: het platform moet DNS-zones kunnen serveren voor domeinen op Diginetwerk/NAFIN, naast of in plaats van publiek-internetdomeinen.
- **Hoge beschikbaarheid**: DNS is kritieke infrastructuur. Uitval leidt tot volledige onbereikbaarheid. Minimaal 3 resolvers per locatie, verspreide plaatsing.

Besluit

Het platform gebruikt een **gelaagd DNS-model met tenant-gescopede resolvers per cluster en een centrale platform-DNS-laag** voor platformbrede naamgeving. External-dns automatiseert het aanmaken van DNS-records op basis van Kubernetes-resources. CoreDNS is de cluster-interne resolver; een platform-owned DNS-service (Bind9 of PowerDNS) serveert de authoritative zones.

Onderzochte implementaties

CoreDNS (Kubernetes-native): De standaard cluster-interne DNS-resolver in Kubernetes. Wordt bij elk cluster meegeleverd via kubeadm/Hypershift. Pluggable architectuur (forward, cache, rewrite, health). Beperking: CoreDNS is een cluster-interne resolver het lost op binnen een cluster maar is geen autoritatieve nameserver voor het hele platform. Aanvullende oplossing nodig voor cross-cluster en platformbrede naamgeving.

external-dns (Kubernetes-operator): Automatiseert het aanmaken en verwijderen van DNS-records op basis van Kubernetes Service- en Ingress-resources. Ondersteunt meerdere DNS-backends (Route53, Cloudflare, PowerDNS, Bind9, etc.). CNCF-project, brede adoptie. Essentieel voor automatisering — zonder external-dns moet elke nieuwe service handmatig worden geregistreerd.

PowerDNS (open source): Autoritatieve nameserver met PostgreSQL/MySQL-backend, REST API voor programmatisch beheer en externe-dns-integratie. Actief onderhouden, breed gebruikt bij overheden en ISPs. Nadeel: vereist een databaselaag voor de zone-opslag.

BIND9: De meest verbreide autoritatieve nameserver ter wereld. Stabiel, bewezen, breed gedocumenteerd. Beperking: traditioneel op basis van zonefiles, minder geschikt voor dynamisch beheer via API tenzij aangevuld met tooling. Minder cloud-native dan PowerDNS.

Knot DNS: Performante open source autoritatieve nameserver, gebouwd door CZ.NIC (Tsjechische overheids-CA). Geen resolverfunctionaliteit (puur autoritatief). Interessant vanuit het perspectief van EU-herkomst.

Split-horizon via CoreDNS rewrite-plugin: Dezelfde hostnaam kan naar verschillende backends worden gerouteerd op basis van de bron van de query. Maakt het mogelijk interne adressen te serveren aan interne resolvers en publieke adressen aan externe resolvers, zonder aparte zonefiles.

Technologiekeuze

CoreDNS per cluster (standaard) + PowerDNS als platform-autoritatieve nameserver + external-dns voor automatische recordbeheer. Split-horizon via forwarding-regels in CoreDNS: intern verkeer wordt doorgestuurd naar de platform-DNS, met een eigen recursive resolver.

Motivatie

DNS is de meest onderschatte infrastructurele component. Elk ander ADR veronderstelt werkende DNS. Een expliciete architectuurkeuze voorkomt dat DNS organisch groeit tot een onbeheersbare mix van handmatige records, vergeten zones en hardgecodeerde IP-adressen.

Automatisering via external-dns is niet optioneel op een platform van deze schaal: met tienduizenden services is handmatig recordbeheer onmogelijk.

Consequenties

- Het platform heeft authoritative DNS-zones nodig voor interne domeinen.
- Elke tenant-cluster krijgt CoreDNS geconfigureerd met forwarding-regels naar de platform-DNS voor cross-cluster resolutie.
- DNS-servers moeten per ODC-locatie aanwezig zijn (geen cross-locatie DNS als single point of failure).
- Tenants die eigen domeinen inbrengen, moeten een delegatie-mechanisme krijgen — dit vereist uitwerking in de tenant-onboarding-procedure (ADR-G03).
- DNSSEC is een vereiste voor publiek-bereikbare zones (overheidsbeleid). De platform-PKI (ADR-G54) moet DNSSEC-sleutelbeheer ondersteunen of daarvoor wordt een aparte voorziening getroffen.

ADR-G56: Datacenter network fabric: underlay architectuur en switch-automatisering

Status	Voorgesteld
Laag	Networking
Afhankelijk van	ADR-G01, ADR-C07, ADR-G05, ADR-C60
Maakt mogelijk	ADR-G57
Datum	2026-06-03

Context en probleemstelling

Het platform draait op bare-metal servers verspreid over minimaal drie ODC-locaties. Alle IP-verkeer tussen servers, tussen clusters en tussen locaties loopt over fysieke switches. Die switches vormen samen de underlay: het fundament waarop Cilium (ADR-C07) zijn overlay bouwt. Zonder een expliciete underlay-architectuur groeit het netwerk organisch en zijn beheer, schaalbaarheid en storingsdiagnose onmogelijk op schaal.

De architectuurvraag is: welk fysiek netwerktopologie-model en welke mate van automatisering worden gehanteerd voor de datacenter-fabric van het platform?

Concrete deelvragen:

- Spine-leaf of traditionele driesterren-topologie?
- Hoe wordt BGP-peering met Cilium georganiseerd (ADR-C07 veronderstelt BGP-integratie)?
- Hoe worden switches geautomatiseerd geconfigureerd (zero-touch provisioning)?
- Welke bandbreedtevereisten gelden per servertype (compute, storage, GPU)?
- Hoe wordt oost-west-verkeer (tussen servers in zelfde ODC) van noord-zuid-verkeer (naar buiten) gescheiden?

Overwegingen:

- **Spine-leaf**: flat two-tier topologie, alle servers op gelijke afstand, schaalt lineair, standaard in hyperscale- en carrier-omgevingen. Vereist meer switches per ODC dan traditionele topologie.
- **Multi-vendor netwerkhardware** (zie ook ADR-G65): als het platform meerdere switchleveranciers ondersteunt, moet de fabric-automatisering leveranciersonafhankelijk zijn.
- **Soevereiniteit**: SONiC (open source NOS van Microsoft, nu Linux Foundation) biedt een leveranciersonafhankelijke network operating system op commodity-switches. Alternatieven zijn vendor-specifieke NOS (Arista EOS, Juniper JunOS).
- **Provisioning**: Onafhankelijke netwerk-fabric-ADR's komen grotendeels te vervallen: metal-stack (ADR-C60) levert dit.

Besluit

[Nog geen besluit — voorgelegd ter bespreking]

Onderzochte implementaties

Spine-leaf met SONiC: Twee-tier topologie op commodity-hardware (Broadcom Trident/Tomahawk chipsets) met SONiC als NOS. Volledig open source, leveranciersonafhankelijk, brede adoptie bij hyperscalers. Beheer via Ansible/Netbox. BGP-underlay sluit direct aan op Cilium BGP Control Plane (ADR-C07).

Spine-leaf met vendor-NOS (Arista/Juniper): Zelfde topologie maar met gelicenseerde NOS. Rijpere tooling, betere enterprise-support, maar vendor lock-in op NOS-niveau. Arista EOS en CVX zijn breed beschikbaar bij Nederlandse ODC-aanbieders.

Drielaags traditioneel (core/distribution/access): Bewezen topologie, hogere latentie oost-west, minder schaalbaar voor east-west heavy Kubernetes-workloads. Niet aanbevolen voor nieuwe opbouw.

Technologiekeuze

[Nog vast te stellen]

Motivatie

Een geautomatiseerde, consistente underlay is een basisvereiste voor betrouwbare BGP-peering met Cilium en voor zero-touch provisioning van servers (ADR-C01).

Consequenties

- Keuze voor NOS bepaalt grotendeels de aanbestedingsstrategie voor netwerkhardware.
- Switch-automatisering (Netbox + Ansible of NAPALM) moet worden geïntegreerd in de bootstrap-pipeline (ADR-G60).
- Bandbreedte-eisen per tier moeten worden vastgesteld op basis van de compute- en storage-configuratie.

Sessienotities

> Kernvraag: open NOS (SONiC) vs. vendor-NOS met betere support? En: is commodity-hardware voldoende voor ODC-eisen, of zijn er certificeringseisen die vendor-hardware vereisen?

ADR-G57: Firewall architectuur segmentatie en filtering op platform- en tenant-niveau

Status	Voorgesteld
Laag	Networking
Afhankelijk van	ADR-C07, ADR-G02, ADR-G10, ADR-G56
Maakt mogelijk	ADR-G38
Datum	2026-04-20

Context en probleemstelling

De opdracht noemt firewalls expliciet als technische afhankelijkheid die moet worden uitgewerkt. Op een multi-tenant platform met workloads tot DEP VERTROUWELIJK (ADR-G10) is netwerksegmentatie en -filtering een beveiligingsvereiste.

De architectuurvraag is: hoe worden firewalls ingericht voor het platform: op welk niveau, door wie beheerd, en hoe verhouden platform-firewalls zich tot tenant-firewalls?

De complexiteit zit in de gelaagdheid: Cilium (ADR-C07) biedt al L3/L4 NetworkPolicies en L7-filtering via service mesh (ADR-C38). De vraag is welke aanvullende firewalling nodig is, en op welk niveau (fysiek, hypervisor, software-defined).

Niveaus waarop filtering kan plaatsvinden:

1. **Fysieke switches** (ACL's op underlay — ADR-G56): grof, snel, beperkte flexibiliteit
2. **Dedicated firewall-appliances** (voor ODC-perimeter en inter-ODC): voor noord-zuid-verkeer
3. **Cilium NetworkPolicies** (L3/L4, per pod/namespace/cluster): platform-beheerd
4. **Cilium service mesh / Istio** (L7, per service): optioneel tenant-beheerd (ADR-C38)
5. **Tenant-eigen firewall** (virtuele firewall als managed dienst): voor tenants met hoge isolatie-eisen

Overwegingen:

- BIO en NIS2 vereisen aantoonbare netwerksegmentatie en logging van gefilterd verkeer.
- SEAL-4 impliceert dat de platform-operator de firewallregels kan inzien en auditen.
- Tenants met DEP VERTROUWELIJK workloads kunnen aanvullende isolatie-eisen hebben.
- Een dedicated hardware firewall-appliance introduceert vendor-afhankelijkheid — past dit bij de open source-strategie?

Besluit

[Nog geen besluit — voorgelegd ter bespreking]

Onderzochte implementaties

Cilium als primaire firewall (software-defined): Cilium NetworkPolicies als vervanging van traditionele firewalls voor oost-west-verkeer. Perimeter-filtering (noord-zuid) via een open source software firewall (OPNsense, pfSense, of Linux nftables). Voordeel: volledig geautomatiseerd, GitOps-beheerd, geen vendor lock-in. Nadeel: minder vertrouwd bij security-auditors gewend aan hardware firewalls.

Hardware perimeter-firewall + Cilium intern: Dedicated firewall-appliances (Palo Alto, Fortinet, CheckPoint) voor ODC-perimeter en inter-ODC verkeer. Cilium voor intern oost-west. Voordeel: bewezen enterprise-model, breed geaccepteerd in overheidsaudits. Nadeel: vendor lock-in, hogere kosten, minder automatiseerbaar.

Virtuele firewall als managed dienst per tenant: Tenants die hogere isolatie nodig hebben krijgen een virtuele firewall-instantie (bijv. Palo Alto VM-Series of open source). Verhoogt isolatie maar ook operationele complexiteit.

Technologiekeuze

[Nog vast te stellen] Mix is mogelijk dat voor sommige alles via virtuele FW kan met enkel Cilium voor andere workloads meerdere type firewalls nodig zijn.

Motivatie

Expliciete firewallarchitectuur is vereist door de opdracht en door BIO/NIS2. De keuze bepaalt hoeveel van de filtering software-defined (Cilium) versus hardware-gebaseerd is.

Consequenties

- Firewallregels voor het platform moeten onderdeel worden van de GitOps-pipeline (ADR-C55).
- Logging van firewall-events moet worden gekoppeld aan de observability-stack (ADR-G41) en SOC-integratie (ADR-G63).
- Voor tenants met DEP VERTROUWELIJK workloads moet een aanvullende segmentatiestrategie worden uitgewerkt (ADR-G38).

Sessienotities

> Kernvraag: accepteert de overheidsauditor Cilium als volwaardige vervanging van een hardware firewall voor DEP VERTROUWELIJK? Of is een dedicated hardware perimeter verplicht?

Er zullen meerdere lagen moeten zijn. Buiten firewall met DDoS filter, dan eigen buiten muur met firewalls, dan tussen netwerken zitten allemaal eigen firewalls zodat data nooit van hoog naar laag vertrouwlijk kan stromen. WAP zit er los tussen, L4 zit er los tussen. Zowel verticale als horizontale firewalling.

Patroon herkenning ook in de platform onder laag.

Schaalbaarheid, voor het platform onderlaag IaaS hoe zorgen we dat die laag voldoende schaalbaar is en alle traffic hier doorheen kan. Ook hier moet security eisen goed afgedekt kunnen worden voor de klanten van de overheid (gaan voor 80%).

ADR-G58: Backup en disaster recovery strategie

Status	Voorgesteld
Laag	Overkoepelend
Afhankelijk van	ADR-C05, ADR-G01, ADR-G62
Maakt mogelijk	ADR-G38
Datum	2026-04-13

Context en probleemstelling

De opdracht vereist expliciet een uitwerking van de “backup-restore-omgeving inclusief processen en procedures”. Zonder een platformbrede backup- en DR-strategie kunnen tenants geen zinvolle SLA’s afspreken en voldoet het platform niet aan BIO-basiseisen voor beschikbaarheid en herstel.

De architectuurvraag is: welke backup- en disaster recovery-mechanismen biedt het platform, op welk niveau, en wat is de verantwoordelijkheidsverdeling tussen platform en tenant?

Drie lagen vereisen een antwoord:

1. **Platform-data** (etcd, Kubernetes-state, Vault/KMS-data, DNS-zones): verlies hiervan is fataal voor het hele platform.
2. **Persistent volumes van tenants** (Ceph block/file/object — ADR-C05/C06): de data van tenant-workloads.
3. **Applicatie-consistente backups**: niet alle data-herstel is voldoende op volume-niveau; databases vereisen applicatie-bewuste snapshots.

Overwegingen:

- Minimaal 3 ODC-locaties (ADR-G01) biedt geografische spreiding, maar replicatie ≠ backup.
- RTO (Recovery Time Objective) en RPO (Recovery Point Objective) zijn nog niet vastgesteld — dit hangt af van ADR-G62.
- Tenants zijn zelf verantwoordelijk voor applicatie-consistente backups, maar het platform moet de infrastructuur bieden.
- SEAL-4: backup-data moet onder dezelfde soevereiniteitseisen vallen als primaire data.

Besluit

[Nog geen besluit — voorgelegd ter bespreking]

Onderzochte implementaties

Velero (Kubernetes-native backup): Open source tool voor back-up van Kubernetes-resources en persistent volumes. Integreert met Ceph (via CSI snapshots) en S3-compatibele object storage (ADR-C06). CNCF-project, brede adoptie. Beperking: applicatie-consistentie vereist aanvullende hooks per workload.

Ceph native snapshots + replicatie: Ceph ondersteunt volume-snapshots en cross-cluster replicatie (RBD mirroring). Snel, storage-laag native. Beperking: geen Kubernetes-resource backup; alleen data-volumes.

Kasten K10 (Veeam): Commercieel product specifiek voor Kubernetes backup. Rijk aan features (applicatie-consistent, policy-gedreven, multi-cluster). Nadeel: commerciële licentie, vendor lock-in.

Combinatie Velero + Ceph RBD mirroring: Platform-eigen combinatie die volume-replicatie (Ceph) combineert met Kubernetes-state backup (Velero). Geen vendor lock-in, vereist operationele kennis van beide tools.

Technologiekeuze

[Nog vast te stellen]

Motivatie

Backup en DR zijn een expliciete opdrachteis en een BIO-basisvereiste. Zonder dit is geen enkele SLA-toezegging geloofwaardig.

Consequenties

- RTO/RPO-doelen moeten worden vastgesteld (ADR-G62) voordat toolkeuze definitief is.
- Back-updata moet worden opgeslagen op een geografisch gescheiden locatie (minstens één van de drie ODC's).
- Tenant-onboarding (ADR-G03) moet informatie bevatten over de verantwoordelijkheidsverdeling backup platform vs. tenant.
- Backup-policies moeten onderdeel worden van de managed diensten catalogus (ADR-G49).

Sessienotities

> Wat biedt het platform standaard, en wat is opt-in voor tenants? En: welk RPO is realistisch op Ceph-replicatie?

ADR-G59: Gebruiksgebaseerde dienstverlening (metering)

Status	Voorgesteld
Laag	Perifere diensten
Afhankelijk van	ADR-G02, ADR-C04, ADR-G49
Maakt mogelijk	
Datum	2026-08-11

Context en probleemstelling

De opdracht vereist waarden die moeten worden gemeten om doorbelasting te kunnen ondersteunen. Het gaat dus niet om de kosten van een waarde, maar enkel om de waarde die kan worden gemeten voor eventuele doorbelasting. Het financiële model is onderdeel van het Operating Model, dus zal later pas nader beschreven worden. De domeinarchitectuur (MIDO) stelt “gebruiksgebaseerde dienstverlening” als een van de vijf NIST-kenmerken van cloud: gebruik moet automatisch worden gemeten en transparant zijn voor aanbieder en afnemer.

Zonder metering kan de Clouddienst geen kostentransparantie bieden, kunnen tenants desgewenst niet worden afgerekend op gebruik, en kan de bedrijfsvoering van de Clouddienst niet een breed pallet mogelijke financiële modellen ondersteunen.

De architectuurvraag is tweeledig:

1. **Wat wordt gemeten?** Welke eenheden worden per laag bijgehouden?
2. **Hoe wordt gemeten?** Welke tooling verzorgt metering, aggregatie en rapportage?

Meetbare eenheden per laag (minimale set):

- **Compute**: vCPU-uur per cluster/namespace, geheugen (GB·uur)
- **Storage**: opslag (GB·maand) per volume-type (block, file, object), I/O-operaties
- **Netwerk**: ingress/egress datavolume (GB) per tenant, inter-ODC-verkeer
- **GPU**: GPU-uur per GPU-type
- **Managed diensten**: instanties × tier × uur (database, queue, etc.)

Overwegingen:

- Overheidsbekostiging werkt op basis van meerjarenbegroting, niet pay-per-use. Metering hoeft niet real-time te zijn maar wel periodiek (maandelijks/kwartaal) en auditeerbaar.
- Gemeten dienstverlening is geen commercieel facturatiesysteem maar een administratief verantwoordingsinstrument.
- De domeinarchitectuur noemt een “clearinghouse” voor gecoördineerde bekostiging — de Clouddienst moet hierop kunnen aansluiten afhankelijk van toekomstige besluitvorming hierover n.a.v. het Operating Model.
- Metering-data mag niet als single point of failure werken: als de metering uitvalt, blijven workloads draaien.
- FinOps tagging op resource niveau is noodzakelijk

Besluit

Om te komen tot een realiseerbare gebruiksgebaseerde dienstverlening en inzicht voor een afnemer te verschaffen, is het noodzakelijk om een traceerbare tagging strategie te hanteren vanaf de feitelijke resources. Alle resources dienen een tagging mogelijkheid te ondersteunen en er dient een vastgestelde taggingstrategie te worden gehanteerd.

[Nog geen besluit — voorgelegd ter bespreking]

Onderzochte implementaties

OpenCost (CNCF): Open source Kubernetes-kostentoewijzingstool. Toewijzing van kosten per namespace, label en cluster. Integreert met Prometheus. Kan worden uitgebreid met custom pricing-modellen. Brede adoptie, CNCF-sandbox project.

Kube-opex-scheduler / Prometheus + custom dashboards: Metering via Prometheus-metrics (kube-state-metrics, node-exporter, ceph-exporter) gecombineerd met Grafana-dashboards en custom aggregatielogica. Maximale flexibiliteit, maar meer zelfbouw.

CloudZero / Apptio (commercieel): Commerciële FinOps-platforms met rijke rapportage. Niet passend voor een soeverein platform dat geen data naar externe SaaS-diensten stuurt.

Odoo / OpenERP integratie: Voor overheidsboekhouding-integratie. Buiten scope platformarchitectuur maar relevant voor de aansluiting op financiële systemen van afnemers.

Technologiekeuze

[Nog vast te stellen]

Motivatie

Gemeten dienstverlening is een expliciete opdrachteis en een NIST-cloudkenmerk.

Consequenties

- Per tenant-onboarding (ADR-G03) moeten kostencodes en rapportage-ontvangers worden vastgelegd.
- Metering-data moet minimaal 7 jaar worden bewaard (overheidsadministratieplicht).
- De observability-stack (ADR-G41) en metering-stack delen infrastructuur (Prometheus) maar hebben gescheiden bewaar- en toegangsbeleid.

Sessienotities

> Kernvraag: welke granulariteit is bestuurlijk vereist (namespace, cluster, organisatie)? En: sluit het de Clouddienst aan op een centraal clearinghouse of is elke organisatie zelf verantwoordelijk voor hun eigen doorbelasting aan interne afdelingen? Dit zal worden her-ijkt zodra besluitvorming over het Operating Model duidelijkheid biedt over het financiële model.

ADR-G60: Bootstrap proces en Git-platform voor platformbeheer

Status	Voorgesteld
Laag	Perifere diensten
Afhankelijk van	ADR-C01, ADR-C02, ADR-C23, ADR-C55
Maakt mogelijk	
Datum	2026-07-29

Context en probleemstelling

De opdracht vereist een uitwerking van het bootstrapproces en de Git-omgeving. Dit zijn de twee meest fundamentele vragen van een GitOps-platform: hoe wordt de allereerste installatie uitgevoerd (bootstrap), en welke Git-infrastructuur beheert de gewenste toestand van het platform?

Het bootstrapprobleem is klassiek: om een cluster te draaien heb je een cluster nodig. Kubernetes moet ergens vandaan komen voordat het zichzelf kan beheren. En de GitOps-reconciliator (ArgoCD of Flux — ADR-C55) heeft een Git-server nodig voordat die Git-server zelf op het platform kan draaien.

De architectuurvraag is:

1. **Bootstrap**: hoe wordt een leeg datacenter omgezet in een werkend management-cluster zonder handmatige tussenkomst?
2. **Git-platform**: welke Git-server beheert de platformconfiguratie, en hoe is deze gehost?

Overwegingen:

- SEAL-4: de Git-server die platformconfiguratie beheert mag geen externe afhankelijkheid zijn (geen GitHub, geen GitLab SaaS). Het moet een zelf-gehoste instantie zijn.
- **Chicken-and-egg bootstrap**: het eerste management-cluster moet worden opgestart zonder dat er al een management-cluster is. Dit vereist een tijdelijke “seed”-omgeving of een bootstrap-aanpak zoals Cluster API’s pivot-mechanisme.
- **Immutable OS** (ADR-C02, Talos Linux): de bootstrap-procedure voor Talos is API-driven, niet via SSH. Dit heeft gevolgen voor hoe het bootstrapproces werkt.
- **Multi-ODC**: het bootstrap-proces moet herhaalbaar zijn voor elke nieuwe ODC-locatie.

Besluit

[Nog geen besluit — voorgelegd ter bespreking]

Onderzochte implementaties

Talos + CAPI bootstrap (pivot-aanpak): Eerste cluster wordt opgezet via een tijdelijke lokale machine (laptop of jumphost) met `talosctl` en `clusterctl`. Na opstart van het management-cluster wordt de CAPI-state “gepivot” naar het management-cluster zelf. Volledig declaratief, geen SSH vereist.

Metal³ + PXE bootstrap: Servers booten via PXE en ontvangen hun Talos-configuratie via de Metal³-provisioning-API. Vereist een initiële Metal³-instantie die op een kleine seed-server draait (Raspberry Pi of vergelijkbare jumphost). Schaalbaar voor grote aantallen servers.

Gitea (zelf-gehost, Go): Lichtgewicht open source Git-server. Lage resource-overhead, geschikt voor platform-interne Git. Minder features dan GitLab maar voldoende voor GitOps-use-case.

GitLab CE (self-hosted): Rijker platform (CI/CD, container registry, issue tracking). Hogere resource-overhead. Kan ook worden gebruikt als CI/CD-platform (ADR-G70).

Forgejo (Gitea-fork, open governance): Fork van Gitea met meer community-governance. Functioneel vergelijkbaar.

Technologiekeuze

Voor de GIT omgeving in het platform wordt gebruik gemaakt van Forgejo voor infrastructuur zaken.

Motivatie

Zonder bootstrap-procedure en interne Git-server kan het platform niet onafhankelijk van externe services draaien — wat een directe SEAL-4-vereiste raakt.

Consequenties

- Het bootstrap-proces moet worden gedocumenteerd als runbook en getest in een lab-omgeving voordat productie-uitrol.
- De interne Git-server is een kritieke platformcomponent: HA-configuratie vereist.
- Toegang tot de interne Git-server moet worden beheerd via het IAM-systeem (ADR-G06).

Sessienotities

> Kernvraag: Gitea/Forgejo (licht, simpel) vs. GitLab CE (rijk, zwaarder)? En: combineert het Git-platform met CI/CD (ADR-G70) of zijn dit gescheiden diensten? Voor de echte “core” van het platform, houden we een break glass procecure waarbij er altijd met een laptop in het datacenter de eerste bootstrap van de core infrastructuur gedaan kan worden.

ADR-G61: Operatorframework: beheer van complexe managed diensten

Status	Voorgesteld
Laag	Managed diensten
Afhankelijk van	ADR-C12, ADR-C55, ADR-G49
Maakt mogelijk	ADR-G50, ADR-G51
Datum	2026-04-20

Context en probleemstelling

De opdracht vereist expliciet een uitwerking van het operatorframework. Managed diensten zoals databases, message queues en caching zijn niet te beheren met simpele Kubernetes-Deployments: ze hebben een eigen lifecycle (primary/replica-failover, schaling, backup, upgrade, configuratie-tuning). Het Kubernetes Operator-patroon is de standaardaanpak om deze complexiteit declaratief te beheren.

De architectuurvraag is: welk framework of welke aanpak wordt gehanteerd voor het bouwen, beheren en publiceren van operators op het platform — en hoe worden operators door tenants gebruikt?

Concrete vragen:

- Bouwt het platformteam eigen operators, of worden bestaande operators uit de community gebruikt?
- Via welk mechanisme worden operators beschikbaar gesteld aan tenants?
- Hoe worden operators bijgewerkt zonder downtime van de managed dienst?
- Welke kwaliteitseisen (certificering, support, lifecycle) gelden voor operators in de catalogus?

Overwegingen:

- De Operator Lifecycle Manager (OLM) is de standaard voor operator-distributie in OpenShift-omgevingen en wordt breed gebruikt in de CNCF-ecosysteem.
- Operators zijn code: ze vereisen onderhoud, beveiligingsupdates en compatibiliteitstests met nieuwe Kubernetes-versies.
- Niet elk overheidsplatformteam heeft de capaciteit om eigen operators te schrijven en te onderhouden.

Besluit

[Nog geen besluit — voorgelegd ter bespreking]

Onderzochte implementaties

Operator Lifecycle Manager (OLM) + OperatorHub: OLM beheert de installatie, updates en afhankelijkheden van operators. OperatorHub.io biedt een catalogus van gecertificeerde operators. Breed gebruikt, maar een OperatorHub-afhankelijkheid introduceert externe connectiviteit — een interne mirror is nodig voor SEAL-4.

Helm + Argo CD (GitOps-operator-distributie): Operators worden als Helm-charts beschikbaar gesteld via de interne Git-server en uitgerold via ArgoCD/Flux (ADR-C55). Eenvoudiger dan OLM, maar minder automatisch lifecycle-beheer.

Crossplane (control plane framework): Crossplane maakt het mogelijk om cloud-resources (ook interne) te beheren via Kubernetes Custom Resources. Kan worden gebruikt als alternatief voor vendor-specifieke operators. Complexer dan Helm maar krachtiger voor composable infrastructure.

Bestaande community-operators (Zalando, Percona, Strimzi): PostgreSQL (CloudNativePG of Zalando), MySQL (Percona), Kafka (Strimzi), Redis (Redis Operator). Volwassen, actief onderhouden, geen eigen bouw vereist. Afhankelijkheid van community-support en -roadmap.

Technologiekeuze

[Nog vast te stellen]

Motivatie

Operators zijn een schaalbare manier om complexe stateful diensten (databases, queues, etc...) te beheren op een platform van deze omvang. Zonder operatorframework wordt managed dienst-beheer ad-hoc en moelijker beheersbaar.

Consequenties

- Het platformteam moet een toetsingsproces definiëren voor operators die worden opgenomen in de catalogus (ADR-G49). t.w. LCM en versiebeheer, uitrolprocedures, ...
- Operators moeten worden gespiegeld in de interne container registry om externe afhankelijkheden te vermijden.
- Tenant-toegang tot operators moet worden beheerd via het IAM-systeem en de resource-hiërarchie (ADR-G67).
- Logging en metrics van operators moeten beschikbaar zijn voor gebruikers.

Sessienotities

> Kernvraag: OLM (rijker, complexer) vs. Helm/GitOps (simpeler, voldoende)? En: welke operators zijn in scope voor v1 van de catalogus?

ADR-G62: Non-functionele eisen: beschikbaarheid, RTO/RPO en capaciteitslimieten

Status	Voorgesteld
Laag	Overkoepelend
Afhankelijk van	ADR-001, ADR-G01
Maakt mogelijk	ADR-G58, ADR-G64
Datum	2026-04-20

Context en probleemstelling

De opdracht vereist een uitwerking van schaalbaarheid per laag inclusief effecten, en doorsnijdende thema's die voor elke laag moeten worden uitgewerkt. Zonder vastgestelde non-functionele eisen zijn alle andere architectuurkeuzes onverankerd: er is geen toetsing mogelijk of een keuze goed genoeg is.

Non-functionele eisen zijn de grenzen waarbinnen het platform moet opereren. Ze zijn de contractuele basis voor tenant-SLA's en de ontwerpinput voor de architectuur.

De architectuurvraag is: welke beschikbaarheids-, herstel- en capaciteitsdoelen gelden voor het platform, per laag?

Categorieën:

1. **Beschikbaarheid**: welk uptime-percentage wordt nagestreefd per dienst? (99,9% = 8,7u/jaar downtime; 99,99% = 52m/jaar)
2. **RTO** (Recovery Time Objective): hoelang mag het duren voordat een dienst hersteld is na een storing?
3. **RPO** (Recovery Point Objective): hoeveel dataverlies is acceptabel bij een storing?
4. **Capaciteitslimieten**: wat is de maximale schaling per laag, en wat zijn de effecten als die grens bereikt wordt?
5. **Degraded mode**: hoe gedraagt het platform zich als één ODC-locatie uitvalt?

Overwegingen:

- Verschillende workload-klassen hebben verschillende eisen: een dev/test-omgeving heeft andere SLA-eisen dan een DigiD-productieomgeving.
- De opdracht stelt dat het platform "tienduizenden servers" moet kunnen hosten. Maar wat is de maximale schaling per cluster, per ODC, per platform?
- BIO schrijft voor dat beschikbaarheidseisen worden vastgesteld op basis van risicoanalyse.

Besluit

[Nog geen besluit — voorgelegd ter bespreking]

Onderzochte implementaties

Dit ADR beschrijft geen technologie-implementaties maar te kiezen doelwaarden. Richtinggevende opties:

Beschikbaarheidsklassen (voorstel ter bespreking):

- Klasse A (productie, kritieke GDI-voorzieningen): 99,95% beschikbaarheid
- Klasse B (productie, standaard): 99,9% beschikbaarheid
- Klasse C (niet-productie): geen SLA-garantie Deze klasses hoeven niet in elke regio direct beschikbaar te zijn, er zou ook 1 regio primair voor Klasse A kunnen terwijl een andere regio meer focus heeft op Klasse B en C.

Ondersteuning wordt vastgelegd in de SLA en kan hiervan afwijken.

Capaciteitslimieten (voorlopige richtgetallen):

- Max nodes per cluster: 5.000 (Kubernetes-praktijkgrens voor grote clusters)
- Max clusters per management-cluster: 1.000 (afhankelijk van Hypershift-schaling)
- Max tenants per platform: nader te bepalen op basis van ODC-capaciteit
- GPU-capaciteit: nader te bepalen na hardware-selectie (ADR-G13)

Technologiekeuze

N.v.t. — dit ADR stelt doelwaarden vast, geen technologiekeuze.

Motivatie

Zonder vastgestelde non-functionele eisen is het platform een architectuur zonder toetsingskader. RTO/RPO zijn de input voor de backup-strategie (ADR-G58) en voor de keuze van HA-configuraties in elke laag.

Consequenties

- Alle laag-specifieke ADRs moeten worden getoetst aan de hier vastgestelde eisen.
- Beschikbaarheidsklassen worden onderdeel van het tenant-onboarding-proces (ADR-G03).
- Capaciteitslimieten moeten worden gemonitord en actief worden beheerd (ADR-G64).
- Platformbeschikbaarheid staat los van de applicatiebeschikbaarheid en die laatste is de verantwoordelijkheid van de afnemer (ondersteund door het enablement proces)

Sessienotities

> Kernvraag: welke beschikbaarheidsklassen en bijbehorende SLA's zijn realistisch voor v1 van het platform? En: wie stelt de capaciteitslimieten vast — de platformoperator of per ODC?

ADR-G63: SOC-integratie en security monitoring

Status	Voorgesteld
Laag	Beveiliging
Afhankelijk van	ADR-G10, ADR-G38, ADR-G41, ADR-G57
Maakt mogelijk	
Datum	2026-08-07

Context en probleemstelling

De opdracht vereist expliciet “integratie met externe securitydienstverlening zoals een (of meerdere) SOC(’s)”. NIS2 en BIO schrijven voor dat organisaties die vitale of gevoelige infrastructuur beheren aantoonbaar beveiligingsincidenten detecteren, melden en afhandelen. Een platform dat DEP VERTROUWELIJK workloads host, valt hier zonder twijfel onder.

De architectuurvraag is: hoe integreert de Clouddienst met een of meerdere Security Operations Centers, en welk deel van de security monitoring is platform-eigen versus uitbesteed aan een externe SOC?

Deelvragen:

- Welke events en logs worden naar de SOC gestuurd (platform-events, tenant-events, of beide)?
- Hoe wordt de vertrouwelijkheid van tenant-data geborgd als een SOC meekijkt?
- Is er één centrale SOC voor de hele Clouddienst, of heeft elke tenant een eigen SOC-koppeling?
- Voldoet de SOC-integratie aan de meldplicht van NIS2 (CSIRT-NL, NCSC)?

Overwegingen:

- Een overheidsplatform kan meerdere SOC’s bedienen: een platform-SOC voor de infrastructuur zelf, en tenant-SOC’s voor de workloads. Deze mogen geen zicht hebben op elkaars data.
- SIEM-integratie (Security Information and Event Management) is de technische laag onder SOC-dienstverlening. De platformkeuze voor SIEM (open source vs. commercieel) bepaalt de integratiemogelijkheden.
- Het NCSC en het CISO Rijk stellen eisen aan de meldplicht en aan de beveiligingseisen voor rijksinfrastructuur. De SOC-integratie moet hierop aansluiten.
- NIS2 verplicht tot melding van significante incidenten binnen 24 uur (eerste melding) en 72 uur (gedetailleerde melding).

Besluit

[Nog geen besluit — voorgelegd ter bespreking]

Onderzochte implementaties

Platform-eigen SIEM (OpenSearch/Elastic + Wazuh): Open source SIEM op basis van OpenSearch Security Analytics of Wazuh. Platform-intern, geen externe datastromen. Voordeel: volledige controle, SEAL-4-conform. Nadeel: operationeel zwaar, vereist eigen SOC-analisten of uitbesteding van alleen de analyse.

Splunk / Microsoft Sentinel (commercieel): Volwassen SIEM-platforms met rijke detectie-regels en SOC-integraties. Nadeel: data verlaat de Clouddienst naar een SaaS-dienst (Sentinel = Azure), wat SEAL-4 en AVG-vragen oproept.

Wazuh (open source, zelf-gehost): Specifiek gebouwd voor security monitoring, intrusion detection en compliance-rapportage. Integreert met OpenSearch. Kan worden gecombineerd met een extern SOC dat via API of syslog-feed wordt gevoed.

Gefedereerd model (platform-SOC + tenant-SOC's): de Clouddienst levert gefilterde security-events aan tenant-SOC's via een gestandaardiseerde API (bijv. STIX/TAXII of syslog). Tenants bepalen zelf welk SOC zij gebruiken. Het platform-SOC bewaakt alleen de infrastructuurlaag.

Technologiekeuze

[Nog vast te stellen]

Motivatie

SOC-integratie is een expliciete opdrachteis en een NIS2-verplichting. Zonder dit is de Clouddienst juridisch niet compliant voor overheidsgebruik op het niveau waarvoor het bedoeld is.

Consequenties

- De observability-stack (ADR-G41) en de SOC-integratie delen logging-infrastructuur maar hebben gescheiden bewaar- en toegangsbeleid.
- Tenant-log-data mag nooit zonder toestemming van de tenant worden doorgegeven aan het platform-SOC.
- Een incident response procedure (playbook) moet worden opgesteld als onderdeel van de operationele documentatie.
- NIS2-meldplicht vereist een formeel contactpunt richting NCSC/CSIRT-NL.

Sessienotities

> Kernvraag: wie levert de SOC-dienst voor de Clouddienst zelf — inhouse of marktpartij? En: wat is de grens tussen platform-SOC (infrastructuur) en tenant-SOC (workload)? Verdere beschouwing op basis van het nader op te stellen Operating Model.

ADR-G64: Autoscaling en resource quota beheer

Status	Voorgesteld
Laag	Compute
Afhankelijk van	ADR-C04, ADR-C10, ADR-G62
Maakt mogelijk	ADR-G59
Datum	2026-04-13

Context en probleemstelling

De opdracht vereist een uitwerking van “autoscaling van services” als onderdeel van het management-platform. Autoscaling is een kernbelofte van cloudplatforms: tenants kunnen capaciteit automatisch op- en afschalen zonder handmatige tussenkomst. Tegelijk moet het platform voorkomen dat één tenant alle resources opeist (noisy-neighbor effect).

De architectuurvraag is: hoe worden autoscaling en resource quota ingericht op platform- en tenant-niveau, en hoe wordt de schaalbaarheid begrensd?

Drie lagen van autoscaling in Kubernetes:

1. **Horizontal Pod Autoscaler (HPA)**: schaalt het aantal pod-replica’s op basis van CPU/geheugen of custom metrics.
2. **Vertical Pod Autoscaler (VPA)**: past de resource-requests/limits per pod aan.
3. **Cluster Autoscaler / CAPI-scaling**: voegt nodes toe aan of verwijdert ze uit een cluster op basis van scheduling-druk.

Quota-mechanismen:

- **ResourceQuota**: limiteert totaal CPU/geheugen per namespace.
- **LimitRange**: stelt standaard en maximale requests/limits per pod/container.
- **PriorityClass**: bepaalt welke workloads voorrang krijgen bij capaciteitsschaarste.

Overwegingen:

- Op een bare-metal platform is node-autoscaling complexer dan op hyperscale cloud: nieuwe nodes moeten fysiek worden geprovisioned via CAPI + Metal^3 (ADR-C01), wat minuten duurt in plaats van seconden.
- Tenants moeten kunnen autoscalen binnen hun toegewezen quota zonder platformgoedkeuring per schaalstap.
- Het platform heeft een totale capaciteitslimiet (ADR-G62) — als die grens nadert, moet capaciteits-management worden geactiveerd.

Besluit

[Nog geen besluit — voorgelegd ter bespreking]

Onderzochte implementaties

Kubernetes-native HPA + KEDA: HPA voor standaard CPU/geheugen-scaling, KEDA (Kubernetes Event-driven Autoscaling) voor event-gedreven scaling (queue-depth, Prometheus-metrics, etc.). CNCF-graduated project, breed gebruikt.

CAPI Cluster Autoscaler: Schaalt het aantal nodes in een tenant-cluster via CAPI. Integreert met Metal³ voor bare-metal node-provisioning. Trager dan cloud-scaling maar volledig geautomatiseerd.

Karpenter: AWS-ontwikkelde maar open source node-provisioner, sneller dan standaard cluster-autoscaler. Nog beperkte bare-metal ondersteuning (primair voor cloud). Interessant voor de toekomst.

Hierarchical Namespace Controller (HNC) + quota-propagatie: Voor quota-beheer in een resource-hiërarchie (ADR-G67): quota worden van organisatie-niveau naar cluster- en namespace-niveau gepropageerd.

Technologiekeuze

[Nog vast te stellen]

Motivatie

Autoscaling is een expliciete opdrachteis en een basisverwachting van elke cloudafnemer. Zonder geautomatiseerde scaling moeten tenants handmatig capaciteit aanvragen, wat het platform fundamenteel minder aantrekkelijk maakt dan hyperscale alternatieven.

Consequenties

- Quota-allocatie per tenant moet onderdeel worden van het onboarding-proces (ADR-G03).
- Metering (ADR-G59) moet quota-gebruik bijhouden voor doorbelasting.
- Node-autoscaling op bare-metal verloopt via Gardener (ADR-C20) en metal-stack (ADR-C60). De provisioning-pipeline moet worden uitgewerkt met autoscaling-triggers in gedachten.

Sessienotities

> Kernvraag: welk minimum en maximum aantal nodes per tenant-cluster is realistisch op basis van de hardware-capaciteit? En: hoe wordt capaciteitsschaarste gecommuniceerd naar tenants?

ADR-G65: Multi-vendor strategie voor hardware en netwerkhardware

Status	Voorgesteld
Laag	Datacenter
Afhankelijk van	ADR-001, ADR-002, ADR-G13
Maakt mogelijk	ADR-G56, ADR-G68
Datum	2026-07-29

Context en probleemstelling

De opdracht vereist voor zowel compute als netwerk expliciet een uitwerking van de “multi-vendorstrategie en de effecten ervan”. Dit is geen details-vraag maar een fundamentele architectuurkeuze: hoeveel hardwareleveranciers opereren gelijktijdig op het platform, en welke consequenties heeft dit voor provisioning, certificering en operationeel beheer?

ADR-G13 behandelt de selectiestrategie voor compute-hardware per generatie. Dit ADR behandelt de bredere vraag: welk multi-vendor beleid hanteert het platform over alle hardware-categorieën heen, en hoe wordt vendor lock-in gedefinieerd en beperkt?

De Aanbestedingswet vereist dat de architectuur niet zo wordt ingericht dat slechts één leverancier kan leveren. Dit betekent niet dat altijd meerdere leveranciers tegelijk in productie zijn, maar wel dat de architectuur hiervoor openstaat.

Kernspanning:

- **Meer leveranciers** = minder lock-in, betere leveringszekerheid, meer complexiteit in provisioning en certificering.
- **Minder leveranciers** = eenvoudiger beheer, snellere certificering, grotere afhankelijkheid.

Hardware-categorieën met leveranciersdiversiteit-vraagstuk:

- **Servers (compute)**: Dell, HPE, Supermicro, Lenovo, Inspur — Redfish-standaard maakt abstractie mogelijk.
- **Netwerkhardware (switches, routers)**: Arista, Juniper, Cisco, Nokia, commodity (Broadcom ASIC) — NOS-keuze bepaalt lock-in.
- **GPU-hardware**: NVIDIA (dominant), AMD, Intel — beperkte keuze, NVIDIA is de facto standaard.
- **Storage-hardware**: Ceph abstraheert van hardware, maar schijven en HBA’s hebben hun eigen leveranciers.
- **HSM (Hardware Security Modules)**: Thales, Entrust, Utimaco — PKCS#11-standaard maakt abstractie mogelijk.

Besluit

Per generatie design (Block) zijn twee of max drie vendoren actief. Eén vendor levert een te grote lock-in op. Max 3 is op te lossen met software om de variaties in firmware op te vangen. Let op: wel sturen op zoveel mogelijk gebruik van Open Standaarden. Implementatie-optie 3, vendoren toelaten zolang maar

aan de standaarden wordt voldaan, zien we niet als reëel. De open standaarden zijn een filter in de vorm van eisen aan de hardware.

Onderzochte implementaties

Eén leverancier per hardware-generatie (generatie-rotatie): Per inkoopgeneratie wordt één leverancier gekozen na aanbesteding. Bij de volgende generatie kan een andere leverancier worden gekozen. Voordeel: lage operationele complexiteit per generatie. Nadeel: tijdelijke lock-in per generatie, leverancier-mix bij generatiemix in productie.

Twee leveranciers tegelijk per categorie (actief/actief): In productie draaien servers van twee leveranciers naast elkaar. Vereist dat de provisioning-stack (Ironic/Metal^3) beide correct ondersteunt inclusief firmware-variaties. Hogere complexiteit, lagere lock-in.

Open standaarden als scheiding (Redfish, IPMI, PKCS#11, SONiC): Vendor-abstractie via open beheerstandaarden, zodat leveranciers uitwisselbaar zijn zonder code-wijzigingen in de provisioning-pipeline. De standaard bepaalt de grens, niet de leverancierskeuze.

Technologiekeuze

[Nog vast te stellen]

Motivatie

Multi-vendor strategie is een expliciete opdrachteis en een aanbestedingsrechtelijke vereiste. De keuze bepaalt de complexiteit van de provisioning-pipeline en de risicobeheersing van de supply chain.

Consequenties

- De provisioning-pipeline (ADR-C01, ADR-C60) moet worden getest met hardware van meerdere leveranciers.
- Certificeringsaanpak (resultaatdocument) moet rekening houden met hardware-variatie.
- Inkoopstrategie moet de multi-vendor keuze ondersteunen — dit is een aanbestedingsvraagstuk buiten de architectuur maar met directe impact erop.

Sessienotities

> Kernvraag: is “één leverancier per generatie, wisselbaar tussen generaties” voldoende voor Aanbestedingswet-compliance? Of is gelijktijdige multi-vendor in productie vereist?

ADR-G66: Haven/Haven+ en standaarden conformance

Status	Voorgesteld
Laag	Overkoepelend
Afhankelijk van	ADR-001, ADR-002
Maakt mogelijk	alle laag-specifieke ADRs
Datum	2026-08-07

Context en probleemstelling

De opdracht stelt als eis dat de architectuur voldoet aan de Haven/Haven+ standaard.

Haven definieert welke capabilities een containerplatform minimaal moet bieden en hoe ze moeten werken, zodat workloads zonder herschrijven kunnen worden verplaatst tussen Haven-conforme platforms.

Haven+ is een geopinieerde keuzeset vanuit het CNCF ecosysteem ten behoeve van developer productivity.

De architectuurvraag is: op welke manier voldoet het de Clouddienst aan Haven/Haven+, en waar wijkt het af of gaat het verder?

Haven+ vereist (samenvatting van de standaard):

- Kubernetes als container-orkestratieplatform
- IAM op basis van OIDC/RBAC
- Observability (status, logging, monitoring)
- Metering
- PostgreSQL als database-dienst
- S3-compatibele object storage
- API-ontsluiting van alle platformfunctionaliteit
- Self-service portal

Spanningen de Clouddienst-architectuurkeuzes:

- De Clouddienst kiest voor bare-metal Kubernetes (ADR-001) — Haven stelt geen eisen aan de onderliggende laag, dus dit is conform.
- De Clouddienst kiest voor Cilium (ADR-C07) — Haven specificeert geen CNI, dus conform.

Besluit

De Clouddienst streeft naar volledige Haven-conformance als eis, maar plaatst Haven+ voorlopig out of scope. Uiteraard kan Haven+ als dienst geleverd worden, bijvoorbeeld binnen een tenant.

Onderzochte implementaties

Volledige conformance: Alle Haven-capabilities worden geïmplementeerd conform de standaard. Afwijkingen worden expliciet gemotiveerd en gedocumenteerd.

Conformance met uitbreidingen: Haven als minimum, NDS-specifieke uitbreidingen (SEAL-4, multi-tenancy, bare-metal) als aanvulling. Workloads die Haven-conformant zijn draaien zonder aanpassing op de Clouddienst.

Eigen standaard leidend: de Clouddienst definieert een eigen capability-set die Haven omvat maar niet beperkt is tot Haven. Risico: divergentie van de overheidsstandaard.

Technologiekeuze

N.v.t. — dit ADR stelt conformance-positie vast.

Motivatie

Haven-conformance is een expliciete opdrachteis en borgt dat de Clouddienst interoperabel is met andere overheidscloudplatforms en dat workloads portabel zijn.

Consequenties

- Elke laag-ADR moet worden getoetst aan de Haven-vereisten.
- Afwijkingen van Haven (bewuste of onvermijdelijke) moeten worden gedocumenteerd met motivatie.
- Haven is een levende standaard — het platform moet een lifecycle-mechanisme hebben om met nieuwe versies mee te bewegen (ADR-G52).
- De PostgreSQL-eis van Haven+ dwingt tot een besluit in ADR-G50, maar Haven+ is voor nu out of scope geplaatst.

Sessienotities

> Open vraag: wat is de actuele Haven+-versie en wie beheert de conformance-toetsing? Is er een formeel certificeringsproces, of is conformance zelf-verklaard?

ADR-G67: Resource-hiërarchie: organisatie-, cluster- en projectmodel

Status	Goud
Laag	Multi-tenancy
Afhankelijk van	ADR-G02, ADR-C04, ADR-G06
Maakt mogelijk	ADR-G03, ADR-G59, ADR-G64
Datum	2026-07-07

Context en probleemstelling

Het platform bedient tientallen overheidsorganisaties, elk met meerdere teams, projecten en omgevingen (ontwikkeling, test, productie). Zonder een expliciet resource-hiërarchie-model worden quota, facturering, toegangsrechten en beleidsregels onbeheersbaar: wie mag wat aanmaken, tot welke grens, en wie betaalt ervoor?

ADR-G02 stelt dat elke tenantorganisatie een eigen dedicated Kubernetes-cluster krijgt. ADR-C04 werkt dit uit op compute-niveau. De vervolgvraag is: hoe worden overheidsorganisaties, clusters, projecten en namespaces in een samenhangend model geordend, zodat toegangsbeheer, quota en doorbelasting op het juiste niveau kunnen worden gedefinieerd en gedelegeerd?

De hiërarchie maakt het mogelijk om:

- Quota op organisatieniveau te definiëren en te verdelen naar teams en projecten.
- Toegangsrechten te delegeren zonder centrale toestemming per handeling.
- Doorbelasting (ADR-G59) toe te wijzen aan de juiste kostenplaats.
- Beleidsregels (NetworkPolicies, admissionpolicies) te laten overerven door lagere niveaus.

Een kritieke randvoorwaarde is dat de bestuurlijke complexiteit van de gehele overheid niet in het platformmodel wordt getrokken. De organisatiedefinitie moet bruikbaar zijn voor onboarding zonder dat de interne structuur van ministeries, agentschappen of gemeenschappelijke regelingen volledig in kaart hoeft te zijn gebracht.

Onderzochte modellen

Model 1: Plat model — Organisatie → Cluster

De organisatie is het enige groeperingsniveau; elke cluster behoort direct aan een organisatie.

Voordelen: eenvoudigste model, eenvoudig te redeneren over rechten en quota.

Nadelen: geen subgroepering mogelijk binnen een cluster. Grote ministeries kunnen geen afzonderlijke omgevingen of teams delegeren. Alle workloads in een cluster delen één autorisatiescope. Onvoldoende voor een platform dat de volledige overheid bedient.

Model 2: Organisatie → Cluster → Project → Namespace (gekozen)

De organisatie is de tenantgrens. Een cluster is infrastructuur in eigendom van de organisatie. Een project groepeert workloads binnen een cluster (per omgeving of per team). Een namespace is de deployment-eenheid binnen een project.

Voordelen: toegangsbeheer op twee niveaus — organisatielidmaatschap stuurt platformtoegang, projectlidmaatschap stuurt workloadtoegang. Infrastructuurzaken (clusters, nodepools) zijn strikt gescheiden van workloadzaken (projecten, namespaces). Schaalbaar naar honderden organisaties zonder modelwijziging.

Nadelen: iets complexer dan het platte model.

Model 3: Organisatie → Project → Cluster

Een project groepeert clusters in plaats van workloads. Teams werken op projectniveau en krijgen meerdere clusters onder hun project.

Voordelen: sluit aan op bepaalde cloudprovider-modellen.

Nadelen: conflicteert met de praktijk van teams die binnen een cluster werken, niet over clusters heen. Vermengd infrastructuurgroepering met workloadgroepering. Bemoeilijkt quota-propagatie en billing per organisatie.

Model 4: Diep genest — Organisatie → Map → Map → ... → Project → Cluster

Maximale flexibiliteit voor complexe organisatiestructuren via vrij te definiëren mappen.

Voordelen: past zich aan elke organisatiestructuur aan.

Nadelen: diepe nesting leidt tot onduidelijkheid over beleidsovererving. Archiefregels, quota en RBAC worden onoverzichtelijk. Meer complexiteit dan het platform nu rechtvaardigt; introductie van mappen kan altijd later als bewezen behoefte bestaat.

Besluit

Organisatie → Cluster → Project → Namespace.

Een organisatie vertegenwoordigt een overheidsorganisatie en vormt de tenantgrens. Clusters zijn beheerde Kubernetes-clusters in van de organisatie, geprovisioned door het platform. Projecten zijn logische groeperingen binnen een cluster, bijvoorbeeld per omgeving (acceptatie, productie) of per team. Namespaces zijn de deploymenteenheden binnen een project.

Toegangsbeheer is op twee niveaus gedefinieerd: organisatielidmaatschap bepaalt wie platformtoegang heeft; projectlidmaatschap bepaalt wie workloads in een specifiek project kan beheren.

De organisatiedefinitie is bewust eenvoudig gehouden: de organisatie is de betalende partij en de bestuurlijk verantwoordelijke entiteit, niet de volledige interne structuur van die organisatie. Interne delegatie (teams, subeenheden) verloopt via projecten.

Motivatie

Model 2 is het enige model dat de drie kernfuncties van de hiërarchie tegelijk vervult: infrastructuurisolatie op clusterniveau (ADR-G02, ADR-C04), gedelegeerd toegangsbeheer op projectniveau (ADR-G06), en quota- en billing-aggregatie op organisatieniveau (ADR-G59, ADR-G64).

Model 1 is te plat voor de omvang van de overheid: grote organisaties zoals ministeries met meerdere uitvoeringsorganen of teams hebben sub-groeperingen nodig. Model 3 inverteert de infrastructuur- en workloadlagen op een manier die niet aansluit op hoe teams dagelijks werken. Model 4 introduceert complexiteit die niet gerechtvaardigd is zonder bewezen behoefte.

De keuze voor een platform-breed uniform model — in tegenstelling tot een configureerbaar model per tenant — is bewust. Een enkelvoudig, consistent model maakt auditing, billing en platformbeheer aantoonbaar beheersbaar. Configureerbare hiërarchieën per tenant verplaatsen de complexiteit naar de operationele laag.

Consequenties

Elke namespace behoort aan precies één project, elk project aan één cluster, elk cluster aan één organisatie. Dit is een harde structuureis zonder uitzonderingen.

Het onboarding-proces (ADR-G03) maakt onderscheid tussen drie rollen: organisatie-onboarding (platformteam), cluster-provisioning (infrastructuurbeheerder), en project-aanmaak (tenant-admin via self-service). Elk niveau heeft eigen verantwoordelijkheden en bevoegdheden.

Quota-aggregatie en doorbelasting (ADR-G59) lopen altijd op organisatieniveau. Projecten hebben eigen sub-quota, maar de organisatiegrens is de rekeneenheid voor facturering.

Het IAM-systeem (ADR-G06) moet autorisatie kunnen toewijzen op zowel organisatieniveau als projectniveau. Dit zijn twee afzonderlijke autorisatiescopes die nooit worden samengevoegd.

De self-service interface (ADR-C10) weerspiegelt de hiërarchie: organisatiebeheerders zien clusters, projectbeheerders zien namespaces binnen hun project.

Nodepools zijn een infrastructuurzaak beheerd op clusterniveau. Projectgebruikers hebben geen zichtbaarheid op of toegang tot nodepools.

Sessienotities

> [10-03-2026] Vier modellen afgewogen. Model 2 gekozen op basis van scheiding infrastructuur- vs. workloadzaken en schaalbaarheid naar de volledige overheid. [07-07-2026] ADR herschreven: besluit vastgesteld, HNC/Capsule expliciet afgevallen, solutiion keuze gedelegeerd naar C20-beslissing. Circulaire verwijzing naar G06 in “Maakt mogelijk” gecorrigeerd.

ADR-G68: Hardware-lifecycle management: refresh, end-of-support en decommissioning

Status	Voorgesteld
Laag	Datacenter
Afhankelijk van	ADR-G13, ADR-C60, ADR-G65
Maakt mogelijk	ADR-G62
Datum	2026-04-20

Context en probleemstelling

ADR-G13 behandelt de selectie van hardware. Dit ADR behandelt wat er daarna gebeurt: hoe worden servers, switches en overige hardware beheerd gedurende hun levensduur, en hoe worden ze vervangen of buiten gebruik gesteld?

Op een platform van tienduizenden servers is hardware-lifecycle management een operationeel-architectureel vraagstuk. Zonder beleid hierover worden servers te lang in productie gehouden (beveiligingsrisico's door end-of-support firmware), of worden ze te vroeg vervangen (onnodige kosten).

De architectuurvraag is: welk lifecycle-beleid en welke processen hanteert het platform voor hardware van aanschaf tot decommissioning?

Fasen van hardware-lifecycle:

1. **Ontvangst en inventarisatie**: rack-and-stack, serienummer-registratie, firmware-baseline.
2. **Commissioning**: opname in de provisioning-pipeline (ADR-C01, ADR-C60), baseline-configuratie.
3. **Operationeel beheer**: firmware-updates, hardwarefouten, RMA-procedures.
4. **End-of-support**: leverancier stopt met firmware-patches. Wanneer wordt hardware vervangen?
5. **Decommissioning**: veilige data-wissing, fysieke verwijdering, duurzame verwerking.

Overwegingen:

- Servergarantie loopt typisch 3–5 jaar. Firmware-support kan langer doorlopen maar wordt onzeker.
- SEAL-4 stelt eisen aan de supply chain — bij vervanging moeten nieuwe servers voldoen aan dezelfde soevereiniteitscriteria.
- Data-wissing bij decommissioning moet voldoen aan overheidsstandaarden (NEN-EN 15713 of DoD 5220.22-M equivalent).
- Duurzaamheid (sustainability) is een doorsnijdend thema in de Domeinarchitectuur — circulaire verwerking van hardware is relevant.
- Het platform moet kunnen draaien tijdens een hardware-refresh: rolling replacement zonder downtime.

Besluit

Rollerend x-jaarcyclus: per generatie design staat tevoren vast wanneer hardware wordt vervangen. Deze wordt niet vervangen binnen het huidige design (Block A, Block B, etc) maar ingezet als uitbreiding of start van het nieuwste block.

Per block kan worden gekozen voor leveranciers? Relatie met G65

Onderzochte implementaties

CMDB-gestuurd lifecycle-beleid (Netbox + Ansible): Hardware wordt bijgehouden in Netbox (asset management) met levensduur-attributen. Geautomatiseerde rapportage geeft aan welke hardware de end-of-support datum nadert. Ansible-playbooks orchestreren de vervangingsprocedure.

Fabrikant-lifecycle als leidraad: End-of-support datum van de fabrikant bepaalt het vervangingsmoment. Eenvoudig beleid, maar fabrikanten kunnen end-of-support vervroegen (Broadcom/VMware-precedent).

Rollerend x-jaarscyclus: Ongeacht fabrikant-lifecycle wordt hardware na x jaar vervangen (kan per component ander beleid zijn). Voorspelbaar voor planning en inkoop, maar mogelijk te vroeg of te laat afhankelijk van fabrikant. In combinatie met een op maat gemaakt DCIM (andere ADR)

Technologiekeuze

Beleid en operatie zijn in combinatie specifiek voor de oplossing, een eigen DCIM ligt voor de hand op deze schaal. (nieuw ADR nodig?)

Motivatie

Hardware-lifecycle is een operationele realiteit die architectureel moet worden verankerd. Zonder beleid leidt dit tot beveiligingsrisico's (verouderde firmware) en onvoorziene vervanging die de bedrijfsvoering verstoort.

Consequenties

- De provisioning-pipeline (ADR-C01) moet decommissioning ondersteunen: nodes moeten graceful kunnen worden verwijderd uit clusters.
- Data-wissing-procedure moet worden vastgelegd als onderdeel van de operationele documentatie.
- Hardware-lifecycle beïnvloedt de meerjarenbegroting van het platform — dit moet worden meegenomen in het dienstverleningsportfolio (resultaatdocument).

Sessienotities

> Kernvraag: wie is verantwoordelijk voor hardware-lifecycle — de platformoperator of de ODC? En: welke data-wissing-standaard is verplicht voor DEP VERTROUWELIJK hardware?

ADR-G69: Koppeling met publieke clouddiensten: hybride model

Status	Voorgesteld
Laag	Overkoepelend
Afhankelijk van	ADR-001, ADR-002, ADR-G04
Maakt mogelijk	ADR-G49
Datum	2026-04-20

Context en probleemstelling

De opdracht vereist expliciet een uitwerking van de “inrichting van koppeling met publieke clouddiensten”. De Domeinarchitectuur stelt dat de overheid meerdere typen aanbieders wil bedienen: overheidsdatacenters, Nederlandse/Europese aanbieders én internationale hyperscalers. NDS Cloud is primair gericht op soevereine infrastructuur, maar de architectuurvraag is: welke rol spelen publieke clouddiensten in het totaalplaatje? Bijvoorbeeld als je de data in de soevereine overheidscloud wil draaien en de applicatie in de public cloud.

Drie mogelijke posities:

1. **NDS Cloud als primaire platform, publieke cloud als opt-in uitbreiding**: tenants kunnen specifieke diensten (bijv. AI/ML-training, burst-capaciteit) op publieke cloud draaien, mits dit is toegestaan op basis van dataclassificatie.
2. **NDS Cloud en publieke cloud als gelijkwaardige opties in een marktplaats**: conform Domeinarchitectuur §3.3 — tenants kiezen per workload de meest geschikte aanbieder. Voor het voorbeeld zitten we op 2, maar de integratielaag zit expliciet niet in scope.
3. **NDS Cloud als volledig geïsoleerd platform**: geen structurele koppeling met publieke cloud. Tenants moeten een expliciete uitzondering aanvragen.

Overwegingen:

- SEAL-4 (ADR-002) is van toepassing op het NDS Cloud-platform zelf, niet per definitie op elke tenant-workload. Tenants kunnen zelf besluiten lagere SEAL-niveaus te accepteren voor bepaalde workloads.
- AVG en BIO stellen eisen aan data-locatie: persoonsgegevens en DEP VERTROUWELIJK data mogen niet zonder meer naar publieke cloud.
- Hybride architectuur vereist netwerkverbindingen tussen NDS Cloud ODC’s en publieke cloud regio’s — dit moet worden geborgd via Diginetwerk of beveiligde VPN (ADR-G04).
- De Domeinarchitectuur noemt uitdrukkelijk dat NDS Cloud niet de enige aanbieder hoeft te zijn — “We onderscheiden meerdere typen aanbieders.”

Besluit

[Nog geen besluit — voorgelegd ter bespreking]

Onderzochte implementaties

Cluster API provider voor publieke cloud (burst): CAPI ondersteunt providers voor AWS, Azure, GCP naast Metal³. Een tenant-cluster kan worker-nodes laten uitrollen op publieke cloud bij capaciteitspieken. Data blijft op NDS Cloud (control plane), compute kan elders staan. Voordeel: elasticiteit. Nadeel: complexe data-soevereiniteitsgrens.

Gestandaardiseerde API-koppeling (SECA/GAIA-X): Via de Sovereign European Cloud API (SECA) of GAIA-X-standaarden kunnen NDS Cloud-diensten worden gefedereerd met gecertificeerde Europese cloudaanbieders. Toekomstgericht maar nog niet volwassen.

Expliciete scheiding (geen automatische hybrid): Tenants die publieke cloud willen gebruiken, regelen dit buiten het NDS Cloud-platform. NDS Cloud biedt alleen de interne soevereine omgeving. Eenvoudig, geen architecturele complexiteit, maar minder flexibel.

Marktplaatsmodel (conform Domeinarchitectuur): NDS Cloud is één aanbieder in een bredere overheidsmarktplaats. Tenants kiezen per workload uit het aanbod. NDS Cloud hoeft geen directe technische koppeling te hebben met andere aanbieders — de marktplaats coördineert.

Technologiekeuze

[Nog vast te stellen]

Motivatie

De opdracht vereist een expliciete positiebepaling. Zonder dit kader nemen tenants ad-hoc beslissingen over publieke cloud-gebruik, buiten het architectuurkader.

Consequenties

- Dataclassificatiebeleid (ADR-G10) bepaalt welke workloads wel of niet naar publieke cloud mogen.
- Als hybride koppeling wordt gekozen, moet netwerkbeveiliging (ADR-G57) en IAM-federatie (ADR-G06) worden uitgebreid voor publieke cloud.
- De resultaatdocument-component “aansluitmodel marktpartijen” moet dit hybride model beschrijven.

Sessienotities

> Kernvraag: is NDS Cloud een gesloten soeverein platform of een aanbieder in een bredere marktplaats? Dit is deels een beleidsmatige keuze die buiten de architectuur ligt maar er sterk op doorwerkt.

ADR-G70: CI/CD platform dienst voor tenant-workloads

Status	Voorgesteld
Laag	Managed diensten
Afhankelijk van	ADR-G49, ADR-G60, ADR-C55
Maakt mogelijk	
Datum	2026-08-11

Context en probleemstelling

De Domeinarchitectuur (§4.3.1) noemt CI/CD-diensten als “nader te bepalen” basisaanbod. De opdracht beschrijft dat de Clouddienst “continuous delivery en IaC moet ondersteunen”. Voor DevSecOps-teams is CI/CD-functionaliteit de ruggengraat van hun werkwijze — zonder geïntegreerde CI/CD moeten tenants hun eigen tooling meebrengen en integreren.

De architectuurvraag is: biedt de Clouddienst een managed CI/CD-dienst aan tenants, en zo ja, welke tooling en op welk niveau?

Scope-afbakening is cruciaal:

- **Platform-CI/CD** (ADR-C55, ADR-G60): GitOps-reconciliatie van platformconfiguratie ten behoeve van de Clouddienst zelf. Dit is al in scope.
- **Tenant-CI/CD**: het pipeline-platform dat tenant-teams gebruiken om hun applicaties te bouwen, testen en deployen. Dit is het onderwerp van dit ADR.

Twee posities zijn mogelijk:

1. **Managed CI/CD als platformdienst**: de Clouddienst biedt een gedeelde of dedicated CI/CD-instantie per tenant.
2. **Bring-your-own CI/CD**: tenants brengen hun eigen CI/CD-tooling mee (GitHub Actions, GitLab, Jenkins). De Clouddienst biedt alleen de benodigde integratiepunten (container registry, Kubernetes API).

Overwegingen:

- Een managed CI/CD-dienst verhoogt de platformwaarde maar ook de operationele complexiteit.
- Veel overheidsorganisaties hebben al een CI/CD-investering (Azure DevOps, GitLab, GitHub Enterprise).
- Een interne container registry is hoe dan ook nodig — ook als tenants eigen CI/CD gebruiken.
- SEAL-4: als CI/CD-tooling extern is (GitHub.com, Azure DevOps SaaS), verlaat broncode de soevereine omgeving.

De dienst CI/CD moet passen bij de dienstlaag die geleverd wordt. Dit past bij ene hogere dienstlaag (latere plateau), en is o.a. afhankelijk van het nader te ontwerpen Operating Model

Besluit

[Nog geen besluit — voorgelegd ter bespreking]

Onderzochte implementaties

GitLab CE (self-hosted, all-in-one): GitLab CE biedt Git-hosting, CI/CD (GitLab Runners), container registry en package registry in één platform. Kan worden gecombineerd met of vervangen door de Git-dienst uit ADR-G60. Hoge operationele waarde voor tenants, maar zwaar qua resource-gebruik.

Gitea/Forgejo + Tekton: Lichte Git-server (ADR-G60) gecombineerd met Tekton als cloud-native CI/CD-platform (CNCF-project). Modulaire aanpak, maar vereist meer configuratie per tenant.

Gitea/Forgejo + Woodpecker CI: Lichte open source CI/CD specifiek ontworpen voor Gitea-integratie. Eenvoudiger dan Tekton, minder geschikt voor complexe enterprise-pipelines.

Bring-your-own (platform biedt alleen registry): Het platform biedt een interne container registry (Harbor of Zot) en een Kubernetes-API. Tenants gebruiken hun eigen CI/CD. Laagste complexiteit voor het platform, hoogste eigen verantwoordelijkheid voor tenants.

Technologiekeuze

[Nog vast te stellen]

Motivatie

CI/CD is een kernonderdeel van de DevSecOps-werkwijze die de Domeinarchitectuur voorschrijft. De positiebepaling (managed vs. bring-your-own) heeft directe invloed op de platformwaarde van de Clouddienst en de operationele last voor de tenants.

Consequenties

- Een interne container registry is minimaal vereist, ongeacht de CI/CD-keuze. Dit moet als afzonderlijke platformdienst worden uitgewerkt. Dat wil zeggen dat elke Container-/image registry air-gapped is, dus nooit rechtstreeks verbonden met het internet. Dit is een maatregel om te voorkomen dat ongewenste (base) images of images met te hoge kwetsbaarheden gebruikt worden. De implicatie daarvan is dat er een procedure nodig is voor toestemming voor nieuwe base images, met korte doorlooptijden.
- Als managed CI/CD wordt gekozen, moet dit onderdeel worden van de managed diensten catalogus (ADR-G49) met bijbehorend lifecycle-beleid (ADR-G52).
- De Git-dienst (ADR-G60) en CI/CD-dienst kunnen dezelfde technologie gebruiken (bijv. GitLab CE voor beide) — dit moet worden afgestemd.

Sessienotities

> Kernvraag: is een managed CI/CD-dienst in scope voor v1, of biedt het platform alleen de integratiepunten? En: wat is de positie ten aanzien van tenants die al GitLab SaaS of Azure DevOps gebruiken?

ADR-G71: Cluster-lifecycle

Status	Voorgesteld
Laag	Provisioning
Afhankelijk van	ADR-C01, ADR-C04, ADR-C20
Datum	2026-06-01

Context en probleemstelling

Het platform gebruikt hosted control planes: control planes draaien als pods op een management-cluster (ADR-C20). Dat principe staat niet ter discussie. De vraag is welk framework de volledige clusterlifecycle eromheen verzorgt, en hoe dat aansluit op de rest van de stack, met name bare-metal provisioning (ADR-G72) en de dag-2-operatie.

Twee frameworks voldoen aan het principe: Gardener en Hypershift (Red Hat). De keuze raakt ook de soevereiniteitsambitie (ADR-002) en het architectuurmodel met losse CNCF-componenten (ADR-001).

Besluit

Het platform kiest Gardener voor de clusterlifecycle.

Gardener vormt samen met metal-stack (ADR-G72) één keten van provisioning tot clusterlifecycle. Deze keten is in de PoC ingezet en werkend aangetoond, en wordt verder technisch getest.

Onderzochte implementaties

Gardener: drie clustertypes (Garden, Seed, Shoot). Beheert de control plane en levert dag-2-operatie ingebouwd: upgrades, credential-rotatie, autoscaling, hibernation en self-healing. Native integratie met metal-stack. Valt onder de NeoNephos Foundation (Linux Foundation Europe). Getest in de PoC.

Hypershift: hosted control planes voor OpenShift en Kubernetes, bare-metal via Metal³/Ironic, native integratie met Cluster API. Lichter en flexibeler in componentkeuze, bewezen in de managed OpenShift-diensten van Red Hat. De dag-2-operatie moet zelf worden ingevuld. Afgevallen, zie Motivatie.

Kamaji: lichter, maar minder volwassen. Afgevallen op schaal en volwassenheid.

Motivatie

Gardener levert de dag-2-operatie die bij Hypershift apart gebouwd en beheerd zou moeten worden. Dat scheelt een substantiële eigen ontwikkel- en beheerlast.

De combinatie met metal-stack geeft één samenhangende keten van fysieke server tot tenant-cluster. Met Hypershift ligt de koppeling met Metal³/Ironic voor de hand, en daarmee een provisioningroute die in ADR-G72 is afgevallen.

Gardener valt onder Europese governance. Hypershift trekt de stack richting het OpenShift-ecosysteem, met Amerikaanse governance. Dat past minder bij SEAL-4 en bij een architectuur van losse CNCF-componenten.

Gardener is in de PoC ingezet en functioneert. De verdere technische tests lopen.

Consequenties

- Gardener is meer opinionated dan Hypershift. Het legt keuzes op in andere lagen, onder andere via eigen abstracties voor machinebeheer die overlappen met Cluster API. Daar staat ingebouwde dag-2-operatie tegenover.
- ADR-C01 en ADR-C60 worden in lijn gebracht met de keten metal-stack en Gardener.
- Het beheerteam moet eigen Gardener-expertise opbouwen. De leercurve is steil.

Bekende risico's

- **Lock-in op het framework**: omdat Gardener opinionated is, is een latere overstap naar een ander framework kostbaar. De keuze is in de praktijk moeilijk omkeerbaar.
- **Ecosysteem**: de doorontwikkeling van Gardener leunt sterk op een beperkt aantal organisaties. Dit raakt de SEAL-4-eis rond langetermijnautonomie (ADR-002).
- **Schaal**: de PoC toont werking aan, niet werking op de doelschaal met honderden tenant-clusters. Opschaling moet in de implementatiefase worden gevalideerd.

ADR-G72: Bare-metal provisioning

Status	Voorgesteld
Laag	Provisioning
Afhankelijk van	ADR-C01, ADR-C60
Maakt mogelijk	ADR-G71
Datum	2026-06-01

Context en probleemstelling

Het platform moet fysieke servers en de netwerkfabric daartussen geautomatiseerd in gebruik nemen, herinrichten en uitfaseren, op een doelschaal van tienduizenden servers verspreid over meerdere ODC-locaties.

Twee routes zijn overwogen. Cluster API (CAPI) met Metal³ en Ironic provisioneert alleen compute; netwerkprovisioning blijft een apart vraagstuk. metal-stack provisioneert compute en netwerk in één keten.

De keuze hangt samen met de clusterlifecycle (ADR-G71): metal-stack integreert native met Gardener, CAPI/Metal³ met Hypershift. Beide afstemmingspunten moeten daarom in samenhang worden besloten.

Besluit

Het platform kiest metal-stack voor bare-metal provisioning.

metal-stack levert compute- én netwerkprovisioning (spine-leaf BGP/EVPN, VRF per tenant) in één geautomatiseerde keten. In de PoC is deze keten ingezet en werkend aangetoond, in combinatie met Gardener voor de clusterlifecycle.

metal-stack is open source en ondersteunt servers van meerdere leveranciers. Dit is daarmee geen leveranciersgevoelige keuze.

Onderzochte implementaties

metal-stack: geïntegreerde compute- en netwerkprovisioning, Kubernetes-native, ondersteunt servers van meerdere leveranciers (Dell, HPE, Supermicro). In productie bij financiële instellingen sinds 2020. Getest in de PoC. Vereist switches met SONiC (onder andere Edgecore).

CAPI + Metal³ + Ironic: compute-provisioning via Kubernetes-resources, netwerkprovisioning apart. Ironic is niet bewezen op de doelschaal. Koppelt aan de Hypershift-route (ADR-G71 optie B). Afgevallen, zie Motivatie.

Motivatie

Eén keten voor compute en netwerk verlaagt het integratierisico op schaal. Met CAPI/Metal³ wordt netwerkprovisioning een apart project, inclusief de dag-2-operatie eromheen. Dat is precies de koppeling tussen losse componenten die op schaal het meeste risico draagt.

De keuze is in de PoC getest en sluit aan op de Gardener-richting in ADR-G71.

Consequenties

- ADR-C01 en ADR-C60 worden in lijn gebracht met dit besluit.
- Hardwareselectie (ADR-G13, ADR-G65) moet rekening houden met de SONiC-eis voor switches.
- ADR-G56 (network fabric) wordt grotendeels ingevuld door metal-stack en moet daarop worden herzien. ADR-G57 (firewall-architectuur) blijft nodig voor de beveiligingseisen die buiten de provisioningketen vallen.

Bekende risico's

- **Switchafhankelijkheid**: de SONiC-eis beperkt de keuze in netwerkhardware en staat op gespannen voet met de multi-vendorstrategie (ADR-G65). Dit moet daar expliciet worden afgewogen.
- **Schaal**: de PoC toont werking aan, niet werking op de doelschaal. Ook voor metal-stack is de doelschaal nog niet aangetoond. Opschaling moet in de implementatiefase worden gevalideerd.
- **Ecosysteem**: de community rond metal-stack is kleiner dan die rond CAPI/Metal³. De doorontwikkeling leunt op een beperkt aantal organisaties. Dit raakt de SEAL-4-eis rond langetermijnautonomie (ADR-002) en vraagt om eigen expertise in het beheerteam.

ADR-G73: Hypervisor-strategie (no-hypervisor, KubeVirt optioneel)

Status	Zilver
Laag	Compute
Afhankelijk van	ADR-C03
Datum	2026-06-01

Context en probleemstelling

Het platform draait Kubernetes direct op bare-metal, zonder hypervisorlaag (ADR-001). Een deel van de overheidsworkloads draait echter op virtuele machines en is niet of niet op korte termijn te containeriseren. Het platform moet die workloads kunnen hosten (ADR-C03).

De vraag is welke rol VM-ondersteuning in het ontwerp krijgt. Wordt het een pijler waar de rest van de stack op wordt ingericht, of een optionele voorziening voor tenants die het nodig hebben?

Dat verschil is groot. Een platformbrede eis voor VM's met live-migratie straalt door de hele stack: het dwingt RWX-storage af, Multus voor meerdere netwerkinterfaces, IOMMU- en SR-IOV-configuratie op alle nodes en eventueel Kata Containers.

Besluit

Het platform houdt geen hypervisorlaag aan als uitgangspunt. VM-ondersteuning via KubeVirt is een optionele Tool die per tenant wordt aangeboden waar dat nodig is. Live-migratie is geen platformbrede eis, maar hooguit een eigenschap van die Tool voor een tenant die het expliciet nodig heeft.

Dit uitgangspunt, bare-metal Kubernetes zonder hypervisorlaag, is in de PoC toegepast.

Onderzochte implementaties

Geen hypervisor, KubeVirt als optionele Tool: bare-metal Kubernetes als basis. VM's via KubeVirt alleen waar nodig, zonder dat de eisen aan storage, netwerk en nodes platformbreed gelden. Toegepast in de PoC.

KubeVirt als platformpijler met live-migratie: VM's eersteklas, live-migratie als harde eis. Legt RWX-storage (raakt ADR-C05), Multus (raakt ADR-C07), IOMMU/SR-IOV en Kata op aan alle nodes. Afgevallen, zie Motivatie.

Motivatie

Een harde eis voor VM's met live-migratie maakt een randgeval dragend voor storage, netwerk en node-configuratie. Elke node en elke storageklasse moet dan de zwaarste VM-workload aankunnen, ook als de meeste tenants alleen containers draaien.

Door VM's optioneel te maken blijft de basis eenvoudig, terwijl de capaciteit beschikbaar blijft voor tenants die het echt nodig hebben. Dit sluit aan op ADR-C03, dat live-migratie ook niet als harde eis formuleert.

Consequenties

- ADR-C05 motiveert de keuze voor Ceph nog deels met RWX voor KubeVirt live-migratie als platformeis. Die motivatie moet in lijn worden gebracht met dit besluit. De keuze voor Ceph zelf blijft staan op de andere gronden: één systeem voor block, file en object storage.
- Netwerkkeuzes (ADR-C07) hoeven Multus niet platformbreed te ondersteunen.
- ABB BA14 (Virtual Machine runtime) blijft bestaan, maar gemarkeerd als optionele Tool.

Bekende risico's

- **Onderschatting van de VM-vraag**: als een groter deel van de overheidsworkloads VM's nodig heeft dan verwacht, groeit de optionele Tool uit tot een de facto pijler. Het ontwerp moet die groei kunnen opvangen zonder herinrichting achteraf.
- **Beschikbaarheid bij onderhoud**: zonder live-migratie leiden node-onderhoud en node-vervanging (ADR-C24) tot downtime van VM-workloads. Tenants met VM's moeten dat weten en er in hun applicatie-architectuur rekening mee houden.
- **Gemengde nodeconfiguratie**: nodes die de VM-Tool draaien hebben andere configuratie nodig dan de rest. Dat moet in het node- en capaciteitsbeheer (ADR-C23, ADR-G64) worden belegd.

ADR-G74: Secrets- en certificaatbeheer per cluster

Status	Brons
Laag	Perifere diensten
Afhankelijk van	ADR-003, ADR-G53, ADR-G54
Datum	2026-08-21

Context en probleemstelling

Het platform heeft secrets-beheer en certificaatbeheer nodig, voor platformcomponenten en voor tenant-workloads. ADR-G53 en ADR-G54 werken een centraal model uit: één platform-eigen KMS (HA-Vault of OpenBao, minimaal drie nodes) en een centrale PKI met een offline root op HSM.

In de PoC is een decentraal model gebouwd en gevalideerd. Secrets en certificaten worden daar per cluster beheerd met Sealed Secrets en cert-manager, zonder centrale KMS-omgeving.

De architectuurvraag is: wordt secrets- en certificaatbeheer centraal voor het hele platform ingericht, of per cluster?

Daarbij moeten twee vragen gescheiden worden. De eerste vraag is waar secrets en certificaten worden opgeslagen en uitgegeven. De tweede vraag is hoe het platform de hardwarematige root-of-trust invult die SEAL-4 vraagt (ADR-002). Die tweede vraag bestaat ongeacht het antwoord op de eerste.

Besluit

Secrets- en certificaatbeheer wordt per cluster ingericht als standaard platformtool, conform het model dat in de PoC is gevalideerd. Er komt geen centrale KMS-omgeving voor het hele platform.

De hardwarematige root-of-trust voor SEAL-4 wordt als aparte capaciteit belegd en valt buiten dit besluit.

Onderzochte implementaties

Per cluster (gevalideerd in de PoC): Sealed Secrets voor versleutelde secrets in Git, cert-manager voor uitgifte en rotatie van certificaten. Per cluster geïnstalleerd en via GitOps beheerd.

Voordelen: geen centrale omgeving om te bouwen, te beveiligen en aan te besteden. Past bij het GitOps-beheermodel (ADR-C12, ADR-C55). Uitsluitend OSI-open-source componenten (ADR-003). Een compromittering blijft beperkt tot één cluster. Werkend aangetoond in de PoC.

Nadelen: geen centraal auditpunt voor sleuteltoegang. Governance, sleutelback-up en rotatie moeten per cluster uniform worden afgedwongen.

Centrale KMS en PKI (ADR-G53, ADR-G54): een HA-cluster met Vault of OpenBao, de bijbehorende PKI-engine en een offline root op HSM.

Voordelen: één centraal auditpunt en een sterke basis voor een formele SEAL-4-claim.

Nadelen: grote operationele last en een extra aanbesteding. Een centrale component waarvan alle clusters afhankelijk worden. HashiCorp Vault valt sinds 2023 onder de Business Source License en is daarmee niet OSI-open-source, wat botst met ADR-003. OpenBao is de open source fork, maar is minder volwassen.

Technologiekeuze

Sealed Secrets en cert-manager, beide onder Apache 2.0-licentie.

Motivatie

De PoC laat zien dat het decentrale model werkt. Het centrale model voegt een grote en gevoelige werkstroom toe, met operatie, aanbesteding en licentierisico, terwijl dat niet nodig is voor het primaire doel: veilig beheer van secrets en certificaten per cluster.

Het sterkste argument voor centralisatie is de hardwarematige root-of-trust voor SEAL-4. Die eis rechtvaardigt echter geen centralisatie van al het secrets-beheer en wordt daarom apart ingevuld.

Consequenties

- ADR-G53 en ADR-G54 worden herzien naar dit model. Tot die herziening spreken zij dit besluit tegen.
- De sealing keys van Sealed Secrets moeten per cluster worden geback-upt en geroteerd. Verlies van een sleutel betekent dat de secrets van dat cluster niet meer te ontsleutelen zijn. Dit wordt onderdeel van het backupontwerp (ADR-G58).
- Audit op sleutel- en certificaatgebruik loopt via de centrale observability (ADR-G41), omdat er geen centraal KMS-auditpunt is.
- De HSM-root-of-trust voor SEAL-4 wordt als aparte capaciteit belegd. Dit is een open gap.
- De bouwstenen BA24 (Secret Management) en BA25 (PKI) blijven bestaan, maar worden gemodelleerd als per-cluster tool.

Open vragen

1. Hoe en waar wordt de HSM-root-of-trust voor SEAL-4 ingevuld, en hoe wordt de per-cluster PKI daarin verankerd?
2. Is een centraal auditpunt voor sleuteltoegang vereist voor DEP VERTROUWELIJK workloads (ADR-G10, ADR-G38), of volstaat audit via ADR-G41?
3. Hoe verhoudt de per-cluster PKI zich tot PKIoverheid voor tenant-facing certificaten (zie ADR-G54)?

ADR-G75: Interne IP-adressering, IPv4 intern, IPv6 extern

Status	Voorgesteld
Laag	Networking
Afhankelijk van	ADR-G05, ADR-G23
Datum	2026-06-01

Context en probleemstelling

ADR-G05/G23 leggen IPv6-first vast, ook intern: “intern platformverkeer is IPv6-only zodra alle componenten dat ondersteunen”. Tijdens de PoC is het overheids-IPv6-beleid zo gelezen dat het bindt aan extern bereikbare diensten, niet aan interne infra.

De praktische vraag: bindt het IPv6-mandaat de interne pod/service-adressering, of alleen het ingress/externe vlak?

Besluit

Voorstel: scope IPv6 tot het externe/ingress-vlak en houd intern IPv4, in lijn met de PoC. Herzie G05/G23 op dit punt. Duidelijke richting; stem bij het team.

Extern bereikbare diensten zijn IPv6 (en dual-stack waar nodig); pod- en service-CIDR’s intern blijven IPv4. Zo wordt aan het overheids-IPv6-beleid voldaan zonder dual-stack over de hele interne stack.

Onderzochte implementaties

IPv4 intern / IPv6 extern: interne CIDR’s IPv4 (RFC1918); IPv6 op ingress/externe diensten. Vermijdt dual-stack-complexiteit en onvolwassen IPv6-ondersteuning in operators/Helm-charts.

IPv6-first intern (NDS G05/G23): dual-stack met intern IPv6-only als einddoel. Verdubbelt de testmatrix en bedrijfsvoering over elke operator, Helm-chart en netwerkpolicy.

Motivatie

Intern IPv6 raakt elk component en verdubbelt het operationele oppervlak, terwijl het beleidsdoel (extern bereikbaar via IPv6) al met IPv6 op het ingress-vlak wordt gehaald.

Consequenties

- G05/G23 herzien: IPv6-mandaat begrensd tot extern/ingress; intern IPv4.
- Tenant-workloads die intern IPv6 nodig hebben kunnen dat per cluster opt-in krijgen.
- Sluit aan op de Cilium-keuze (ADR-C07)

Componentspecifieke besluiten

ADR-C01: Bare-metal Kubernetes provisioning-strategie

Status	Goud
Laag	Provisioning
Afhankelijk van	ADR-001, ADR-C60, ADR-C20
Maakt mogelijk	ADR-C23, ADR-C24
Datum	2026-08-07

Context en probleemstelling

De Clouddienst draait op bare-metal servers in meerdere (Overheids)DataCenters (cf/ notitie Verkenning en Kamerbrief). Een nieuwe server komt binnen als inactieve hardware in een rack en moet worden omgezet naar een werkende Kubernetes-node in het juiste tenant-cluster zonder handmatige tussenkomst. Op de schaal van tienduizenden servers is elke vorm van handmatig installatiewerk economisch en operationeel onhoudbaar.

De architectuurvraag is: hoe verloopt de volledige provisioning-pipeline van lege hardware tot werkende Kubernetes-node, en hoe wordt die pipeline declaratief en auditeerbaar beheerd?

Dit ADR beschrijft het architectuurprincipe en de samenhang tussen de twee componenten die de pipeline realiseren. De afzonderlijke keuzes zijn vastgelegd in:

- **ADR-C60** — hardware provisioning: welk systeem verzorgt de fysieke laag (servers + switches)?
- **ADR-C20** — control plane topologie: welk framework beheert de cluster-lifecycle?

De eisen die de keuze inperken:

- **Aanbestedingswet en open source**: alle tooling moet open source zijn met brede community-ondersteuning.
- **Herhaalbaarheid**: dezelfde provisioning-pipeline werkt voor alle ODC-locaties.
- **Schaalbaarheid**: duizenden nodes per cluster, tientallen clusters per ODC.
- **Declaratieve state**: de gewenste toestand van servers en clusters is vastgelegd in Git en wordt daar declaratief tegen gereconcilieerd.
- **Audit trail**: elke provisioning-actie is traceerbaar (BIO, NIS2).
- **Geïntegreerd netwerk**: compute-provisioning en netwerk-provisioning verlopen als één pipeline; een mismatch tussen server-state en switch-state is niet acceptabel op schaal.

Besluit

De Clouddienst hanteert **volledig geautomatiseerde, declaratieve provisioning van bare-metal Kubernetes-clusters** via een tweeledige pipeline:

1. **Hardware-laag (ADR-C60)**: metal-stack verzorgt de geïntegreerde provisioning van servers én switches. Een nieuwe server wordt gedetecteerd, geïnspecteerd, voorzien van een OS en voorzien van de juiste netwerkconfiguratie (switch-poort, VRF, BGP-peering) in één geautomatiseerde keten.

2. **Cluster-lifecycle-laag (ADR-C20)**: Gardener neemt de geproviseerde node op in het juiste tenant-cluster. Gardener beheert de volledige cluster-lifecycle: aanmaken, schalen, upgraden, credential rotation, en verwijderen.

De grens tussen de twee lagen is scherp: metal-stack levert een werkende, netwerk-geconfigureerde node; Gardener bepaalt in welk cluster die node terechtkomt en beheert de cluster-state daarna.

Onderzochte implementaties

metal-stack + Gardener (gekozen) Geïntegreerde compute- én netwerk-provisioning (metal-stack) gecombineerd met Kubernetes-native cluster-lifecycle (Gardener). Beide zijn bewezen in productie; eén coherente stack, natieve integratie tussen de twee lagen. Zie ADR-C60 en ADR-C20 voor de volledige afweging per laag.

CAPI + Metal³ + Ironic (afgevallen) Cluster API als cluster lifecycle-framework, Metal³ en Ironic als bare-metal provisioning. Compute-only: netwerk-provisioning is een volledig apart probleem dat buiten deze stack valt. Ironic is onbewezen op de beoogde schaal van 50.000 servers.

MAAS (afgevallen) Bare-metal provisioning van Canonical. Compute-only, niet Kubernetes-native, commerciële support-afhankelijkheid. Afgevallen op dezelfde gronden als CAPI/Metal³.

Tinkerbell (afgevallen) Workflow-gebaseerde bare-metal provisioning, CNCF-sandbox. Sterk in initiële provisioning, zwak in cluster-lifecycle en netwerk-integratie. Afgevallen als complete provisioning-oplossing.

Technologiekeuze

metal-stack (hardware-laag) + **Gardener** (cluster-lifecycle-laag).

De twee componenten zijn complementair: metal-stack eindigt waar Gardener begint. Samen realiseren ze de volledige pipeline van lege hardware naar werkende tenant-cluster, declaratief beheerd via Git (ADR-C55).

Motivatie

De keuze voor een tweeledige pipeline is gedreven door de scheiding van verantwoordelijkheden. Hardware-provisioning en cluster-lifecycle zijn fundamenteel verschillende problemen een tool die beide oplost is ofwel beperkt in één van de twee, ofwel monolithisch en moeilijk te vervangen. Door de twee lagen expliciet te scheiden en elk met een gespecialiseerd systeem in te vullen, blijft de stack aanpasbaar.

De integratie van compute- en netwerk-provisioning in één tool (metal-stack) is een bewuste keuze. Op de schaal van de Clouddienst is een mismatch tussen server-state en switch-state een serieus operationeel risico. Elk handmatig koppelvlak is een potentieel consistentieprobleem. metal-stack elimineert dat risico door beide domeinen in één pipeline samen te brengen.

Consequenties

Volledige automatisering: van “server in rack” tot “node beschikbaar in tenant-cluster” verloopt zonder handmatige stappen. Elke afwijking van de gewenste toestand wordt automatisch gecorrigeerd via de GitOps-reconciliatie (ADR-C55).

Hardware-compatibiliteit: metal-stack vereist SONiC-compatibele switches. Dit is een directe eis aan de hardware-selectie (ADR-G13, ADR-G65). Zie ADR-C60 voor de volledige hardware-consequenties.

Gardener als cluster-lifecycle: de node-opname in een tenant-cluster verloopt via Gardener. Dit veronderstelt dat ADR-C20 definitief Gardener kiest. Als dat teamoordeel anders uitvalt, vervalt de samenhang van deze pipeline en moeten C01 en C60 opnieuw worden afgewogen.

Immutable OS: de OS-installatie op nodes (ADR-C02) is een onderdeel van de metal-stack provisioning-pipeline. Talos Linux of Flatcar wordt als OS-image meegegeven aan metal-stack.

Audit trail: alle provisioning-acties via metal-stack en Gardener worden gelogd naar de observability-stack (ADR-G41), conform BIO en NIS2.

Open vragen

1. **Afhankelijkheid van C20-besluit**: deze ADR is samenhangend met de Gardener-keuze in ADR-C20. Als het team Hypershift kiest, vervalt de metal-stack + Gardener-keten en moeten C01 en C60 opnieuw worden afgewogen tegen CAPI/Metal³.
2. **OS-installatie via metal-stack**: hoe wordt de Talos-image-pipeline (ADR-C23) geïntegreerd met de metal-stack provisioning? Dit is een implementatiedetail dat bij de technische uitwerking moet worden vastgelegd.

Sessienotities

> [Origineel] Metal³/Ironic vs metal-stack was open in ADR-C60. Dat ADR bepaalde de CAPI-basis; de bare-metal provider eronder kon later wijzigen.
>
> [2026-06-03] De keuze is herzien. C60 besluit metal-stack als hardware-provisioning-laag; C20 besluit Gardener als cluster-lifecycle-laag. C01 beschrijft de samenhang tussen beide.

ADR-C02: Host-besturingssysteem voor platformnodes

Status	**Vastgesteld**
Laag	Provisioning
Afhankelijk van	ADR-001
Maakt mogelijk	ADR-C23, ADR-C24, ADR-C07
Datum	2026-08-07

Context en probleemstelling

Elke fysieke node draait een besturingssysteem. Die keuze bepaalt het aanvalsoppervlak, de patchstrategie, het beheerparadigma en de auditeerbaarheid van de onderlaag. Op tienduizenden nodes is het traditionele model (SSH + configuration management) niet schaalbaar en niet veilig genoeg voor een SEAL-4-ambitie.

De architectuurvraag is: welk type besturingssysteem draait op de platformnodes?

Eisen:

- Open source met verifieerbare build-herkomst (SEAL-4, supply chain).
- Minimaal aanvalsoppervlak; elke configuratiewijziging traceerbaar.
- Declaratief beheerbaar; handmatige configuratie op duizenden nodes is onhoudbaar.
- Ondersteuning voor containerd, cgroups v2, eBPF (Cilium) en CSI-drivers.

Onderzochte implementaties

Flatcar Container Linux (Linux Foundation, Apache 2.0): Fork van CoreOS. Immutable filesystem, automatische updates via Omaha-protocol. SSH is standaard aanwezig maar kan worden uitgeschakeld. Lang productie-track-record in grootschalige Kubernetes-deployments. Governance onder de Linux Foundation. Breed ecosysteem en brede hardwareondersteuning.

Talos Linux (Sidero Labs, MPL 2.0): Volledig immutable, API-gedreven OS zonder SSH, shell of pakketbeheerder. Configuratie uitsluitend via `talosctl` of Kubernetes-CRD. Minimaal aanvalsoppervlak. Updates als atomaire A/B image-swap. Nadeel: ander paradigma vergt omscholing; Sidero Labs is commercieel, al is Talos zelf open source.

Garden Linux (SAP / Linux Foundation, Apache 2.0): Debian-gebaseerd minimaal OS ontwikkeld voor cloud-native Kubernetes-omgevingen. Geoptimaliseerd voor geautomatiseerde, reproduceerbare node-builds. Actieve ontwikkeling binnen de Linux Foundation. Beperkt productie-bewijs buiten het SAP-ecosysteem.

Bottlerocket (AWS, Apache 2.0): Immutable OS voor Kubernetes, open source maar sterk gericht op AWS-deployments. Beperkte bare-metal-ondersteuning. Afgevallen op ecosysteem-afhankelijkheid.

Fedora CoreOS / Red Hat: Semi-immutable via `rpm-ostree`. Sterk verbonden met OpenShift. Afgevallen op governance (Red Hat/IBM).

Traditioneel Linux (Ubuntu, RHEL, Debian): Ongeschikt voor de Clouddienst. Configuratiedrift onvermijdelijk, SSH-beheer is een audit-last, aanvalsoppervlak te groot.

Besluit

Flatcar Container Linux als primaire host-OS. Talos Linux als gevalideerd secundair alternatief voor specifieke use-cases waar het Flatcar-paradigma beperkend is.

Motivatie

Flatcar combineert een bewezen productiegeschiedenis met governance onder de Linux Foundation — relevant voor SEAL-4 (ADR-002). Het immutable filesystem en het Omaha-updateprotocol sluiten aan op de update-strategie (ADR-C23, ADR-C24). Het brede ecosysteem verlaagt de drempel voor hardwareondersteuning en driver-integratie ten opzichte van striktere alternatieven.

Talos biedt een strikter aanvalsoppervlak maar introduceert een hoger omscholingsrisico en een commerciële bestuursstructuur die Flatcar's Linux Foundation-governance niet heeft. Als secundair alternatief blijft Talos beschikbaar voor specifieke nodes of use-cases.

Garden Linux is inhoudelijk relevant maar heeft buiten het SAP-ecosysteem nog geen bewezen schaal op bare-metal. Wordt bij toekomstige evaluaties opnieuw bezien.

Consequenties

SSH-toegang op productienodes is uitgesloten. Incident response en debugging verlopen via log-aggregatie en de platform-API.

Node-configuratie wordt declaratief beheerd in Git (ADR-C55). Bestaande Ansible/Puppet-runbooks zijn niet bruikbaar; het beheerteam wordt opgeleid in het Flatcar-beheerparadigma.

Node-images met aanvullende kernel-modules (GPU-drivers voor ADR-C11) moeten reproduceerbaar worden gebouwd en ondertekend door de Clouddienst. Vertrouwen op upstream-images is voor SEAL-4 onvoldoende.

De rollback-strategie via A/B image-swap wordt uitgewerkt in ADR-C24.

Sessienotities

> [04-03-2026] Talos was initiële voorkeur op aanvalsoppervlak. Flatcar gekozen op productierijpheid, ecosysteem en Linux Foundation-governance. [07-07-2026] Garden Linux toegevoegd als onderzochte optie. Talos Omni (Sidero) als externe beheercomponent afgewezen wegens SEAL-4-concerns.

ADR-C03: Ondersteuning voor virtuele machines

Status	**Vastgesteld**
Laag	Compute
Afhankelijk van	ADR-001, ADR-C04
Maakt mogelijk	ADR-C05
Datum	2026-07-07

Context en probleemstelling

Het platform is primair container-native: Kubernetes is de orchestratielaag en containers zijn de eersteklas werklasteenheid. Maar een substantieel deel van de bestaande overheidsapplicaties draait op virtuele machines en kan niet op korte termijn worden gecontaineriseerd. Sommige applicaties zullen nooit worden gecontaineriseerd — denk aan legacy Windows-applicaties, specifieke middleware-stacks en applicaties die een volledig besturingssysteem vereisen.

De architectuurvraag is: hoe biedt het platform VM-ondersteuning aan tenants, zodanig dat VM's en containers naast elkaar draaien op dezelfde infrastructuur, beheerd worden via dezelfde API en tooling, en onderworpen zijn aan hetzelfde isolatie- en beveiligingsmodel?

De opdracht stelt expliciet dat het platform "virtuele machines moet kunnen hosten en beheren". De keuze voor het architectuurmodel (ADR-001) voor baremetal kubernetes is gemaakt om hypervisor technologie te vermijden.

De keuze heeft consequenties voor meerdere lagen:

- **Storage** (ADR-C05): VM-disks zijn persistent volumes zouden ReadWriteMany moeten ondersteunen, indien live migration een eis zou zijn.
- **Networking** (ADR-C07): VM's moeten geïntegreerd zijn in het Kubernetes-netwerkmodel met dezelfde NetworkPolicies en observability als containers, indien live migration een eis zou zijn.
- **Tenantisolatie** (ADR-C04): VM's moeten dezelfde isolatiegaranties bieden als containers — een VM van tenant A mag geen resources van tenant B kunnen benaderen.
- **Hardware-passthrough**: sommige workloads vereisen directe toegang tot GPU's of netwerkhardware (SR-IOV) voor performance. Dit doorbreekt de standaard abstractielaag en vereist specifieke configuratie.

Besluit

Het platform integreert VM-ondersteuning in de Kubernetes-orchestratielaag in plaats van een aparte virtualisatieoplossing naast Kubernetes te draaien. VM's worden Kubernetes-objecten: ze worden aangemaakt, beheerd en verwijderd via de Kubernetes API, net als pods. De hypervisor (KVM/QEMU) draait op dezelfde bare-metal nodes als de container-workloads.

Live migration wordt echter niet als harde eis geformuleerd; VMs kunnen per tenant aangeboden worden indien dat noodzakelijk is. Dat voorkomt harde eisen die doorwerken in de storage, network, en nodeconfiguratie.

Dit betekent dat er één beheerparadigma is voor het hele platform. Tenants beheren hun VM's met kubectl of via de Kubernetes API, in dezelfde namespace als hun containers. NetworkPolicies, RBAC en quota's gelden voor zowel containers als VM's. Het platformteam beheert één stack, niet twee.

Het standaard isolatieprofiel is KVM-hardwareversnelling: VM's draaien in een KVM-process met hardware-ondersteunde virtualisatie (Intel VT-x/AMD-V). Dit biedt sterke isolatie via de hardware-hypervisor, vergelijkbaar met traditionele virtualisatieplatformen. Voor workloads die extra isolatie vereisen (bijvoorbeeld DEP VERTROUWELIJK) kan Kata Containers worden ingezet, waarbij de VM in een extra isolatielaag draait.

Onderzochte implementaties

KubeVirt (Kubernetes-native VM's): CNCF-incubating project dat VM's als Kubernetes Custom Resources (VirtualMachine, VirtualMachineInstance) implementeert. Gebruikt KVM/QEMU als hypervisor. VM's worden gescheduled door de Kubernetes-scheduler en geïntegreerd in het Kubernetes-netwerk via de CNI. Storage via standaard PersistentVolumeClaims. Live migration is ingebouwd. Multus-ondersteuning voor meerdere netwerk-interfaces per VM. SR-IOV en GPU-passthrough worden ondersteund. Brede community, actief doorontwikkeld. Gebruikt door Red Hat (OpenShift Virtualization), SUSE (Harvester).

Traditionele hypervisor naast Kubernetes (VMware vSphere, Proxmox, oVirt): Een aparte virtualisatieoplossing draait naast Kubernetes op dezelfde of aparte hardware. VM's worden beheerd via een eigen managementinterface, los van Kubernetes. Voordeel: volwassen, bewezen technologie met decennia productie-ervaring. Nadeel: twee aparte beheerstacks, twee aparte netwerkconfiguraties, twee aparte isolatiemodellen. Geen uniforme API voor containers en VM's. Introduceert een afhankelijkheid van een niet-open-source platform (vSphere) of een minder breed geadopteerd open source platform (Proxmox, oVirt). Conflicteert met het principe van ADR-001 (één stack, Kubernetes-native).

Geen VM-ondersteuning (alleen containers): Het platform biedt uitsluitend container-workloads aan. Tenants die VM's nodig hebben worden verwezen naar een aparte voorziening of moeten hun applicaties containeriseren. Voordeel: eenvoudigere stack, geen virtualisatie-complexiteit. Nadeel: sluit een groot deel van de bestaande overheidsworkloads uit. Is in strijd met de opdracht die expliciet VM-ondersteuning vereist.

Technologiekeuze

Gekozen: KubeVirt

KubeVirt is de enige implementatie die VM's als eersteklas Kubernetes-objecten aanbiedt. Het sluit naadloos aan op het Kubernetes-native principe uit ADR-001: één API, één tooling, één beheerparadigma voor zowel containers als VM's. De CNCF-incubating status en de brede adoptie (Red Hat, SUSE) bieden continuïteitsgaranties.

Het standaard isolatieprofiel (KVM-hardwareversnelling) biedt voldoende beveiliging voor het overgrote deel van de workloads. De hardware-hypervisor isoleert VM's van elkaar en van het host-OS op CPU-niveau.

Motivatie

De keuze voor Kubernetes-native VM's in plaats van een aparte hypervisor is gedreven door het principe van één beheerstack. Een apart virtualisatieplatform naast Kubernetes zou betekenen: twee API's, twee netwerkconfiguraties, twee storagekoppelingen, twee monitoringsystemen, twee teams met verschillende expertisegebieden. Dat verdubbelt de operationele complexiteit voor het platformteam en de leercurve voor tenants.

KubeVirt elimineert deze verdubbeling. Een VM is een VirtualMachine-object in Kubernetes. Het wordt gescheduled door dezelfde scheduler, gemonitord door dezelfde Prometheus, bereikbaar via dezelfde CNI (Cilium), opgeslagen op dezelfde storage (Ceph). De tenant gebruikt dezelfde kubectl-commando's en dezelfde RBAC-structuur.

De optie "geen VM-ondersteuning" is afgevallen op de opdracht: het platform moet VM's ondersteunen. Veel overheidsapplicaties draaien op VM's en zullen dat nog jaren doen. Een platform zonder VM-ondersteuning is voor een groot deel van de overheid niet bruikbaar.

Een aparte hypervisor (vSphere, Proxmox) is afgevallen op het principe van leveranciersonafhankelijkheid (ADR-001) en het principe van één beheerstack. VMware vSphere is bovendien niet open source. Proxmox en oVirt zijn open source maar introduceren een volledig apart beheermodel dat niet integreert met de Kubernetes-tooling.

Consequenties

Alle nodes moeten KVM ondersteunen. De hardware moet Intel VT-x of AMD-V ondersteunen en dit moet zijn ingeschakeld in het BIOS/UEFI. Dit moet worden meegenomen in de hardware-specificaties en de firmware-configuratie (raakt aan ADR-C60 en de ontbrekende firmware-ADR).

Live migration vereist shared storage. Een VM die van node A naar node B migreert moet zijn disk op beide nodes kunnen benaderen. Dit vereist ReadWriteMany (RWX) storage, wat een harde eis is voor de storage-keuze (ADR-C05). Ceph/CephFS levert dit native; Longhorn niet.

Netwerk-integratie via Multus. KubeVirt VM's die meerdere netwerk-interfaces nodig hebben (bijvoorbeeld een management-interface en een data-interface) vereisen Multus CNI als meta-plugin bovenop Cilium. Dit voegt een extra netwerkcomponent toe aan de stack.

GPU-passthrough en SR-IOV. VM's die directe hardware-toegang nodig hebben (GPU voor AI-workloads, SR-IOV voor high-performance networking) vereisen specifieke node-configuratie: IOMMU moet zijn ingeschakeld, device-plugins moeten zijn gedeployed, en de betreffende nodes moeten worden gelabeld. Dit beïnvloedt de scheduling: niet elke node kan deze VM's draaien.

Resource-overhead. Elke VM draait als een KVM/QEMU-process in een pod. Dit heeft meer overhead dan een container maar minder dan een traditionele VM op een standalone hypervisor, omdat er geen apart hypervisor-OS tussen zit. De overhead is circa 100-200 MB per VM voor het QEMU-process plus de toegewezen VM-resources.

Kata Containers als extra isolatielaag. Voor workloads die sterkere isolatie vereisen dan standaard KVM (bijvoorbeeld DEP VERTROUWELIJK workloads die op gedeelde hardware draaien) kan Kata Containers worden ingezet. Kata draait containers en VM's in een extra micro-VM, wat een dubbele isolatielaag biedt. Dit is een optionele configuratie per workload, geen platformwijde verplichting. De performance-impact van Kata moet worden geëvalueerd.

Sessienotities

Standaardprofiel is meer dan voldoende voor de meeste use cases. Wanneer extra isolatie benodigd is kan Kata Containers ingezet worden. SR-IOV is standaard beschikbaar in het standaardprofiel, en voor partitionering van NIC's zou dit als extra use case meegenomen kunnen worden.

ADR-C04: Tenantisolatie-model op compute-niveau

Status	**Vastgesteld**
Laag	Compute
Afhankelijk van	ADR-001, ADR-G02
Maakt mogelijk	ADR-C20
Datum	2026-08-07

Context en probleemstelling

De Clouddienst bedient tientallen overheidsorganisaties met sterk uiteenlopende beveiligingseisen van openbare dienstverlening tot DEP VERTROUWELIJK werkprocessen. De fundamentele architectuurvraag is: op welk niveau wordt de scheiding tussen tenants gerealiseerd in de compute-laag?

Deze keuze is de belangrijkste isolatiebeslissing voor de Clouddienst. ADR-G02 stelt dat elke tenantorganisatie een eigen dedicated Kubernetes-cluster krijgt op fysieke nodes. ADR-C04 werkt dit principe uit op compute-niveau. De opties variëren van een gedeelde Kubernetes control plane met namespace-isolatie (goedkoop, zwakke scheiding) tot volledig gescheiden clusters per tenant (duur, sterkste scheiding). De keuze bepaalt:

- **Beveiligingsgaranties**: kan een kwetsbaarheid in de control plane van de ene tenant de andere raken?
- **Resource-allocatie**: kan een tenant gegarandeerd beschikken over toegezegde capaciteit zonder noisy-neighbor effecten?
- **Zelfbediening**: hoeveel beheerautonomie krijgt een tenant over zijn eigen omgeving?
- **Operationele complexiteit**: hoeveel clusters moet de Clouddienst beheren?
- **Resourcekosten**: elke extra control plane kost geheugen en CPU die niet voor workloads beschikbaar is.

De eisen die de keuze inperken:

- BIO2 en EUCS verplichte aantoonbare isolatie tussen tenants is nodig. Een gedeelde control plane vereist aanvullende bewijslast dat isolatie waterdicht is.
- Tenants moeten eigen RBAC, eigen NetworkPolicies en eigen admission control kunnen inrichten zonder de Clouddienst of andere tenants te raken.
- De Aanbestedingswet vereist dat de Clouddienst geen vendor lock-in introduceert hierdoor mag de isolatieoplossing niet leverancier specifek zijn en moet dus op standaarden gebaseerd zijn.

Besluit

De Clouddienst realiseert tenantisolatie op clusterniveau: elke tenant krijgt een of meedere eigen, dedicated Kubernetes-cluster met een eigen control plane (API server, etcd, controller-manager, scheduler). De tenant heeft volledige beheertoegang tot het eigen cluster maar geen enkele zichtbaarheid op of toegang tot clusters van andere tenants of het management-cluster.

We realiseren tenentisolaie op het hoogste niveau dat mogelijk is binnen Kubernetes (ADR-C20). Er is geen gedeeld etcd, geen gedeelde API server, geen gedeeld scheduler-pad tussen tenants. Een kwetsbaarheid

in de control plane van tenant A kan per definitie niet de control plane van tenant B raken, omdat het fysiek gescheiden processen zijn.

Onderzochte implementaties

Cluster API (dedicated clusters): Elke tenant krijgt een volledig eigen Kubernetes-cluster, geprovisioned via Cluster API. De control plane draait als containers op het management-cluster (in combinatie met ADR-C20 Hosted Control Planes) of als aparte nodes. De tenant krijgt cluster-admin toegang tot het eigen cluster. Lifecycle management (aanmaken, schalen, upgraden, verwijderen) verloopt declaratief via CAPI-resources.

Dedicated clusters via Gardener: Hetzelfde principe geïmplementeerd via Gardener's eigen framework (Garden / Seed / Shoot architectuur). Gardener beheert naast provisioning ook autoscaling, credential rotation en automatische upgrades. Valt onder de NeoNephos Foundation (Linux Foundation Europe). Relevante factor voor SEAL-4 soevereiniteit (ADR-002).

vCluster (virtuele clusters): Elke tenant krijgt een virtueel cluster dat draait als set pods in een gedeeld host-cluster. De virtuele cluster heeft een eigen API server en een eigen (gesimuleerde) etcd, maar deelt de onderliggende worker-nodes en het scheduling-mechanisme met andere tenants. Lagere resource-overhead per tenant (geen dedicated etcd, geen dedicated worker-nodes). De isolatie is minder sterk: de workloads draaien uiteindelijk op gedeelde nodes en het host-cluster is een single point of failure voor alle tenants.

Kamaji (gedeelde control plane met tenant-partitionering): Eén centrale control plane bedient meerdere tenants via gepartitioneerde API-toegang. De laagste resource-overhead maar de zwakste isolatie. Een bug of overbelasting in de gedeelde control plane raakt alle tenants. Kamaji is een relatief jong project met een kleinere community.

Namespace-isolatie (geen apart cluster): Tenants krijgen elk een of meer namespaces in een gedeeld cluster. Isolatie via RBAC, NetworkPolicies en ResourceQuotas. De goedkoopste optie maar de zwakste isolatie — er zijn bekende escalatiepaden van namespace-isolatie naar cluster-niveau als RBAC niet perfect is geconfigureerd. Niet geschikt voor DEP VERTROUWELIJK workloads.

Technologiekeuze

Het architectuurprincipe: dedicated clusters per tenant. Is vastgesteld. Twee implementatiepaden voldoen aan dit principe en blijven beiden in overweging: | Eigenschap | Cluster API | Gardener | |---|---|---| | Eigen control plane per tenant | ✓ | ✓ | | Geen gedeeld etcd | ✓ | ✓ | | Declaratief lifecycle beheer | ✓ | ✓ | | Kubernetes-native API | ✓ | ✓ (eigen CRDs) |

De keuze tussen beide paden — inclusief de SEAL-4-afweging en de integratie met de bare-metal provisioning-stack wordt gemaakt in **ADR-C20**.

Motivatie

De keuze is gedreven door één overheersend principe: op een platform dat overheidsorganisaties bedient met workloads tot DEP VERTROUWELIJK, moet de isolatie zo sterk zijn dat ze niet afhankelijk is van correcte configuratie. Bij namespace-isolatie is één fout in RBAC voldoende om de scheiding te doorbreken. Bij vCluster is een kernel-kwetsbaarheid op de gedeelde worker-node voldoende. Bij dedicated clusters is de isolatie structureel en geen kwestie van juiste configuratie maar van architectuur.

De hogere resourcekosten worden geaccepteerd. Elke tenant-control-plane kost circa 2-4 GB geheugen (API server + etcd + controllers). Bij 50 tenants is dat 100-200 GB — substantieel maar beheersbaar op een platform van 50.000 servers. De combinatie met Hosted Control Planes (ADR-C20) drukt deze kosten verder door control planes als containers te draaien in plaats van als dedicaede VM's.

vCluster is afgevallen ondanks de aantrekkelijk lagere kosten. De isolatiegarantie is onvoldoende voor DEP VERTROUWELIJK: de workloads delen worker-nodes en een gecompromitteerde node geeft potentieel toegang tot pods van meerdere tenants. Kamaji is afgevallen op volwassenheid en community-omvang.

Consequenties

De Clouddienst moet tientallen tot honderden Kubernetes-clusters beheren. Dit vereist volledige automatisering van cluster-lifecycle (aanmaken, upgraden, schalen, verwijderen). Handmatig clusterbeheer is op deze schaal onmogelijk.

Elke tenant-cluster heeft een eigen Kubernetes-versie en eigen configuratie. Het upgradebeleid (ADR-G52) moet afdwingen dat tenants niet oneindig op verouderde versies blijven draaien.

De resource-overhead per tenant is hoger dan bij gedeelde modellen. De inrichting van Hosted Control Planes (ADR-C20) is een directe vervolg-ADR om deze overhead te minimaliseren.

De self-service ervaring voor tenants is sterk: ze krijgen een volledig eigen cluster met cluster-admin rechten. De grens is helder — de tenant beheert alles binnen het cluster, de Clouddienst beheert alles erbuiten (hardware, netwerk, control plane lifecycle).

ADR-C05: Gedistribueerde storage strategie voor bare-metal Kubernetes

Status	Goud
Laag	Storage
Afhankelijk van	ADR-C01, ADR-C02, ADR-C04
Maakt mogelijk	ADR-C06, ADR-G41
Datum	2026-08-07

Context en probleemstelling

De Clouddienst op bare-metal Kubernetes heeft een gedistribueerde storage-laag nodig die drie typen opslag levert: block storage voor VM-disks (via KubeVirt) en persistent volumes voor containers, file storage via gedeelde bestandssystemen (POSIX-compatible) voor workloads die gedeelde toegang vereisen, en object storage met een S3-compatibele API voor ongestructureerde data. De object storage-keuze wordt in ADR-C06 apart behandeld maar is afhankelijk van de hier gemaakte keuze.

De storage-oplossing moet volledig on-premises draaien in de OverheidsDataCenters, meerdere opslagklassen ondersteunen (snelle NVMe SSD voor productiedatabases, SATA SSD voor algemene workloads, HDD voor archivering), en integreren via de standaard Kubernetes CSI-interface. De oplossing moet daarnaast voldoen aan de volgende harde eisen die voortvloeien uit eerdere ADR's:

- ReadWriteMany (RWX) volumes voor KubeVirt live migration (ADR-C03): wanneer een VM wordt gemigreerd tussen nodes moet de storage gelijktijdig door de bron- en doelnode worden benaderd.
- Data-at-rest encryptie per opslagklasse (BIO, EUCS SEAL-4): alle opgeslagen data moet versleuteld zijn.
- Tenantisolatie op storage-niveau (ADR-C04, ADR-G02): data van de ene tenant mag niet toegankelijk zijn voor een andere tenant, ook niet via het opslagsysteem.
- Replicatie over meerdere ODC-locaties (ADR-G01): voor disaster recovery moet data over minimaal twee locaties worden gerepliceerd.
- Disaster Recover, met off-site opslag. Een gerichte aanval op gezamenlijke ODC's zou in het huidige geopolitiek tijdsbeeld een reëel dreigingsscenario kunnen zijn.
- Voldoende performance voor GPU-workloads (capabilities-document): AI-trainingsworkloads vereisen hoge doorvoer bij het lezen van grote datasets.
- Snapshot, clone en backup-functionaliteit voor zowel VM-disks als persistent volumes.

De hoge integratielast van het gekozen architectuurmodel maakt het belangrijk dat de storage-stack een duidelijk operationeel model heeft, goed aansluit op het immutable OS (ADR-C02) en de Cluster API provisioning (ADR-C01), en bij voorkeur brede CNCF-community-ondersteuning geniet.

Er zijn drie serieuze kandidaten geïdentificeerd.

Optie 1: Rook/Ceph

Rook is een CNCF-graduated project dat Ceph als Kubernetes-native storage orkestrator aanbiedt. Ceph is een volwassen, gedistribueerd opslagsysteem dat block storage (RBD), file storage (CephFS) en object

storage (RadosGW) levert vanuit één unified platform. Rook beheert de Ceph-cluster als Kubernetes-operator.

Voordelen: Ceph is het meest volwassen open source gedistribueerde opslagsysteem met meer dan vijftien jaar ontwikkeling. Het levert alle drie de opslagtypen (block, file, object) vanuit één systeem, wat de operationele complexiteit vermindert doordat er maar één storage-platform hoeft te worden beheerd. ReadWriteMany wordt native ondersteund via CephFS, wat essentieel is voor KubeVirt live migration. Ceph schaalt bewezen naar petabyte-niveau. De RADOS-laag biedt configureerbare replicatie (standaard 3x) en erasure coding voor efficiënte opslag van grote datasets. Rook is CNCF-graduated en heeft een actieve community. Encryptie-at-rest wordt ondersteund via dm-crypt/LUKS op OSD-niveau. Ceph biedt native ondersteuning voor meerdere opslagklassen (pools met verschillende replicatiefactoren en mediatypen). Replicatie over meerdere locaties is mogelijk via RBD mirroring en CephFS snapshots.

Nadelen: Ceph is operationeel complex. Het vereist diepgaande kennis van RADOS, plaatsingsgroepen, CRUSH-maps en OSD-management. De minimale opstelling vereist minstens drie nodes met elk minimaal één OSD, wat de instapdrempel voor kleine omgevingen verhoogt. Performance-tuning vereist expertise: verkeerde CRUSH-configuratie kan leiden tot ongelijke datadistributie. Upgrades van Ceph-versies zijn zorgvuldig en vergen planning. De monitoring van een Ceph-cluster is complex en vereist specifieke tooling (Ceph Dashboard, Prometheus exporters).

Optie 2: Longhorn

Longhorn is een CNCF-incubating project, oorspronkelijk ontwikkeld door Rancher/SUSE. Het biedt gedistribueerde block storage voor Kubernetes, specifiek ontworpen voor cloud-native omgevingen.

Voordelen: Longhorn is operationeel eenvoudiger dan Ceph. Het gebruikt geen aparte storage-daemons maar repliceert data via een eigen engine die als container draait. De installatie en het beheer zijn eenvoudiger. Longhorn biedt ingebouwde backup naar S3-compatibele opslag, snapshot-functionaliteit en een overzichtelijke UI. Het is lichtgewicht en geschikt voor kleinere omgevingen. Incremental snapshots en backup werken out-of-the-box.

Nadelen: Longhorn biedt uitsluitend block storage. Er is geen file storage (geen CephFS-equivalent) en geen object storage. Voor RWX-volumes is Longhorn afhankelijk van NFS-provisioning bovenop block volumes, wat minder performant en minder betrouwbaar is dan native RWX. Dit is een fundamenteel probleem voor KubeVirt live migration. De schaalbaarheid is beperkter dan Ceph: Longhorn is niet ontworpen voor petabyte-schaal. Er is geen ondersteuning voor erasure coding, wat betekent dat de opslagoverhead bij 3x replicatie altijd 200% is. Replicatie over meerdere datacenterlocaties wordt niet native ondersteund. De community is kleiner dan die van Ceph.

Optie 3: OpenEBS

OpenEBS is een CNCF-sandbox project dat meerdere storage-engines aanbiedt: Mayastor (high-performance NVMe-based block storage), cStor (feature-rich block storage) en LocalPV (lokale opslag zonder replicatie).

Voordelen: OpenEBS biedt flexibiliteit door de keuze uit meerdere engines per use case. Mayastor levert hoge performance via NVMe-over-Fabrics (NVMe-oF) en is geschikt voor latency-gevoelige workloads. LocalPV is geschikt voor workloads die geen replicatie nodig hebben (bijvoorbeeld lokale caches). De modulaire aanpak maakt het mogelijk om per workload-type de best passende engine te kiezen.

Nadelen: De modulariteit is ook een nadeel: het beheren van meerdere storage-engines verhoogt de operationele complexiteit. Mayastor is relatief jong en heeft een kleinere community dan Ceph. Net als Longhorn biedt OpenEBS uitsluitend block storage — er is geen native file storage of object storage.

RWX-ondersteuning is beperkt. De replicatie over meerdere locaties is niet volwassen. OpenEBS is CNCF-sandbox (niet graduated of incubating), wat een lager volwassenheidsniveau impliceert. cStor wordt niet meer actief doorontwikkeld.

Besluit

Rook/Ceph is gekozen als primaire gedistribueerde storage-oplossing voor block-, file- en object storage. Object storage wordt geleverd via Ceph RadosGW als onderdeel van hetzelfde unified platform (zie ADR-C06). Er wordt geen apart object storage-systeem naast Ceph ingezet.

De keuze sluit Longhorn en OpenEBS uit als primaire storage-laag. Beide ontbreken native RWX-ondersteuning voor KubeVirt live migration (ADR-C03) en leveren niet alle drie de opslagtypen vanuit één unified platform.

Motivatie

De keuze wordt gedreven door drie harde eisen die de andere opties uitsluiten of ernstig beperken.

Ten eerste: ReadWriteMany voor KubeVirt live migration. Het platform moet VM's kunnen migreren tussen nodes zonder downtime (ADR-C03). Dit vereist dat de storage gelijktijdig door meerdere nodes kan worden benaderd. Ceph levert dit native via CephFS. Longhorn en OpenEBS ondersteunen RWX alleen via NFS-workarounds, wat minder betrouwbaar is en een single point of failure introduceert.

Ten tweede: alle drie de opslagtypen in één platform. De Clouddienst heeft block, file én object storage nodig. Ceph levert alle drie vanuit één unified platform (RBD, CephFS, RadosGW). Bij Longhorn of OpenEBS zouden er twee of drie aparte systemen nodig zijn — een block storage systeem plus een apart file storage systeem plus een apart object storage systeem — wat de operationele complexiteit verdrievoudigt.

Ten derde: bewezen schaalbaarheid naar petabyte-niveau. De doelomvang van 50.000 servers genereert een opslagvraag die naar petabytes groeit. Ceph is bewezen op deze schaal bij telecombedrijven, wetenschappelijke instellingen en cloudproviders. Longhorn en OpenEBS/Mayastor zijn niet bewezen op deze schaal.

De operationele complexiteit van Ceph wordt geaccepteerd als investering. De Clouddienst investeert al in diepgaande Kubernetes-expertise (ADR-001); Ceph-expertise is een logische uitbreiding daarvan. Rook als Kubernetes-native operator vereenvoudigt het beheer aanzienlijk ten opzichte van een standalone Ceph-installatie. De Rook-operator beheert de volledige lifecycle van de Ceph-cluster, inclusief het toevoegen en verwijderen van OSD's, upgrades en health monitoring.

De keuze voor Rook/Ceph maakt de object storage-keuze in ADR-C06 eenvoudiger: Ceph RadosGW is dan de logische keuze voor S3-compatibele object storage, omdat het dezelfde onderliggende RADOS-laag gebruikt en geen apart systeem vereist.

Consequenties

De keuze voor Rook/Ceph heeft de volgende consequenties:

Alle block storage (persistent volumes voor containers en VM-disks voor KubeVirt) wordt geleverd via Ceph RBD, geïntegreerd met Kubernetes via de Ceph CSI-driver. File storage wordt geleverd via CephFS, eveneens via CSI. Object storage wordt geleverd via Ceph RadosGW (uit te werken in ADR-C06).

Er moeten meerdere opslagklassen (Kubernetes StorageClasses) worden gedefinieerd: een snelle klasse op NVMe SSD-pools voor productiedatabases en latency-gevoelige workloads, een standaardklasse op SATA SSD-pools voor algemene workloads, en een archiefklasse met erasure coding op HDD-pools voor langetermijnopslag en backups.

Het replicatiemodel wordt per pool geconfigureerd: productie-pools met 3x replicatie binnen een ODC-locatie, aangevuld met asynchrone replicatie naar een tweede ODC-locatie via RBD mirroring voor disaster recovery. De RPO en RTO per opslagklasse moeten worden vastgesteld.

Encryptie-at-rest wordt geïmplementeerd via dm-crypt/LUKS op OSD-niveau. Elke OSD-disk wordt versleuteld met een unieke sleutel. Het sleutelbeheer wordt geïntegreerd met de KMS-architectuur (samenhang met ADR-G06 en de PKI-strategie).

Tenantisolatie op storage-niveau wordt gerealiseerd via aparte Ceph-pools of namespaces per tenant, gecombineerd met Kubernetes RBAC op de StorageClass- en PersistentVolumeClaim-objecten. Quota's worden per tenant ingesteld op pool-niveau.

De Clouddienst moet Ceph-expertise opbouwen. Dit omvat kennis van CRUSH-maps, plaatsingsgroepen, OSD-lifecycle, pool-management, performance-tuning en troubleshooting. Training en het opbouwen van operationele runbooks zijn vereist voordat de Clouddienst in productie gaat.

De Ceph-cluster vereist dedicated OSD-nodes of colocatie van OSD's op de compute-nodes. De keuze hiertussen hangt af van de hardwareconfiguratie en wordt vastgesteld bij de hardware-inkoop. Minimaal drie OSD-nodes per ODC-locatie zijn vereist, met een aanbevolen minimum van vijf voor productiestabiliteit.

De monitoring van de Ceph-cluster wordt geïntegreerd met de platformbrede observability-stack (samenhang met ADR-G41). Ceph exporteert metrics via Prometheus en biedt een eigen dashboard dat optioneel kan worden ontsloten.

Sessienotities

> [20-04-2026] Bij storage BINNEN het cluster is dit een logisch besluit. Wanneer daarnaast ook wenselijk wordt om buiten het cluster storage aan te bieden, dan ligt voor de hand om daarnaast losse Ceph cluster(s) aan te bieden. [Noot bij statuswijziging Zilver] De keuze voor Rook/Ceph is vastgesteld. Performance-benchmarks en een integratietest met KubeVirt live migration zijn validatietaken in de implementatiefase, geen blokkerende voorwaarden voor het architectuurbesluit. ADR-C03 en ADR-C06 hangen hier al hard van af.

ADR-C06: Object storage voor ongestructureerde data

Status	Goud
Laag	Storage
Afhankelijk van	ADR-C05
Datum	2026-04-17

Context en probleemstelling

Naast block en file storage (ADR-C05) heeft het platform een S3-compatibele object storage nodig voor ongestructureerde data: documenten, backups, datasets, container images en AI-trainingsdata. Object storage heeft fundamenteel andere kenmerken dan block storage: het wordt benaderd via HTTP/S3, schaalt horizontaal naar petabytes, en is geoptimaliseerd voor hoge doorvoer bij grote objecten.

De architectuurvraag is: wordt object storage geleverd als onderdeel van het unified storage-platform (ADR-C05), of als aparte gespecialiseerde dienst?

De eisen:

- **S3-compatibele API** — de-facto standaard die alle applicaties ondersteunen
- **Petabyte-schaalbaarheid** — overheidsdata groeit onvoorspelbaar
- **Tenantisolatie** — buckets en data gescheiden per tenant
- **Encryptie-at-rest** per tenant of per bucket
- **Multi-site replicatie** naar tweede ODC voor disaster recovery

Besluit

Ceph RadosGW is gekozen als object storage-oplossing, als onderdeel van het Rook/Ceph unified platform (ADR-C05). Er wordt geen apart object storage-systeem naast Ceph ingezet.

De kernafweging — unified platform (lagere operationele complexiteit) versus een gespecialiseerde oplossing — is beslecht in het voordeel van het unified platform. Één RADOS-laag levert block (RBD), file (CephFS) én object (RadosGW): één team, één set runbooks, één upgrade-cyclus.

Onderzochte implementaties

Ceph RadosGW (onderdeel van Rook/Ceph): S3-compatibele gateway bovenop dezelfde RADOS-laag als block en file storage. Voordeel: als ADR-C05 uitkomt op Rook/Ceph, is RadosGW “gratis” — geen apart systeem, dezelfde OSD’s, dezelfde monitoring, dezelfde runbooks. Cross-site replicatie via ingebouwde multi-site sync. Nadeel: S3-performance is lager dan gespecialiseerde oplossingen bij kleine objecten en hoge concurrency. De RadosGW-gateway is een extra component die moet worden geschaald.

SeaweedFS: Lichtgewicht gedistribueerd bestandssysteem met S3-gateway. Goede performance, eenvoudige architectuur. Nadeel: kleinere community dan Ceph en MinIO, minder productie-ervaring op overheidsschaal. CNCF-status ontbreekt.

Technologiekeuze

Ceph RadosGW (onderdeel van Rook/Ceph, vastgesteld in ADR-C05). Geen apart systeem. Dezelfde OSD's, dezelfde monitoring, dezelfde runbooks als de block- en file storage-laag.

Motivatie

De voorkeur voor een unified platform sluit aan bij het principe van operationele eenvoud. Elk extra systeem in de stack is een extra faalpunt, een extra upgrade-cyclus en een extra expertisegebied. Ceph draait al voor block en file; RadosGW is de directe consequentie van die keuze, niet een aparte afweging.

Consequenties

Bij de keuze voor RadosGW: de RadosGW-gateways moeten worden geschaald voor de verwachte S3-traffic. Per ODC zijn minimaal twee gateways nodig voor beschikbaarheid.

Sessienotities

> Besluit volgt direct uit ADR-C05 (Rook/Ceph). RadosGW is de logische consequentie van het unified platform-principe. MinIO blijft beschikbaar als aanvullende optie indien benchmarks dit rechtvaardigen.

ADR-C07: Dataplane-strategie voor multi-tenant netwerkisolatie binnen bare-metal clusters

Status	Goud
Laag	Networking
Afhankelijk van	ADR-001, ADR-C02, ADR-G02, ADR-C20
Maakt mogelijk	ADR-C08, ADR-C38, ADR-C43
Datum	2026-06-01

Context en probleemstelling

De Clouddienst bedient meerdere overheidsorganisaties op gemeenschappelijke fysieke infrastructuur. Het netwerkverkeer tussen pods, nodes en tenants moet worden geïsoleerd, gefilterd en observeerbaar zijn.

Netwerkisolatie in de Clouddienst wordt op drie onderscheiden lagen gerealiseerd. Deze ADR beslist over één daarvan; de andere twee worden hier benoemd omdat ze anders impliciet aan de dataplane worden toegeschreven.

Laag	Waar	Wat de tenantgrens is	Besloten in
1. Fabric	Datacenternetwerk, switches	Fysieke en logische scheiding in de underlay (VLAN/VRF/VXLAN)	ADR-G56 (nog geen besluit), ADR-C60
2. Pod-netwerk binnen een tenant-cluster	Worker-nodes, exclusief per tenant (ADR-G02)	Hardware: andere tenants draaien op andere nodes. De dataplane regelt verkeer *binnen* het cluster	**Deze ADR**
3. Pod-netwerk van het gedeelde management-cluster	Nodes van het management-cluster (ADR-C20)	eBPF-policy en namespace-scheiding: de gehoste control planes van meerdere tenants draaien als pods op dezelfde nodes	**Deze ADR**, met de consequenties hieronder

De isolatiehiërarchie is daarmee: de scheiding van tenant-*workloads* is structureel geborgd in hardware. Elke tenant draait op eigen, dedicated worker-nodes (ADR-G02, minimaal drie per tenant) en deelt daar geen kernel met een andere tenant; de fabric (laag 1) scheidt dat verkeer in de underlay. De dataplane is op laag 2 de handhavingslaag binnen een cluster, niet de bron van de cross-tenant-scheiding.

Voor de *besturing* van die clusters geldt dat niet. De gehoste control planes (API server, etcd, controller-manager, scheduler) van meerdere tenants draaien als pods in aparte namespaces op dezelfde nodes van een management-cluster (ADR-C20). Daar is de dataplane wél de tenantgrens.

De architectuurvraag is: hoe wordt netwerkverkeer binnen de clusters van de Clouddienst gerouteerd, gefilterd en geobserveerd, zodanig dat verkeer tussen workloads en tussen control planes afdwingbaar is gescheiden, dat de Clouddienst voldoet aan BIO en NIS2 eisen voor netwerkzonering, en dat het operationele team inzicht heeft in verkeersstromen zonder afhankelijk te zijn van externe enterprise-tooling?

De eisen die voortvloeien uit eerdere besluiten:

Bare-metal deployment (ADR-001): er is geen cloud-provider die overlay-netwerken levert. De CNI moet direct op de fysieke infrastructuur werken. Immutable OS (ADR-C02): de CNI moet functioneren zonder kernel-modules die handmatig worden geladen of gecompileerd. Bij Talos Linux is er geen SSH-toegang om dit te doen. Multi-tenancy (ADR-G02): netwerkpolicies moeten per tenant afdwingbaar zijn, met default-deny als uitgangspunt. Tenants draaien op dedicated worker-nodes; de dataplane vult die scheiding aan en vervangt haar niet. Hosted control planes (ADR-C20): de control planes van alle tenants draaien als pods op een gedeeld management-cluster. Daar moet de dataplane de scheiding tussen tenants zelf afdwingen. DEP VERTROUWELIJK (ADR-G10/G38): versleuteling van intra-cluster verkeer moet mogelijk zijn.

Besluit

De Clouddienst implementeert netwerkisolatie op kernel-niveau via eBPF-gebaseerde dataplane-verwerking. Dit betekent dat netwerkpolicies, routing, loadbalancing en observability worden afgehandeld in de Linux-kernel zelf, zonder iptables-ketens of userspace-proxies. Dit levert de laagste latency, de hoogste doorvoer en de beste observability op bare-metal.

Dit besluit geldt voor beide clustertypes, met een verschillende rol per type. In de tenant-clusters handhaaft de dataplane het verkeer tussen workloads binnen het cluster; de scheiding tussen tenants is daar al structureel geborgd door dedicated worker-nodes. In het gedeelde management-cluster is de dataplane wel de primaire tenantgrens tussen de gehoste control planes; daar geldt een sluitend default-deny basisbeleid dat buiten de invloedssfeer van tenants valt.

Het platformteam is eigenaar van het isolatieresultaat, niet van de individuele policy-objecten. Het cluster-lifecycle-framework (ADR-C20) rolt in de namespaces van de gehoste control planes zijn eigen netwerkpolicies uit en reconcilieert die; het basisbeleid wordt daarom aangebracht langs de weg die dat framework daarvoor biedt. GitOps (ADR-C12, ADR-C55) geldt voor de configuratie die het framework consumeert, niet voor de objecten die het framework zelf genereert.

De ingebouwde observability — zichtbaarheid in elke netwerkflow zonder apart te installeren tooling — is een architectuurprincipe, niet alleen een feature. De Clouddienst moet op elk moment kunnen aantonen welk verkeer tussen welke workloads loopt, zonder afhankelijkheid van derde partijen.

Motivatie

De keuze voor eBPF-native dataplane-verwerking is gedreven door drie architectuurprincipes:

Ten eerste: kernel-level enforcement zonder tussenstap. Bij iptables-gebaseerde CNI's worden netwerkpolicies vertaald naar chains van regels die sequentieel worden doorlopen. Bij duizenden pods en honderden policies wordt dit een performance-bottleneck. eBPF-programma's worden direct in de kernel geladen en verwerken verkeer zonder deze overhead. Op bare-metal — waar er geen cloud-provider loadbalancer als vangnet fungeert — is deze directe verwerking essentieel.

Ten tweede: observability als platformcapability, niet als add-on. De domeinarchitectuur vereist "doorgifte van log-informatie" en "monitoring" als basiscapabilities. Door observability in de dataplane in te bouwen — elke netwerkflow is zichtbaar zonder extra tooling — wordt dit een eigenschap van de Clouddienst

zelf, niet iets dat er achteraf bovenop wordt geschroefd. Dit versterkt de compliancepositie voor NIS2 en BIO.

Ten derde: minimale afhankelijkheid van kernel-configuratie. Op een immutable OS (ADR-C02) kan er geen kernel-module worden gecompileerd of geladen via SSH. eBPF-programma's worden geladen via de BPF-syscall die standaard beschikbaar is in moderne kernels, zonder extra modules. Dit sluit naadloos aan op het immutable-infrastructure-principe.

Consequenties

Alle netwerkfunctionaliteit — routing, filtering, loadbalancing, observability — loopt primair via de gekozen eBPF-native CNI. Dit is een single point of dependency voor de volledige netwerkstack. Als de gekozen implementatie een kritieke bug heeft, raakt dat alle tenants.

In het management-cluster is de tenantgrens software, geen hardware. Dit raakt niet de tenant-workloads: die draaien op dedicated worker-nodes per tenant en zijn structureel gescheiden (ADR-G02). Het raakt de gehoste control planes: die van meerdere tenants draaien als pods op dezelfde nodes van een management-cluster en delen daar dus de kernel (ADR-C20). De scheiding tussen tenants is op dat punt een kwestie van correcte eBPF-policy en namespace-configuratie. Bij een misconfiguratie of een omissie in het basisbeleid is netwerkverkeer tussen control planes van verschillende tenants mogelijk. Dit is het zwakste punt in de isolatieketen en het staat haaks op het uitgangspunt van ADR-C04 dat isolatie niet van correcte configuratie afhankelijk mag zijn. De maatregelen die dit beperken:

- Een expliciete deny-all per control-plane-namespace, bovenop de policies die het cluster-lifecycle-framework zelf uitrolt. Zulke frameworks leveren doorgaans een allow-set voor systeemverkeer (DNS, kubelet, VPN, API server) maar geen sluitende deny-all; die moet dus bewust worden toegevoegd, via het daarvoor bedoelde mechanisme van het framework en niet als een parallelle stroom die met de reconciliatie ervan conflicteert.
- Geautomatiseerde verificatie van de *effectieve* policy-set bij elke wijziging, ongeacht welke component de afzonderlijke objecten heeft aangemaakt. Dit is de enige toets die blijft werken als het framework zijn eigen policies herschrijft, en daarmee de kern van de maatregel.
- Flow-logging en alerting op elk verkeer tussen namespaces van verschillende tenants in het management-cluster; dergelijk verkeer is per definitie een incident.
- Versleuteling van intra-cluster verkeer tussen control plane-componenten (ADR-G38, ADR-C38).
- Voor tenants met verhoogde isolatievereisten blijft de uitzondering uit ADR-C20 beschikbaar: dedicated control plane-nodes of een afgeschermd management-cluster per classificatieniveau.

De fabric-laag is nog niet besloten. De structurele scheiding waarop laag 2 leunt, veronderstelt een underlay die tenantverkeer gescheiden houdt. ADR-G56 heeft daarover nog geen besluit. Zolang dat ontbreekt is de isolatieketen van de Clouddienst niet compleet beschreven, ook al is deze ADR zelf op Goud.

Het platformteam moet eBPF-debugging-expertise opbouwen. Traditionele tcpdump-en-iptables-troubleshooting werkt niet; de tooling is anders (bpftool, Hubble CLI, cilium monitor).

De keuze voor kernel-level dataplane-verwerking veronderstelt dat de kernel op alle nodes eBPF ondersteunt. De minimale kernelversie is 4.19; voor volledige functionaliteit is 5.10+ aanbevolen. Dit moet worden meegenomen in de OS-image-pipeline (ADR-C23/C24).

Vervolgbeslissingen over loadbalancing (ADR-C08), service mesh (ADR-C38) en IP-pool-beheer (ADR-C43) bouwen voort op deze keuze.

Sessienotities

Cilium is industriestandaard, beste performance op bare-metal. Vanuit security-oogpunt de beste observability, ingebouwd in plaats van enterprise-oplossingen zoals bij Calico.

Reviewopmerking (augustus 2026): de ADR presenteerde drie netwerklagen als één en claimde daarmee het fundament te zijn onder alle netwerkbeveiliging, terwijl cross-tenant-isolatie in dit ontwerp uit hardware en fabric komt. Ook was het gedeelde management-cluster niet als consequentie benoemd. Beide zijn hierboven verwerkt; de technische keuze voor een eBPF-native dataplane is ongewijzigd.

ADR-C08: Bare-metal loadbalancing: service-exposure en IP-advertentie

Status	Goud
Laag	Networking
Afhankelijk van	ADR-C07
Maakt mogelijk	ADR-C43
Datum	2026-04-20

Context en probleemstelling

Kubernetes biedt op bare-metal geen ingebouwde LoadBalancer-implementatie. In hyperscale cloud-omgevingen wordt dit opgelost door de cloudprovider (AWS ELB, GCP LB, Azure LB). Op een bare-metal platform moet deze functionaliteit expliciet worden geleverd: een mechanisme dat per Service van type `LoadBalancer` een extern bereikbaar IP toewijst en dat IP via het netwerk adverteert zodat verkeer het cluster bereikt.

De architectuurvraag is: welk mechanisme levert LoadBalancer-functionaliteit op bare-metal en hoe worden externe IP's geadverteerd naar het onderliggende ODC-netwerk?

Twee concrete aspecten:

1. **IP Address Management (IPAM)**: wie kiest welk IP-adres wordt toegewezen aan een LoadBalancer-Service, uit welke pool, met welke regels?
2. **Advertentie**: hoe weet de rest van het netwerk dat dat IP bij dit cluster hoort? Opties zijn BGP-peering met de ToR-switches (ADR-G56) of L2-announcements via ARP/NDP.

Eisen die de keuze inperken:

- Integratie met de CNI-keuze (ADR-C07: Cilium): de loadbalancer moet naadloos samenwerken met de dataplane.
- Minimale stack-complexiteit: geen aparte daemon als dezelfde functionaliteit al door een aanwezige component kan worden geleverd.
- BGP-ondersteuning is verplicht: L2-announcements schalen slecht en zijn niet compatibel met gerouteerde fabric-topologieën (ADR-G56).
- Per-tenant IP-pools moeten mogelijk zijn (ADR-C43).

Besluit

Het platform gebruikt **Cilium LB IPAM gecombineerd met Cilium BGP Control Plane** voor LoadBalancer-functionaliteit op bare-metal. Geen aparte loadbalancer-component (zoals MetalLB) wordt geïnstalleerd.

Onderzochte implementaties

Cilium LB IPAM + BGP Control Plane (Apache 2.0, CNCF): Cilium levert sinds v1.13 native LoadBalancer-IPAM (`CiliumLoadBalancerIPPool`) en sinds v1.14 een volwaardige BGP Control Plane (`CiliumBGPPeeringPolicy`, `CiliumBGPAdvertisement`). Services van type `LoadBalancer` krijgen automatisch een IP uit de pool en worden via BGP naar de ToR-switches geadverteerd. Beide functies zijn onderdeel van de Cilium-component die er al is voor de CNI (ADR-C07). Nadeel: relatief nieuw; MetalLB heeft meer productie-mileage.

MetalLB (CNCF-incubation): Standaard open source LoadBalancer voor bare-metal Kubernetes. Ondersteunt BGP-mode en L2-mode. Draait als aparte component naast de CNI. Bewezen en breed gebruikt. Nadeel: extra component in de stack, potentieel overlappende functionaliteit met Cilium BGP.

Kube-VIP: Combineert LoadBalancer en control plane HA. Minder geschikt voor een Hypershift-topologie (ADR-C20) waar control plane HA anders wordt opgelost.

External Load Balancer (F5, Citrix ADC, etc.): Hardware- of VM-gebaseerde loadbalancer extern aan het cluster. Vendor lock-in, grote investering, past niet bij de open source-strategie.

Technologiekeuze

Cilium LB IPAM + Cilium BGP Control Plane.

Motivering:

(1) Cilium is al aanwezig als CNI (ADR-C07);
(2) de BGP Control Plane integreert met de ToR-fabric (ADR-G56) via standaard BGP-peering;
(3) per-tenant IP-pools zijn native ondersteund (ADR-C43);
(4) één minder component betekent één minder lifecycle-afhankelijkheid en één minder punt waar upgrades kunnen fout gaan.

Motivatie

De Cilium-native benadering vermindert stack-complexiteit zonder functioneel verlies. MetalLB zou een redundante component zijn naast Cilium BGP. Cilium heeft in v1.14 en v1.15 de BGP-functionaliteit significant uitgebreid en is inmiddels productie-waardig voor deze use-case.

Door één component verantwoordelijk te maken voor CNI, service mesh en LoadBalancer wordt de debuggability verbeterd: netwerkproblemen zijn traceerbaar binnen één component met één observability-model (Hubble).

Consequenties

- **BGP-peering configuratie**: elk tenant-cluster peert met de ToR-switches van het onderliggende ODC. Dit moet worden uitgewerkt in ADR-G56 (datacenter network fabric) en onderdeel worden van de cluster-provisioning-pipeline (ADR-C01).
- **Per-tenant IP-pools**: granulaire IP-toewijzing per tenant wordt geïmplementeerd via `CiliumLoadBalancerIPPool`-resources met tenant-selectors (ADR-C43).
- **Geen MetalLB**: teams en documentatie die naar MetalLB verwijzen moeten worden bijgewerkt. Dit is een afwijking van wat veel Kubernetes-documentatie standaard aanneemt.

- **BGP-kennis in platformteam**: operationele ondersteuning vereist BGP-skills in het team (peering-configuratie, route-filtering, troubleshooting met `bgpctl`-equivalenten).
- **Version lockstep**: Cilium-versies moeten worden getest tegen de switch-firmware (Arista EOS, SONiC, etc.) om regressies in BGP-gedrag te voorkomen.

Sessienotities

Cilium BGP was in 2023 nog onvolwassen, sinds v1.14/1.15 productiewaardig. Keuze herbevestigd op basis van recente ervaring bij andere overheidsplatforms.

ADR-C10: Primaire beheerinterface en self-service model

Status	Zilver
Laag	Perifere diensten
Afhankelijk van	ADR-001, ADR-C04
Datum	2026-03-04

Context en probleemstelling

Het platform wordt gebruikt door twee groepen: het platformteam (dat de infrastructuur beheert) en tenants (die hun clusters en workloads beheren). Beide groepen hebben een interface nodig om met het platform te interacteren. De architectuurvraag is: wat is de primaire beheerinterface van het platform en hoe wordt self-service voor tenants aangeboden?

Dit is geen keuze voor een specifiek dashboard maar een fundamenteler besluit over het interactiemodel. De opties variëren van een API als primaire interface (waar elke UI een consument van is) tot een UI-gedreven model (waar de UI de primaire ervaring is en de API secundair).

De keuze beïnvloedt:

- **Automatiseerbaarheid**: als de API primair is, kan elke handeling worden geautomatiseerd via scripts, CI/CD en GitOps.
- **Integreerbaarheid**: andere systemen (monitoring, ticketing, CMDB) kunnen direct met de API koppelen.
- **Self-service diepgang**: tenants die vertrouwd zijn met Kubernetes kunnen via kubectl werken; tenants die dat niet zijn hebben een UI nodig.
- **Leveranciersonafhankelijkheid**: een UI is een mening over hoe je het platform beheert. Een API is een contract.

Besluit

De platform-API is de primaire en enige beheerinterface. Alle functionaliteit — van cluster aanmaken tot service deployen — is beschikbaar via de API. Een grafische interface is een consument van die API, geen apart systeem met eigen functionaliteit. Er mag geen operatie zijn die alleen via de UI kan worden uitgevoerd.

Dit betekent dat de API-first is in de letterlijke zin: de API wordt eerst ontworpen en geïmplementeerd, de UI wordt daarna gebouwd als visualisatie van de API-mogelijkheden.

Onderzochte implementaties

API-first met losse UI-keuze: De platform-API (Kubernetes API, uitgebreid met custom resources) is het contract. De UI wordt apart gekozen en kan worden vervangen zonder de API te raken. Opties voor de UI zijn Backstage (extensible developer portal), Headlamp (lichtgewicht Kubernetes dashboard) of een eigen frontend. Voordeel: maximale flexibiliteit, leveranciersonafhankelijkheid op UI-niveau. Nadeel: de UI moet apart worden gebouwd of geconfigureerd.

Geïntegreerd platform met eigen UI (Rancher, OpenShift Console): De UI en de API zijn onderdeel van hetzelfde product. De UI biedt functies die direct in het product zijn gebouwd. Voordeel: out-of-the-box ervaring. Nadeel: koppeling aan het product, de UI bepaalt mede het platform.

Alleen CLI/API, geen UI: Alle interactie via kubectl, talosctl en scripts. Voordeel: eenvoudigst, geen UI te onderhouden. Nadeel: hoge drempel voor minder technische gebruikers en tenant-operators.

Technologiekeuze

Gekozen: API-first. UI-keuze uitgesteld.

De Kubernetes API (uitgebreid met CRDs voor platformspecifieke resources) is het primaire contract. De keuze voor een specifieke UI-technologie is bewust uitgesteld — het is een presentatiekeuze die geen invloed heeft op de architectuur. De UI kan later worden gekozen op basis van de ervaring van het team en de behoeften van de tenants.

Motivatie

De keuze is gedreven door het automatiseringsprincipe. In een platform dat tientallen clusters beheert voor tientallen tenants is handmatig beheer via een UI niet schaalbaar. Elke operatie moet scriptbaar zijn, integreerbaar in CI/CD pipelines en opneembaar in GitOps-workflows. Dat kan alleen als de API het primaire contract is.

De keuze voor een specifieke dashboard-technologie is bewust niet gemaakt. Zoals in de sessie werd opgemerkt: dashboarding is meer persoonlijke voorkeur dan architectuurkeuze. Wat voor de ene operator een goede interface is, is voor de andere een beperking. Door de UI los te koppelen van de API kan elke tenant desgewenst een eigen dashboardkeuze maken.

Consequenties

Alle platformfunctionaliteit moet eerst als API worden ontworpen voordat er een UI voor wordt gebouwd. Dit vereist discipline in het ontwikkelproces: de API is het product, de UI is de versiering.

Tenants die niet technisch zijn hebben op termijn wel een UI nodig voor basistaken (cluster status bekijken, resources monitoren). De keuze hiervoor moet op enig moment worden gemaakt, maar het is een presentatiekeuze die los staat van de platformarchitectuur.

Open vragen

1. **Platform API abstractielaag**: de analyse tijdens de poc noemt een Platform API als abstractielaag over Gardener. Moet het NDS-platform een eigen API-abstractie bieden bovenop de Kubernetes API, of is de Kubernetes API (met CRDs) voldoende?
2. **Self-service portal voor niet-technische tenants**: sommige overheidsorganisaties hebben geen Kubernetes-expertise. Hoe worden zij bediend? Een vereenvoudigde portal die de complexiteit verbergt? Of training als voorwaarde voor platformgebruik?

Sessienotities

Keuze voor dashboard-technologie is niet interessant. Dashboarding is meer persoonlijke voorkeur. Integratie met andere clouddiensten zou voor een andere keuze kunnen zorgen.

ADR-C11: GPU-management op bare-metal Kubernetes

Status	Zilver
Laag	Compute
Datum	2026-06-19

Context en probleemstelling

GPU-nodes vereisen gespecialiseerde drivers, device plugins en monitoring. Operatorgebaseerd, handmatig of componentgebaseerd?

Besluit

Dit laten we buiten het design, kan in v2 of later worden opgelost.

Motivatie

Do the simple thing first. Onze afnemers willen soeverein alternatief voor public cloud.

Randvoorwaarden voor GPU's in de DC's maakt kostenplaatje significant anders, en er ontstaan uitdagingen rond alle datacenterdiensten (cooling, power, network speed, etc)

Consequenties

[Nog uit te werken]

ADR-C12: Declaratief lifecycle management voor platformdiensten

Status	Zilver
Laag	Managed diensten
Afhankelijk van	ADR-001
Maakt mogelijk	ADR-C55
Datum	2026-03-04

Context en probleemstelling

Het platform biedt managed diensten aan (databases, message queues, monitoring, certificaatbeheer) die als Kubernetes-workloads worden uitgerold op de tenant-clusters en op het management-cluster. Deze diensten hebben elk een eigen versiecyclus, eigen configuratieparameters en eigen afhankelijkheden. Het platform moet tientallen diensten beheren over tientallen clusters, elk met eigen versies en configuraties.

De architectuurvraag is: hoe worden platformdiensten uitgerold, geconfigureerd, geüpdate en verwijderd over meerdere clusters heen, zodanig dat de gewenste toestand altijd in een versiebeheerde, auditeerbare bron is vastgelegd en dat afwijkingen automatisch worden hersteld?

Dit is een fundamentele operationele keuze. Zonder een gestandaardiseerd lifecycle-mechanisme ontstaat configuratie-drift: cluster A draait versie 3.2 van een dienst, cluster B draait 3.1, en niemand weet waarom ze verschillen. Bij NIS2 en BIO is aantoonbaarheid van de gewenste toestand een harde eis.

De eisen:

- **Declaratief**: de gewenste toestand van elke dienst op elk cluster moet beschreven zijn in code, niet in een handmatig uitgevoerd script.
- **Versiebeheerd**: elke wijziging moet traceerbaar zijn — wie wijzigde wat, wanneer, en waarom.
- **Auditeerbaar**: de audittrail moet bewijzen dat de daadwerkelijke toestand overeenkomt met de gewenste toestand.
- **Multi-cluster**: dezelfde dienst moet over tientallen clusters kunnen worden uitgerold met cluster-specifieke configuratievariaties.
- **Drift-detectie**: als de daadwerkelijke toestand afwijkt van de gewenste toestand (door handmatige ingrepen, fouten, of aanvallen), moet dit worden gedetecteerd en hersteld.

Besluit

Het platform hanteert het GitOps-principe voor lifecycle management van alle platformdiensten. Git is de enige bron van waarheid. De gewenste toestand van elke dienst op elk cluster is beschreven in een Git-repository als declaratieve manifesten. Een reconciliatie-controller op elk cluster vergelijkt continu de gewenste toestand (in Git) met de daadwerkelijke toestand (in het cluster) en corrigeert afwijkingen automatisch.

Diensten worden verpakt als Helm charts — de de-facto standaard voor het verpakken van Kubernetes-applicaties. Helm biedt templating voor cluster-specifieke configuratie (waarden per omgeving), versiebeheer van dienstdefinities, en een groot ecosysteem van bestaande charts voor gangbare diensten.

De combinatie van Helm (verpakking) en GitOps (uitrol en reconciliatie) levert een volledig declaratieve, versiebeheerde en auditeerbare lifecycle pipeline.

Onderzochte implementaties

Helm + GitOps (ArgoCD of Flux): Helm charts als verpakkingsformaat, uitgerold en gereconcilieerd door een GitOps-controller. De Git-repository bevat per cluster een set Helm releases met values-bestanden. De GitOps-controller detecteert wijzigingen in Git en past ze toe op het cluster. Drift wordt automatisch hersteld. De audittrail is de Git-historie. ArgoCD biedt een UI voor visualisatie; Flux is lichtgewichter en puur CLI/API-gestuurd. De concrete keuze tussen ArgoCD en Flux wordt in ADR-C55 vastgelegd.

Crossplane: Kubernetes-native control plane dat externe en interne resources beheert via declaratieve Kubernetes-objecten (Compositions, Claims). Sterk in het abstraheren van infrastructure provisioning en het aanbieden van self-service platforms. Minder geschikt als primair lifecycle-mechanisme voor Helm-gebaseerde diensten — Crossplane is ontworpen om cloud-resources te beheren, niet om Helm charts uit te rollen. Kan complementair zijn aan Helm+GitOps maar niet als vervanging.

Operator Lifecycle Manager (OLM): Red Hat's mechanisme voor het beheren van Kubernetes-operators via een catalogus. Biedt operator-discovery, dependency resolution en automatische upgrades. Sterk gekoppeld aan het Red Hat/OpenShift-ecosysteem. Minder geschikt voor een platform dat losse CNCF-componenten combineert — OLM dwingt een specifiek operator-packaging-formaat af (bundles) dat niet universeel wordt gebruikt.

Handmatig beheer via kubectl apply: Manifesten worden handmatig uitgerold per cluster. Geen automatische reconciliatie, geen drift-detectie, geen gecentraliseerde audittrail. Op de schaal van dit platform onhoudbaar. Genoemd als anti-patroon, niet als serieuze optie.

Technologiekeuze

Gekozen: Helm + GitOps (ArgoCD of Flux)

Helm is de de-facto standaard voor Kubernetes-applicatieverpakking met het grootste ecosysteem van beschikbare charts. GitOps (via ArgoCD of Flux) levert de reconciliatie-loop die declaratief beheer, drift-detectie en audittrail combineert in één mechanisme. De combinatie is de meest geadopteerde aanpak in de Kubernetes-community voor platformbeheer op schaal.

De keuze tussen ArgoCD en Flux als concrete GitOps-tool wordt in ADR-C55 vastgesteld.

Motivatie

De keuze voor GitOps als lifecycle-principe sluit direct aan op de architectuurfilosofie van het platform. Het hele platform is opgebouwd rond het idee dat de gewenste toestand in code is beschreven: de hardware wordt declaratief beheerd, het OS wordt declaratief geconfigureerd via machineconfiguraties (ADR-C23/C24), de clusters worden declaratief beheerd via Cluster API (ADR-C04). Het zou inconsistent zijn om de platformdiensten vervolgens handmatig of imperatief te beheren.

De Git-audittrail is een directe invulling van de NIS2- en BIO-eis voor aantoonbaarheid. Elke wijziging aan een platformdienst is een commit met auteur, tijdstip en beschrijving. Bij een security-incident kan exact worden vastgesteld welke configuratie op welk moment actief was.

Crossplane is niet gekozen als primair mechanisme maar kan complementair worden ingezet — bijvoorbeeld voor het abstraheren van tenant-facing self-service APIs bovenop de Helm+GitOps-laag. Dit is een vervolgoverwering, geen onderdeel van dit besluit.

OLM is afgevallen vanwege de koppeling aan het Red Hat-ecosysteem, wat in strijd is met het principe van leveranciersonafhankelijkheid (ADR-001).

Consequenties

Alle platformdiensten moeten als Helm charts worden verpakt of als Helm charts beschikbaar zijn. Diensten die niet als Helm chart beschikbaar zijn vereisen het schrijven van een eigen chart.

De Git-repository-structuur voor multi-cluster deployment moet worden ontworpen. De gangbare patronen zijn: een monorepo met per-cluster directories, of een app-of-apps patroon met een centrale repository die per cluster verwijst naar dienst-repositories. Dit is een operationele keuze die bij de implementatie moet worden gemaakt.

Er ontstaan twee reconciliatie-loops die elkaar raken: de GitOps-controller reconcilieert de Helm releases, en de operators binnen die Helm releases reconciliëren de dienst-instanties (bijvoorbeeld: de PostgreSQL-operator reconcilieert de database-clusters). De afbakening tussen deze twee loops moet expliciet zijn — de GitOps-controller beheert de operator-deployment, de operator beheert de instanties.

Het platformteam moet Helm-chart-ontwikkeling en GitOps-pipeline-expertise opbouwen. Dit omvat: chart-templating, values-management per omgeving, secret-management in Git (sealed secrets of external secrets), en het debuggen van reconciliatie-problemen.

Open vragen

1. **Repository-structuur**: monorepo vs. multi-repo? Bij tientallen clusters en tientallen diensten groeit een monorepo snel. Maar multi-repo maakt het moeilijker om een overzicht te houden van welke versie waar draait. De keuze heeft operationele consequenties die in de implementatiefase moeten worden vastgelegd.
2. **Secret management in Git**: secrets mogen niet in plaintext in Git staan. Opties zijn Sealed Secrets (versleuteld in Git, ontsleuteld in het cluster), External Secrets Operator (secrets uit een externe KMS, referentie in Git), of SOPS (versleutelde YAML-bestanden in Git). Dit raakt aan ADR-G53 (secrets- en sleutelbeheer).
3. **Twee reconciliatie-loops**: de grens tussen “GitOps beheert de operator” en “de operator beheert de instantie” moet scherp zijn. Als de GitOps-controller een operator-upgrade uitrolt, triggert dat een cascade van instantie-reconciliaties. De impact-radius van een GitOps-commit kan daardoor groter zijn dan verwacht. Er zijn afspraken nodig over change windows en canary-deployments.
4. **ArgoCD vs. Flux (ADR-C55)**: dit besluit is bewust uitgesteld maar is operationeel urgent. ArgoCD biedt een UI die waardevol is voor overzicht over tientallen clusters. Flux is lichtgewichter en beter geschikt voor volledig geautomatiseerde pipelines zonder menselijke interventie. De keuze hangt af van hoeveel menselijke interactie gewenst is in het uitrolproces.

Sessienotities

Dit is een standaardoplossing met grote adoptie. De Git CI/CD audittrail is erg goed.

ADR-C20: Control plane topologie voor multi-tenant clustermodel

Status	**Vastgesteld**
Laag	Provisioning
Afhankelijk van	ADR-C04, ADR-C60
Maakt mogelijk	ADR-G03
Datum	2026-08-07

Context en probleemstelling

ADR-C04 stelt dat elke tenant een eigen Kubernetes-cluster krijgt met een eigen control plane. De vervolgvraag is: waar draaien die control planes, en hoe worden ze beheerd op de schaal van tientallen tot honderden tenants?

De traditionele benadering — dedicated bare-metal nodes per control plane — is op deze schaal onbetaalbaar en operationeel onbeheersbaar. Bij 50 tenants met elk drie control plane-nodes zou dat 150 servers zijn die uitsluitend beheerwerk doen zonder tenant-workloads te draaien. Elke control plane-upgrade is dan een bare-metal operatie.

De architectuurvraag kent twee niveaus die bewust worden gescheiden:

Niveau 1 — Architectuurprincipe: hoe worden control planes gehost? Dit is vastgesteld (zie Besluit).

Niveau 2 — Framework: welk framework implementeert dat principe? Hier is een aanbeveling gedaan op basis van internet overleggen.

De eisen die het gekozen model moet beantwoorden:

- Minimale resource-overhead per control plane — capaciteit gaat naar tenant-workloads, niet naar beheercomponenten.
- Onafhankelijke upgrades per tenant — niet alle tenants tegelijk.
- Schaalbaarheid naar honderden clusters zonder management-bottleneck.
- Uitval van een management-cluster mag draaiende tenant-workloads niet onderbreken.
- Day-2-operaties ingebouwd of aantoonbaar invulbaar: upgrades, credential rotation, driftdetectie, autoscaling, hibernation.
- Conformiteit met SEAL-4-soevereiniteitseisen (ADR-002): de governance van de gekozen tooling is een expliciete afweging.

Besluit

Principe — vastgesteld: de tenant control planes draaien als containers op een centraal management-cluster, niet als aparte VM's of bare-metal nodes. Dit model heet **Hosted Control Planes (HCP)**. De API server, etcd, controller-manager en scheduler van elke tenant draaien als Kubernetes-pods in een geïsoleerde namespace op een management-cluster. De worker-nodes draaien op bare-metal en verbinden via een beveiligd koppelvlak met de gehoste control plane.

De tenant heeft volledige cluster-admin toegang tot het eigen cluster maar heeft geen zichtbaarheid op de control plane-pods zelf. Dit scheidt platformverantwoordelijkheid (control plane) van tenantverantwoordelijkheid (workloads). Bij uitval van een management-cluster blijven bestaande workloads draaien; alleen configuratiewijzigingen en nieuwe scheduling zijn tijdelijk niet mogelijk.

De management-clusters zijn hiermee het meest kritieke component van de Clouddienst en vereist de hoogste beschikbaarheidsinrichting: gespreid over alle availability zones, met gedegen etcd-backup en een expliciet gedocumenteerde blast radius (zie Consequenties).

Inclusief genomen solution besluit voor Gardener.

Onderzochte implementaties

Hosted Control Planes als principe (vastgesteld) Control planes als containers op een management-cluster. Scheidt platform- en tenantverantwoordelijkheden. Schaalt efficiënt naar honderden clusters. Wordt geïmplementeerd door zowel Gardener als Hypershift.

Dedicated control plane-nodes per tenant (afgevallen) Elke tenant krijgt drie bare-metal nodes exclusief voor de control plane. Sterkste compute-isolatie maar onacceptabele kosten en operationele complexiteit op schaal. Bij 50 tenants: 150 servers die geen tenant-workloads draaien. Elke upgrade is een bare-metal operatie. Afgevallen op kosten en beheerbaarheid.

Kamaji (afgevallen) Lichtgewicht HCP-implementatie als CNCF sandbox project. Technisch interessant maar onvoldoende bewezen op de beoogde schaal. Kleinere community. Afgevallen op volwassenheid.

Technologiekeuze

Twee frameworks zijn conformant aan het HCP-principe. De aanbeveling is Gardener; Hypershift blijft een volwaardige optie ter vergelijking.

Gardener — aanbevolen (NeoNephos Foundation / Linux Foundation Europe)

Gardener implementeert HCP via een geïntegreerd framework met drie clustertypes: Garden (centraal beheer), Seed (host voor control planes) en Shoot (tenant-clusters). Gardener levert Day-2-operaties standaard ingebouwd: geautomatiseerde upgrades, credential rotation, autoscaling, hibernation en self-healing.

Voordelen:

- Day-2-operaties ingebouwd — geen aanvullende bouw vereist.
- Natieve integratie met metal-stack (ADR-C60): provisioning en cluster-lifecycle in één bewezen keten.
- Governance: valt onder de NeoNephos Foundation, onderdeel van Linux Foundation Europe (NextGenerationEU-gefinancierd). Gunstiger voor SEAL-4 dan de VS-georiënteerde governance van Hypershift.
- Basis van de Sovereign Cloud Stack (SCS) referentie-implementatie — directe aansluiting op Europese soevereiniteitsinitiatieven.

Aandachtspunten:

- Gardener is meer opinionated dan Hypershift. Het brengt eigen abstracties mee (MachineDeployments, Gardener machine-controller-manager) die overlappen met standalone Cluster API en daardoor de relatie met ADR-C01 raken — zie Open vragen.
- Grotere initiële leercurve en invasiever framework.

Hypershift — ter vergelijking (Red Hat / CNCF)

Hypershift implementeert HCP als modulair mechanisme dat native integreert met Cluster API en Metal[3]. Componenten zijn vrij te kiezen; Hypershift dicteert geen andere lagen van de stack.

Voordelen:

- Modulair — laat keuze van CNI, storage en GitOps volledig vrij.
- Actieve community, bewezen in productie bij grote managed OpenShift-deployments.
- Directe integratie met standalone Cluster API (ADR-C01).

Aandachtspunten:

- Day-2-operaties zijn niet ingebouwd en moeten volledig apart worden gebouwd — functionaliteit die Gardener standaard levert.
- Herkomst in de Red Hat / OpenShift-wereld creëert spanning met het Model C-principe van losse componenten (ADR-001) en de SEAL-4-governance-eis (ADR-002).

Vergelijking op beslissingscriteria

Criterium	Gardener	Hypershift
HCP-principe	Ja	Ja
Day-2 ingebouwd	Ja — standaard	Nee — apart te bouwen
SEAL-4 governance	Gunstiger (NeoNephos, EU)	Risico (Red Hat/IBM, VS)
Vrijheid losse componenten	Lager (eigen framework)	Hoog
Integratie metal-stack	Natief (ADR-C60)	Via eigen CAPI/Metal[3]-pad
Sovereign Cloud Stack	Ja	Nee
Volwassenheid	Bewezen op schaal	Bewezen op schaal

Motivatie

Het HCP-principe is ondubbelzinnig: de kosten en operationele complexiteit van dedicated control plane-nodes zijn op schaal onacceptabel. Het HCP-model reduceert de overhead per tenant van drie bare-metal nodes naar enkele GB geheugen en CPU-shares op een gedeeld management-cluster.

De aanbeveling voor Gardener volgt uit twee samenhangende argumenten. Ten eerste levert Gardener Day-2-operaties ingebouwd; bij Hypershift moet dit volledig worden opgebouwd. Ten tweede past de EU-gewortelde governance van Gardener beter bij de SEAL-4-ambitie dan de Red Hat/IBM-lijn van Hypershift.

Consequenties

Management-cluster als kritiek component: Er moet besloten worden hoeveel resources er onder een control plane mogen vallen. Op basis van hoe groot de RTO wordt door het bijhouden van alle states, moet besloten worden wanneer er een nieuw managmentcluster nodig is met een nieuwe controlplane. Ook voor de verdeling van klant organisaties.

Blast radius bij management-cluster-uitval: een upgrade of storing raakt potentieel alle tenant-clusters die vallen binnen de betrokken controlplane en managementcluster. De upgrade-procedure is gelaagd eerst het management-cluster, dan tenant control planes in golven. De blast radius wordt expliciet beschreven in het Day-2-operationeel model.

etcd-schaling: het management-cluster etcd bevat de state van alle bij die management cluster aangesloten tenant control planes. Bij honderden tenants groeit de database significant. Performance-monitoring, compaction-strategie en capaciteitsplanning zijn vereist.

Isolatie bij hogere classificaties: voor workloads met verhoogde isolatievereisten blijft de optie van dedicated control plane-nodes of een afgeschermd management-cluster per classificatieniveau beschikbaar als uitzondering op het standaard HCP-model.

ADR-C23: Beheermodel voor immutable nodes

Status	Goud
Laag	Provisioning
Afhankelijk van	ADR-C02, ADR-C12
Maakt mogelijk	ADR-C24
Datum	2026-07-07

Context en probleemstelling

ADR-C02 kiest Flatcar Container Linux als primair host-OS. Flatcar is een immutable besturingssysteem: het root-filesystem is read-only en wordt als geheel vervangen bij een update. SSH is aanwezig in Flatcar maar wordt via de provisioningconfiguratie uitgeschakeld op productienodes. Configuratie van de node-laag verloopt via **Ignition-configs** — declaratieve JSON-documenten die bij eerste boot worden ingelezen en niet meer worden gewijzigd.

De architectuurvraag is: hoe worden Flatcar-nodes geconfigureerd, geüpdated en gediagnosticeerd als SSH op productienodes is uitgeschakeld en het filesystem read-only is?

Dit raakt de dagelijkse operatie: hoe voegt het team een node toe, hoe wordt een configuratiewijziging op honderden nodes doorgevoerd, hoe wordt een probleemnode gedebugd, en hoe rolt het team security-patches uit zonder handmatige ingreep per node?

Aanvullende eis: het beheermodel moet volledig zonder externe SaaS-afhankelijkheden kunnen draaien. Het platformteam behoudt volledige controle over provisioning en configuratie (SEAL-4).

Besluit

Node-configuratie wordt volledig GitOps-gestuurd via **Butane/Ignition-configs**. Elke node-rol (control plane, worker, storage) heeft een Butane-template in Git. Butane wordt gecompileerd naar Ignition-JSON en meegegeven bij node-provisioning. Configuratiewijzigingen verlopen altijd via Git — nooit via directe node-toegang.

Nodes worden behandeld als vervangbaar (cattle, not pets). Een probleemnode wordt niet gerepareerd maar vervangen: nieuw Ignition-config, nieuwe provisioningcyclus, schone node. De oude node wordt gedraineerd en gedeprovisioned.

Onderzochte implementaties

GitOps met Butane/Ignition-configs: Flatcar-nodes nemen bij eerste boot een Ignition-config op die SSH-toegang uitschakelt, systemd-units configureert en disk-layout vastlegt. Butane-templates per node-rol worden in Git beheerd en gecompileerd via de CI/CD-pipeline (ADR-C12). Voordeel: één bron van waarheid, volledig auditeerbaar, geen externe afhankelijkheid, aansluitend op bestaande GitOps-tooling (ADR-C55). Nadeel: elke configuratiewijziging vereist herprovisioning van de betreffende nodes.

Configuration management (Ansible, Puppet, Chef): Traditionele tooling voor node-configuratie via SSH. Vereist SSH-toegang, leidt tot configuratiedrift en schaalt slecht op duizenden nodes. Staat direct haaks op het immutable principe uit ADR-C02. Afgevallen.

Extern node-beheerplatform (SaaS): Gecentraliseerde beheerplatforms voor immutable OS-en (vergelijkbaar met Talos Omni). Voordeel: rijke beheerervaring. Nadeel: externe SaaS-afhankelijkheid, in strijd met SEAL-4-soevereiniteitseis. Afgevallen.

Technologiekeuze

Butane → Ignition als configuratieformaat. Butane-YAML is de mensleesbare bron in Git; gecompileerd naar Ignition-JSON voor gebruik door Flatcar bij boot. Templates worden per node-rol onderhouden: control-plane-template, worker-template, storage-template, GPU-worker-template.

SSH is uitgeschakeld in alle productie-Ignition-configs. Operationeel beheer en debugging verlopen via `journalctl` (via kubectl node-debug), de observability-stack (ADR-G41) en de platform-API. Noodtoegang via BMC/IPMI is voorzien als breakglass-procedure (zie Consequenties).

Upgrades verlopen als A/B image-swap via het Omaha-protocol. De rollback- en update-procedure wordt uitgewerkt in ADR-C24.

Motivatie

De keuze sluit op drie punten aan op bestaande besluiten.

Ten eerste: het GitOps-principe (ADR-C12) — alle platformconfiguratie heeft Git als bron van waarheid; node-configs zijn daarop geen uitzondering. Het vermijdt een tweede beheersysteem naast de bestaande GitOps-tooling.

Ten tweede: SEAL-4-soevereiniteit — een externe beheer-SaaS is niet acceptabel. Het platformteam deployt en beheert de volledige infrastructuur zelf, zonder afhankelijkheid van een externe partij voor node-lifecycle-operaties.

Ten derde: aansluiting op Flatcar's eigen mechanisme — Ignition is het ontworpen configuratiekanaal van Flatcar. Gebruik ervan is geen workaround maar de beoogde werkwijze; de tooling is stabiel en breed getest.

Configuration management via SSH wordt afgewezen omdat het de kernbelofte van een immutable OS ondermijnt: als configuratiedrift via een beheeragent kan worden geïntroduceerd, verlies je de auditeerbaarheid die het immutable model biedt.

Consequenties

SSH is uitgeschakeld op alle productienodes. Operationele toegang verloopt via de Kubernetes-control-plane (kubectl exec, node-debug-pods) en de observability-stack (ADR-G41). Directe node-toegang buiten Kubernetes is geen standaardprocedure.

Butane-templates worden per node-rol onderhouden in Git. Elke wijziging aan een template vereist herprovisioning van de betreffende nodes — nodes worden vervangen, niet gepatcht. De update-volgorde en capaciteitsbeheer tijdens vervanging worden uitgewerkt in ADR-C24.

Configuratiedrift op node-niveau is structureel uitgesloten door het read-only filesystem. Drift op Kubernetes-configuratieniveau (ConfigMaps, RBAC, workloads) valt onder de GitOps-controle van ADR-C12 en ADR-C55.

Een **breakglass-procedure** voor noodtoegang via BMC/IPMI is noodzakelijk voor situaties waar een node niet via de platform-API bereikbaar is. De autorisatie, logging en vier-ogen-vereisten voor deze procedure worden vastgelegd vóór vaststelling van deze ADR.

Sessienotities

> [04-03-2026] GitOps-aanpak gekozen; extern SaaS-beheerplatform afgewezen wegens SEAL-4-concerns. [07-07-2026] ADR volledig herschreven voor Flatcar Container Linux als primair OS (ADR-C02). Talos-specifieke tooling (talosctl, gRPC API, machineconfiguraties) vervangen door Butane/Ignition. Breakglass-procedure benoemd als openstaand punt; wordt uitgewerkt vóór vaststelling.

ADR-C24: Update-strategie voor platformnodes

Status	Goud
Laag	Provisioning
Afhankelijk van	ADR-C02, ADR-C23
Maakt mogelijk	ADR-C55
Datum	2026-07-07

Context en probleemstelling

ADR-C02 kiest Flatcar Container Linux als primair host-OS. Flatcar ondersteunt technisch in-place updates via een A/B-partitieschema (Omaha-protocol). ADR-C23 stelt het beheermodel vast: nodes zijn wegwerpbaar, configuratie is volledig GitOps-gestuurd via Butane/Ignition.

De architectuurvraag is: hoe worden OS-updates en configuratiewijzigingen op productienodes doorgevoerd?

Besluit

Node-vervanging is de enige update-methode. Een node wordt niet in-place gepatcht maar vervangen: een nieuwe node met het bijgewerkte image wordt geprovisioned, de oude node wordt gedraineerd en verwijderd. In-place updates via het A/B-partitieschema worden niet gebruikt.

Onderzochte implementaties

Node-vervanging (rolling replacement): Bij een update wordt een nieuwe node aangemaakt op basis van het nieuwe Flatcar-image en de geldende Butane-config (ADR-C23). Het clusterbeheer-framework (ADR-C20) dient de bestaande werklast, draineert de oude node en verwijdert deze. Voordeel: elke actieve node is aantoonbaar in de gewenste staat; geen mogelijkheid voor partiële updates of configuratiedrift. Eén operationeel model voor alle nodes. Nadeel: vervanging duurt langer dan een in-place reboot.

In-place update via A/B-partitie (Omaha): Flatcar's ingebouwde mechanisme downloadt een nieuw image naar de inactieve partitie en schakelt bij reboot over. Voordeel: sneller dan vervanging. Nadeel: de node zelf blijft bestaan — eventuele drift buiten het OS wordt niet hersteld. Introduceert een tweede operationeel model naast node-vervanging. Afgevallen.

Motivatie

Node-vervanging garandeert dat elke actieve node identiek is aan de definitie in Git. Een in-place update laat de node intact — inclusief eventuele afwijkingen in de omliggende configuratie. Voor een platform dat aantoonbare auditeerbaarheid vereist (BIO2, EUCS SEAL-4) is de garantie van een schone node per update zwaarder dan de tijdsbesparing van een in-place reboot.

Eén update-model maakt operationele procedures, runbooks en training eenvoudiger. Twee modellen naast elkaar — ook al is het tweede formeel een fallback — verhogen de kans op operationele fouten.

Consequenties

Flatcar OS-images worden gebouwd in een eigen pipeline: base image + Kubernetes-componenten + platformconfiguratie → getest image → intern image-register. De pipeline integreert in het CI/CD-systeem (ADR-C12).

Updates worden uitgerold als rolling replacement: per node drain, vervang, valideer. Het clusterbeheerframework (ADR-C20) beheert de volgorde en bewaakt de clusterkapaciteit tijdens de uitrol.

Noodtoegang tot een node die niet via de platform-API bereikbaar is verloopt via BMC/IPMI (breakglassprocedure per ADR-C23) — niet via SSH of in-place OS-interventie.

Talos Linux (secundair alternatief per ADR-C02) volgt hetzelfde node-vervangingsprincipe via zijn eigen API-mechanisme; de update-aanpak is daarmee OS-onafhankelijk.

Sessienotities

> [04-03-2026] Node-vervanging gekozen als enige update-methode; cattle-not-pets principe leidend.
> [07-07-2026] ADR herschreven: Flatcar als primair OS (per ADR-C02), A/B-fallback expliciet afgevallen, SSH-breakglass vervangen door BMC/IPMI-verwijzing (consistent met ADR-C23), open vragen gesloten.

ADR-C38: Service mesh strategie: L7 verkeersbeheer, mTLS en observability per tenant

Status	Goud
Laag	Networking
Afhankelijk van	ADR-C07
Maakt mogelijk	ADR-G09
Datum	2026-08-07

Context en probleemstelling

ADR-C07 heeft de dataplane-strategie vastgelegd: eBPF-native netwerkverwerking op kernel-niveau voor L3/L4 verkeer. Maar de verwachte workloads vereisen ook L7-functionaliteit: verkeersroutering op basis van HTTP-headers, mutual TLS (mTLS) tussen services voor zero-trust-networking, circuit breaking, retries en fine-grained traffic splitting voor canary deployments.

De architectuurvraag is: hoe wordt L7 verkeersbeheer, versleuteling en observability aangeboden aan tenant-workloads, zodanig dat tenants die dit nodig hebben het kunnen gebruiken zonder dat het wordt opgedrongen aan tenants die het niet nodig hebben?

Dit is geen simpele technologiekeuze maar een fundamentele afweging tussen platformbrede uniformiteit en tenant-autonomie. Een verplichte service mesh voor alle workloads verhoogt de complexiteit en resource-overhead voor elke tenant — ook degenen die alleen eenvoudige HTTP-services draaien. Een optionele service mesh geeft tenants keuzevrijheid maar verhoogt de operationele complexiteit van de Clouddienst (twee paden ondersteunen).

De eisen:

- **Zero-trust networking**: diensten mogen niet op netwerkniveau worden vertrouwd. mTLS tussen services is de standaard voor DEP VERTROUWELIJK workloads (ADR-G38).
- **L7 observability**: de Clouddienst moet inzicht bieden in HTTP-niveau metrics (latency per endpoint, error rates, request volumes) bovenop de L3/L4 observability uit ADR-C07.
- **Haven/Haven+ compatibiliteit**: de gekozen aanpak moet passen binnen het Haven/Haven+ profiel dat de domeinarchitectuur als standaard stelt. NB, Haven+ is op dit moment out of scope.
- **Geen verplichte overhead**: tenants met eenvoudige workloads mogen niet worden belast met sidecar-proxy overhead als ze die functionaliteit niet nodig hebben.
- **Tenant-autonomie**: tenants moeten zelf kunnen kiezen of en hoe ze L7-functionaliteit inzetten binnen hun cluster.

Besluit

De Clouddienst biedt L7 verkeersbeheer als **optionele capability per tenant-cluster**, niet als platformwijde verplichting. De basisnetwerklaag (L3/L4) wordt geleverd door de CNI-keuze uit ADR-C07 en is altijd actief. L7-functionaliteit — mTLS, traffic management, L7 observability — wordt als opt-in dienst per cluster aangeboden aan tenants die dit nodig hebben.

De L3/L4 laag (CNI) en de L7 laag (service mesh) zijn bewust gescheiden verantwoordelijkheden. De CNI handelt routing, filtering en basaal loadbalancing af. De service mesh handelt applicatie-niveau verkeersbeheer, versleuteling en observability af. Tenants die L7-functionaliteit nodig hebben, kiezen zelf welke service mesh zij inzetten binnen hun cluster.

Onderzochte implementaties

Cilium service mesh (native): Cilium biedt sinds versie 1.12 een eigen service mesh die sidecar-proxy's elimineert door L7-verwerking in eBPF te doen. Voordeel: geen sidecar-overhead, diepere integratie met de CNI. Nadeel: de L7-functionaliteit is minder volwassen dan die van dedicated service meshes. Sommige L7-features (traffic splitting, fault injection) zijn beperkter. De community is primair gericht op L3/L4; de L7-capaciteiten ontwikkelen zich maar zijn nog niet feature-compleet.

Istio: De meest volwassen en breed geadopteerde service mesh. Volledige L7-functionaliteit: mTLS, traffic management, fault injection, circuit breaking, canary deployments. Sidecar-model (Envoy-proxy per pod) met optioneel ambient mode (per-node proxy, experimenteel). Brede community, uitgebreide documentatie. In Haven+ profiel als werkende configuratie gevalideerd. Nadeel: sidecar-overhead per pod (geheugen en CPU), complexere troubleshooting, en een eigen control plane (istiod) die resources vraagt.

Linkerd: Lichtgewichter dan Istio, geschreven in Rust (data plane) en Go (control plane). Eenvoudiger operationeel model. Minder features dan Istio maar dekt de basiscases (mTLS, L7 metrics, retries). Nadeel: kleinere community dan Istio, minder integratie-ervaring in het Haven+ profiel, beperktere traffic management features.

Geen service mesh: L7-functionaliteit wordt per applicatie opgelost (application-level TLS, applicatie-eigen metrics). De Clouddienst levert alleen L3/L4. Eenvoudigst voor de Clouddienst maar verschuift de complexiteit naar elke individuele tenant.

Technologiekeuze

Cilium voor L3/L4 (platform-breed, altijd actief). Tenants kiezen zelf hun L7-oplossing als opt-in.

Cilium levert de basisnetwerklaag voor alle clusters. Voor L7-functionaliteit kiezen tenants zelf welke service mesh zij inzetten — de Clouddienst doet geen platformwijde afdwinging van een specifieke L7-oplossing. De Clouddienst biedt Istio als aanbevolen en gevalideerde optie voor tenants die een kant-en-klare keuze willen (conform Haven+ profiel), maar staat tenants vrij andere service meshes te gebruiken.

Istio is de aanbevolen optie omdat dit een volwassen service mash is, daarnaast ook onderdeel van Haven+. Tenants zonder L7-behoefte krijgen geen overhead.

Motivatie

De keuze voor een gescheiden L3/L4 en L7 laag is een bewuste architectuurkeuze. De verleiding is om alles in één component te stoppen (Cilium doet L3/L4/L7 allemaal). Maar dat maakt de Clouddienst afhankelijk van de L7-volwassenheid van één component. Door de lagen te scheiden kan Cilium's L7 in de toekomst de aanbevolen optie worden als het volwassen genoeg is, zonder dat de L3/L4 laag wordt geraakt. Omgekeerd kan de aanbevolen L7-keuze wijzigen zonder de CNI te raken.

Het opt-in model is gedreven door het proportionaliteitsbeginsel: de overhead van een service mesh (sidecar-geheugen per pod, extra latency per hop) is alleen gerechtvaardigd voor workloads die L7-functionaliteit daadwerkelijk nodig hebben. Een interne informatie-website heeft geen mTLS nodig; een dienst die DEP VERTROUWELIJK data verwerkt wel.

Consequenties

De Clouddienst ondersteunt twee operationele paden: clusters met en clusters zonder service mesh. Dit verhoogt de complexiteit van troubleshooting, monitoring en documentatie. Runbooks moeten beide scenario's dekken.

Als Istio wordt ingezet als opt-in add-on per cluster vereist dit een gestandaardiseerde deployment — niet elke tenant mag een eigen Istio-versie draaien. De Clouddienst biedt één ondersteunde Istio-versie aan die wordt beheerd als onderdeel van de managed diensten catalogus (ADR-G49) met het bijbehorende upgradebeleid (ADR-G52).

De interactie tussen Cilium en Istio moet zorgvuldig worden geconfigureerd. Cilium moet de traffic van Istio's sidecar-proxy's correct routeren. Dit is een bekende en gedocumenteerde configuratie maar vereist specifieke Cilium-instellingen die in de cluster-templates moeten worden opgenomen.

mTLS via Istio vereist een PKI-architectuur voor het uitgeven en roteren van certificaten. Dit raakt aan de PKI-ADR (ADR-G54). Istio brengt een eigen CA mee (istiod) maar in een overheidsomgeving kan het wenselijk zijn om Istio te koppelen aan een externe CA (cert-manager of een HSM-gebackede CA).

Sessienotities

> Cilium voor L3/L4 networking en observability. Optionele L7 service mesh per tenant — tenants kiezen zelf hun L7. Istio is de meest volwassen en in Haven+ gevalideerde optie als referentie-aanbeveling. De complexiteit van twee systemen is beheersbaar als de service mesh als optionele plugin per cluster wordt aangeboden, niet als platformwijde verplichting.

ADR-C43: Granulaire IP-toewijzing en advertentie per tenant

Status	Goud
Laag	Networking
Afhankelijk van	ADR-C08, ADR-C04
Datum	2026-04-20

Context en probleemstelling

ADR-C08 stelt vast dat het platform Cilium LB IPAM met BGP gebruikt voor service-exposure. De vervolgvraag is: hoe worden IP-adressen toegewezen en geadverteerd op tenantniveau? Bij tientallen tenants met elk eigen services moet het platform per tenant kunnen bepalen welke IP-adressen beschikbaar zijn, welke adressen naar welke BGP-peers worden geadverteerd, en hoe wordt voorkomen dat de ene tenant de IP-ruimte van een andere tenant kan claimen.

De architectuurvraag is: hoe wordt de IP-adresruimte gestructureerd en beheerd per tenant, zodanig dat isolatie gegarandeerd is, het operationele beheer schaalbaar blijft, en de integratie met het ODC-BGP-fabric geautomatiseerd verloopt?

Besluit

IP-adrespools worden per tenant gedefinieerd als Kubernetes-resources (CiliumLoadBalancerIPPool). Elke tenant krijgt een of meer eigen pools met een vooraf toegewezen CIDR-range. De BGP-advertentie is beleidsgedreven: per pool wordt bepaald naar welke BGP-peers de adressen worden geadverteerd. Dit maakt het mogelijk om de IP-ruimte van tenant A naar andere peers te adverteren dan die van tenant B, of om bepaalde services alleen intern zichtbaar te maken.

Onderzochte implementaties

Cilium LB IPAM met service-gebaseerde pool-selectie: Elke tenant krijgt een CiliumLoadBalancerIPPool-resource in het eigen cluster. De pool bevat een CIDR-range die exclusief aan die tenant is toegewezen. Services selecteren een pool via labels of annotations. BGP-advertentie wordt per pool geconfigureerd via CiliumBGPPeeringPolicy. Voordeel: native Cilium-functionaliteit, geen extra component. Granulaire controle over wat waar wordt geadverteerd. Nadeel: de pool-administratie (welke tenant welke range) moet worden beheerd — bij tientallen tenants is dit een administratieve last.

MetalLB met address pools: MetalLB biedt vergelijkbare pool-functionaliteit maar als apart component naast de CNI. Voordeel: onafhankelijk van CNI-keuze. Nadeel: een extra component naast Cilium dat gedeeltelijk dezelfde functionaliteit biedt. Introduceert een integratiepunt.

Statische IP-toewijzing zonder pool-management: Elke service krijgt een handmatig toegewezen IP-adres. Voordeel: eenvoudig. Nadeel: schaalt niet, foutgevoelig, geen automatische failover.

Technologiekeuze

Gekozen: Cilium LB IPAM met service-gebaseerde pool-selectie en beleidsgedreven advertentie

Voortvloeiend uit ADR-C07 (Cilium) en ADR-C08 (Cilium LB IPAM + BGP). Geen extra component nodig.

Motivatie

De keuze houdt de netwerkstack compact: Cilium handelt routing, filtering, loadbalancing, IPAM en BGP-advertentie af in één component. Het toevoegen van MetalLB zou een functionele overlap creëren die de troubleshooting complexer maakt.

De granulariteit van pool-selectie maakt het mogelijk om per tenant of zelfs per dienst te bepalen welke IP-range wordt gebruikt en naar wie die wordt geadverteerd. Dit is essentieel voor multi-tenancy: een tenant die alleen interne diensten aanbiedt kan een niet-geadverteerde pool krijgen, terwijl een tenant met publieke diensten een pool krijgt die naar de internet-edge wordt geadverteerd.

Consequenties

Er moet een IP-adresplan worden opgesteld dat per tenant een CIDR-range reserveert. Dit plan moet ruimte laten voor groei zonder fragmentatie. Het raakt aan de bredere IP-adresseringsstrategie (ADR-G05).

BFD (Bidirectional Forwarding Detection) moet worden geconfigureerd voor snelle failover-detectie op de BGP-sessies. De standaard BGP-hold-timer van 90 seconden is te lang voor productie-workloads; BFD detecteert uitval in sub-seconden.

De BGP-peering-configuratie per ODC moet worden afgestemd met het netwerkteam dat de leaf-switches beheert. Elke node peert met de leaf-switches; de advertenties per tenant moeten consistent zijn over alle nodes.

Sessienotities

> Je kunt heel granulair te werk gaan wat je wel en niet adverteert. Je kunt een grote pool ophakken in meerdere specifieke stukjes.

ADR-C55: GitOps mate van menselijke interventie in een geautomatiseerd platform

Status	Brons
Laag	Managed diensten
Afhankelijk van	ADR-C12
Datum	2026-05-13

Context en probleemstelling

ADR-C12 legt vast dat platformdiensten declaratief worden beheerd via Helm en GitOps-principes. De vervolgstap is hoe de reconciliatie richting de clusters wordt ingericht.

De discussie gaat daarbij minder over tooling en meer over beheerfilosofie.

Hoe strikt willen we GitOps echt doorvoeren? Mag een operator nog handmatig ingrijpen? Hoe gaan we om met drift? En hoeveel centrale controle willen we houden over deployments en wijzigingen?

Daar zit ook de spanning:

- volledig geautomatiseerd reconciliëren vanuit Git
- tegenover traditionele change- en beheerinterventies

De oplossing moet:

- multi-cluster deployment ondersteunen
- declaratief blijven werken
- drift kunnen detecteren en herstellen
- schaalbaar blijven over meerdere clusters
- voldoende audittrail bieden voor BIO/NIS2

Besluit

Nog niet vastgesteld.

Wel is duidelijk dat:

- Git de primaire bron van waarheid wordt
- wijzigingen herleidbaar moeten zijn
- drift automatisch detecteerbaar moet zijn
- tooling ondersteunend is aan het gekozen beheermodel

De keuze gaat dus uiteindelijk vooral over:

- hoeveel handmatige interventie nog acceptabel is
- hoe streng GitOps wordt afgedwongen
- hoe operationeel beheer zich verhoudt tot declaratief beheer

Onderzochte implementaties

ArgoCD

GitOps-platform met centrale UI en uitgebreide multi-cluster ondersteuning.

Sterk in:

- operationeel overzicht
- visuele controle
- handmatige approvals en sync windows
- centrale sturing

Aandachtspunten:

- relatief zware control plane
- risico dat operators buiten Git om gaan werken
- grotere nadruk op handmatige operatie

Flux CD

GitOps-controller zonder centrale UI, volledig Kubernetes-native.

Sterk in:

- API-first beheer
- volledig Git-gebaseerde workflows
- lichte architectuur
- weinig extra abstrahering

Aandachtspunten:

- minder centraal overzicht
- troubleshooting meer CLI/API-georiënteerd
- hogere operationele kennisdrempel

Gecombineerde inzet

ArgoCD voor centraal overzicht en Flux voor workloadreconciliatie.

Voordeel:

- combinatie van zichtbaarheid en automatisering

Nadeel:

- twee modellen naast elkaar
- extra beheerlast
- meer complexiteit in upgrades en troubleshooting

Motivatie

Deze ADR gaat vooral over de manier waarop het platform beheerd wordt.

Niet alleen:

- “welke tool kiezen we?”

Maar vooral:

- “hoe strikt willen we declaratief beheer afdwingen?”

Een streng GitOps-model betekent:

- Git als enige source of truth
- minimale handmatige interventie
- automatische drift-correctie

Een meer traditioneel beheermodel laat meer ruimte voor:

- handmatige syncs
- approvals
- operatorinterventies

Beide richtingen hebben gevolgen voor:

- auditability
- beheerprocessen
- platformcomplexiteit
- incidentafhandeling

Consequenties

- Git wordt onderdeel van de formele beheerketen
- Handmatige wijzigingen kunnen automatisch worden teruggedraaid
- Secret management moet nog apart worden uitgewerkt
- Disaster recovery van de GitOps-laag moet worden meegenomen
- Beheerprocessen moeten aansluiten op declaratief werken

Open vragen

- Hoe strikt handhaven we Git als enige source of truth?
- Zijn handmatige syncs in productie toegestaan?
- Hoe past GitOps binnen bestaande change-processen?
- Werken we met één centrale control plane of meerdere?
- Hoe gaan we secrets beheren? ## Sessienotities

Volgens mij moeten we oppassen dat we hier niet primair een tooldiscussie voeren terwijl de operationele governance nog niet scherp is. De echte vraag is niet ArgoCD of Flux, maar: hoeveel menselijke interventie willen we nog in een GitOps-model toestaan? > Aanpassing naar aanleiding van bovensteaande notitie. De discussie verschuift hiermee van een toolkeuze naar de vraag welk beheermodel past bij een hoog geautomatiseerd platform.

ADR-C60: Hardware provisioning: geïntegreerde compute- en netwerk-provisioning

Status	Zilver
Laag	Provisioning
Afhankelijk van	ADR-001, ADR-G13, ADR-G65
Maakt mogelijk	ADR-C01, ADR-C20, ADR-C07, ADR-G72
Datum	2026-04-20

Context en probleemstelling

Het platform bestaat uit tienduizenden servers en honderden netwerk-switches in meerdere Overheids-DataCenters. Wanneer een server het datacenter binnenkomt en in het rack wordt geplaatst, moet deze automatisch worden omgezet naar bruikbare platformcapaciteit: gedetecteerd, geïnspecteerd, voorzien van een besturingssysteem, gekoppeld aan het juiste netwerksegment en opgenomen in het juiste tenant-cluster — zonder handmatige tussenkomst.

De kernspanning in deze ADR is: worden compute-provisioning en netwerk-provisioning als één geïntegreerde pipeline aangestuurd, of als gescheiden systemen met een koppelvlak ertussen?

Dit is een architectuurkeuze met directe gevolgen voor drie domeinen:

1. **Compute-provisioning**: detecteren, inspecteren en installeren van servers via BMC (IPMI/Redfish), firmware-configuratie, OS-installatie en opname als Kubernetes-node.
2. **Netwerk-provisioning**: configureren van switch-poorten, VLAN's, VRF's en BGP-peering zodat de server op het juiste netwerksegment terechtkomt met de juiste tenant-isolatie op switch-niveau.
3. **Cluster-integratie**: toewijzen van de geproviseerde node aan het juiste Kubernetes-cluster (samenwerking met ADR-C01 en ADR-C20).

De eisen die de keuze inperken:

- **Schaal**: bij tienduizenden servers en drie ODC-locaties is elke handmatige stap een bottleneck.
- **Atomaire consistentie**: als compute en netwerk apart worden beheerd, kan een mismatch ontstaan — server geprovisioned maar switch niet geconfigureerd, of andersom. Hoe meer handmatige overdracht tussen systemen, hoe groter het risico op inconsistenties.
- **Time-to-deploy**: de tijd tussen "server in rack" en "node beschikbaar in tenant-cluster" is een kritieke operationele metric.
- **Multi-tenancy op netwerkniveau**: tenant-isolatie (VRF, VLAN, BGP per tenant) moet worden meegenomen in de provisioning, niet achteraf worden geconfigureerd.
- **Declaratief beheer**: consistent met ADR-001 en het GitOps-principe (ADR-C12) moet hardware-configuratie declaratief zijn — gewenste toestand in code, afwijkingen automatisch gecorrigeerd.
- **Auditeerbaar**: elke wijziging aan hardware moet traceerbaar zijn (BIO, NIS2).

Besluit

metal-stack als geïntegreerde compute- én netwerk-provisioning-laag. metal-stack levert de volledige provisioning-pipeline van lege server tot geconfigureerde switch-poort tot werkende Kubernetes-node in één declaratief beheerde keten.

Onderzochte implementaties

metal-stack Geïntegreerde bare-metal provisioning van compute én netwerk, Kubernetes-native, multi-vendor serverondersteuning (Dell, HPE, Supermicro). Beheert spine-leaf BGP/EVPN-fabric en VRF-per-tenant als onderdeel van de provisioning-pipeline. In productie bij financiële instellingen in Duitsland sinds 2020

Vereist: SONiC-compatibele switches (Edgecore of equivalent). Dit is een harde hardware-eis die de switch-selectie in ADR-G13/G65 inperkt.

Voordeel: één geautomatiseerde keten van “server in rack” naar “node beschikbaar in cluster”, zonder handmatige overdracht tussen compute- en netwerk-teams. Directe integratie met Gardener (ADR-C20) voor cluster-lifecycle.

Nadeel: kleinere community dan CAPI+Metal³. Hardware-selectie is beperkter door SONiC-vereiste.

CAPI + Metal³ + Ironic (afgevallen) Cluster API als Kubernetes-native cluster lifecycle-framework, Metal³ als CAPI-provider voor bare-metal, Ironic als provisioning-engine. Compute-only: netwerk-provisioning is een volledig apart probleem dat niet door deze stack wordt opgelost. De netwerk-provisioning zou een apart, ongekoppeld project worden, met alle integratie- en consistentierisico’s van dien.

Voordeel: bredere CAPI-community, directe integratie met Hypershift (ADR-C20 optie B). Afgevallen omdat netwerk-provisioning niet is opgelost en Ironic-schaling een open risico is.

MAAS (Canonical, afgevallen) Volwassen bare-metal provisioning van Canonical. Compute-only, niet Kubernetes-native, commerciële support-afhankelijkheid voor enterprise-features. Afgevallen op dezelfde grond als CAPI/Metal³: netwerk-provisioning onopgelost, en leveranciersbinding.

Technologiekeuze

metal-stack als geïntegreerde hardware-provisioning-laag, in combinatie met **Gardener** (ADR-C20) voor cluster-lifecycle. De twee componenten vormen samen de volledige provisioning-pipeline van bare-metal hardware naar werkende tenant-cluster.

Switch-hardware moet SONiC-compatibel zijn. Dit is een directe eis aan ADR-G13 (hardware-selectiestrategie) en ADR-G65 (multi-vendor strategie).

Motivatie

De kernvraag is niet “welk provisioning-systeem is het best” maar “kunnen we ons veroorloven om twee losgekoppelde systemen te bouwen en beheren?” Het antwoord is nee. Op de schaal van tienduizenden servers en drie ODC-locaties is een mismatch tussen compute- en netwerk-state een serieus operationeel risico. Elk handmatig koppelvlak tussen twee systemen is een potentieel consistentieprobleem.

metal-stack elimineert dat risico door compute en netwerk in één pipeline samen te brengen.

De hardware-beperking (SONiC-switches) is reëel maar beheersbaar. SONiC is breed ondersteund door meerdere switch-fabrikanten (Edgecore, Dell, Celestica) en sluit aan op de open source-strategie van het platform.

Consequenties

Cascade naar hardware-ADR's: switch-selectie in ADR-G13 en ADR-G65 moet SONiC-compatibiliteit als harde eis meenemen. Vendor-lock-in op switch-niveau wordt beperkt doordat SONiC op hardware van meerdere fabrikanten draait, maar de keuze sluit propriëtaire NOS-oplossingen (Arista EOS, Juniper JunOS als primaire NOS) uit.

Netwerk-fabric-ADR's grotendeels overbodig: ADR-G56 (datacenter network fabric) en ADR-G57 (firewall-architectuur) behandelen deels vraagstukken die metal-stack oplost. Deze ADR's moeten worden herzien om te bepalen wat metal-stack dekt en wat er aanvullend nodig is (perimeter-firewall, DDoS-mitigatie).

Integratie met Gardener: de cluster-lifecycle (ADR-C20) sluit natief aan op metal-stack. De provisioning-pipeline eindigt waar Gardener begint: metal-stack levert de node, Gardener neemt de node op in het tenant-cluster.

ADR-G72 opstellen: de metal-stack-keuze verdient een eigen ADR (G72) die de technische architectuur van de metal-stack-implementatie beschrijft, inclusief de spine-leaf BGP/EVPN-configuratie, VRF-per-tenant-model en de integratie met Gardener.

Declaratief beheer: metal-stack-configuratie wordt beheerd via de bestaande GitOps-pipeline (ADR-C55). Elke hardware-wijziging is een commit.

Audit trail: alle provisioning-acties via metal-stack worden gelogd naar de observability-stack (ADR-G41), conform BIO en NIS2.

Open vragen

1. **Switch-hardware selectie**: welke SONiC-compatibele switches worden ingekocht? Dit raakt ADR-G13 en G65 direct. Moet worden afgestemd met hardware-selectietraject.
2. **Scope G56/G57**: welk deel van de netwerk-fabric-ADR's blijft relevant naast metal-stack? Perimeter-firewall en DDoS-mitigatie vallen buiten de metal-stack-scope en hebben een eigen ADR nodig.

Sessienotities

> [2026-04-20] De keuze hangt af van hoe het programma de afweging maakt tussen integratiegemak en componentvrijheid. De koppeling aan Gardener versus Hypershift maakt dit tot een samenhangende keuze: metal-stack + Gardener, of CAPI/Metal³ + Hypershift.

Overige besluiten

ADR-N01: Service model

Status	Voorgesteld
Laag	Overig
Maakt mogelijk	ADR-G61
Datum	2026-04-30

Context en probleemstelling

Het inzetten van externe operators is een latere strategische fase; de eerste en meest cruciale stap is het definitief vaststellen van het operationele service model. Dit service model fungeert als het primaire leidraad voor onze governance en risicobeperking. Het bepaalt de demarcatie, oftewel de formele afbakening van de verantwoordelijkheden. Deze afbakening is bepalend omdat het direct dicteert of we een 'buy'-strategie moeten volgen (waarbij we de functionaliteit via een bestaand platform of een leverancier ingekochte operator benutten) of een 'build'-strategie (het ontwikkelen van een volledig eigen, unieke oplossing). Deze beslissing over de operationele grenzen moet dus vóórdat we verder ingeterm ineerd op operatoren gaan, om te voorkomen dat we architecturale fricties en governanceproblemen tegenkomen.

Besluit

[Nog geen besluit genomen]

Onderzochte implementaties

Het selectieproces van de dienstverleningsarchitectuur is fundamenteel en vereist een nauwkeurige afweging tussen de drie primaire modellen: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), en volledig Managed Service Delivery. Deze keuze is geen stilistische afweging, maar heeft een directe en significante impact op de benodigde functionaliteiten en de mate van abstractie die het platform – en/of de managed service leverancier – moet ondersteunen. Specifiek dicteert dit model de vereiste beheerniveaus, de architecturale complexiteit en de aard van de geïmplementeerde beveiligingslagen. Daarnaast speelt de selectie van de provider een kritieke rol. Het is momenteel onduidelijk of het traject zal worden beheerd door één enkele, gecontracteerde managed service provider (MSP), of dat men kiest voor een gedecentraliseerd ecosysteem. De keuze tussen een uniforme leverancier en een marktgebaseerd ecosysteem heeft consequenties voor governance, interoperabiliteit en de uiteindelijke flexibiliteit van het gehele systeem

Motivatie

[Nog uit te werken]

Consequenties

[Nog uit te werken]

ADR-N02: Organisatorische inrichting

Status	Brons
Laag	Overig
Datum	2026-06-19

Context en probleemstelling

Hoe gaan we de organisatie verandering te werk stellen. Welke organisatie, gaat dit leveren aan de hele overheid. Zorgen dat het niet een sub onderdeel wordt.

Besluit

[Nog geen besluit genomen]

Motivatie

[Nog uit te werken]

Consequenties

[Nog uit te werken]